

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

**HỆ THỐNG CẢNH BÁO XÂM NHẬP QUA
HÀNH VI LEO RÀO ỨNG DỤNG EDGE AI**

Ngành Công Nghệ Kỹ Thuật Máy Tính

GVHD: TS. VŨ CHÍ CƯỜNG

Sinh viên:

HỒ SỸ NGUYỄN

MSSV : 22119203

LÂM NGUYỄN GIA MINH

MSSV : 22119199

TP. Hồ Chí Minh, tháng 6 năm 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

**HỆ THỐNG CẢNH BÁO XÂM NHẬP QUA
HÀNH VI LEO RÀO ỨNG DỤNG EDGE AI**

Ngành Công Nghệ Kỹ Thuật Máy Tính

GVHD: TS. VŨ CHÍ CƯỜNG

Sinh viên:

HỒ SỸ NGUYỄN

MSSV : 22119203

LÂM NGUYỄN GIA MINH

MSSV : 22119199

TP. Hồ Chí Minh, tháng 6 năm 2026



TP. HCM, ngày 24 tháng 06 năm 2026

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ và tên sinh viên: Lâm Nguyễn Gia Minh MSSV: 22119199
Hồ Sỹ Nguyên MSSV: 22119203
Ngành: Công nghệ kỹ thuật máy tính
Giảng viên hướng dẫn: TS. Vũ Chí Cường
Ngày nhận đề tài: 14/03/2026 Ngày nộp đề tài: 24/06/2026

1. Tên đề tài: Hệ thống cảnh báo xâm nhập qua hành vi leo rào ứng dụng Edge AI

2. Các số liệu, tài liệu ban đầu

- Kiến thức nền tảng về trí tuệ nhân tạo, thị giác máy tính, học sâu và xử lý ảnh...
- Tài liệu về mô hình phát hiện đối tượng YOLOv8, mạng MobileNetV2, Transfer Learning và Fine-tuning.
- Bộ dữ liệu hình ảnh người gồm hai lớp: Normal và Intrusion.
- Tài liệu kỹ thuật về Raspberry Pi 4, webcam USB, buzzer, relay và các mạch điều khiển ngoại vi.
- Các công cụ và thư viện phục vụ huấn luyện, triển khai mô hình gồm Python, PyTorch, OpenCV, ONNX và ONNX Runtime.
- Kiến thức xây dựng Web Dashboard, truyền phát video và điều khiển thiết bị qua GPIO.

3. Nội dung thực hiện đề tài

- Tìm hiểu tổng quan về hệ thống camera giám sát thông minh và các phương pháp phát hiện hành vi xâm nhập bằng trí tuệ nhân tạo.
- Thu thập, xây dựng, gán nhãn và tiền xử lý bộ dữ liệu hình ảnh gồm hai trạng thái Normal và Intrusion.
- Nghiên cứu và tích hợp YOLOv8 để phát hiện người, cắt vùng ảnh chứa người từ khung hình camera.
- Xây dựng, huấn luyện và đánh giá mô hình MobileNetV2 để phân loại hành động bình thường và hành động leo rào.
- Chuyển đổi mô hình sang định dạng ONNX và triển khai pipeline YOLOv8 kết hợp MobileNetV2 trên Raspberry Pi 4
- Xây dựng cơ chế Intrusion Frame Counter nhằm ổn định kết quả nhận diện và hạn chế cảnh báo sai.
- Thiết kế, thi công và tích hợp các khối phần cứng gồm khối nguồn, buzzer, relay, đèn chiếu sáng và nút reset.
- Xây dựng Web Dashboard phục vụ giám sát hình ảnh, theo dõi trạng thái, điều khiển thiết bị và quản lý lịch sử sự kiện.
- Tích hợp toàn bộ phần cứng và phần mềm thành hệ thống phát hiện, cảnh báo xâm nhập hoàn chỉnh.
- Thực nghiệm và đánh giá hệ thống dựa trên độ chính xác, Precision, Recall, F1-score, tốc độ xử lý, độ trễ và khả năng phản hồi phần cứng.
- Hoàn thiện mô hình sản phẩm và báo cáo đồ án tốt nghiệp

GIẢNG VIÊN HƯỚNG DẪN

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: HỆ THỐNG CẢNH BÁO XÂM NHẬP QUA HÀNH VI
LEO RÀO ỨNG DỤNG EDGE AI

Sinh viên: + Lâm Nguyễn Gia Minh

MSSV: 22119199

+ Hồ Sỹ Nguyên

MSSV: 22119203

Hướng dẫn: T.S. Vũ Chí Cường

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

phương pháp hợp lý

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

Kết quả

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

phù hợp

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Hợp lý

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

kỹ năng tập trung

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Đúng

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Đúng

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

Kết quả tìm được phân tích kỹ hơn

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Đúng

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

hướng dẫn đạt yêu cầu của một khóa luận tốt nghiệp và được phép bảo vệ

Tp. HCM, ngày 24 tháng 6 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)

Vũ Chí Cường

* Giáo viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài, chúng em đã nhận được nhiều sự quan tâm, hỗ trợ và động viên từ quý thầy cô, gia đình và bạn bè. Đây là nguồn động lực quan trọng giúp chúng em hoàn thành quá trình nghiên cứu, xây dựng, triển khai và đánh giá hệ thống phát hiện, cảnh báo xâm nhập qua hành động leo rào.

Trước hết, chúng em xin bày tỏ lòng biết ơn sâu sắc đến giảng viên hướng dẫn là thầy Vũ Chí Cường đã tận tình định hướng, góp ý và hỗ trợ chúng em trong suốt quá trình thực hiện đề tài. Những nhận xét và chỉ dẫn của thầy đã giúp chúng em điều chỉnh phương pháp nghiên cứu, hoàn thiện nội dung báo cáo, cũng như nhìn nhận rõ hơn các ưu điểm, hạn chế và hướng phát triển của hệ thống.

Chúng em xin gửi lời cảm ơn chân thành đến quý thầy cô trong khoa đã trang bị cho chúng em những kiến thức nền tảng về lập trình, xử lý ảnh, trí tuệ nhân tạo, hệ thống nhúng và thiết kế phần cứng. Những kiến thức này là cơ sở quan trọng để chúng em có thể tiếp cận bài toán phát hiện hành vi bất thường bằng thị giác máy tính, đồng thời vận dụng vào việc xây dựng một hệ thống hoàn chỉnh có khả năng hoạt động trên thiết bị thực tế.

Chúng cũng xin cảm ơn gia đình và bạn bè đã luôn động viên, tạo điều kiện và hỗ trợ chúng em trong quá trình học tập cũng như thực hiện đề tài. Sự khích lệ đó đã giúp chúng em có thêm tinh thần để vượt qua những khó khăn khi xây dựng mô hình AI, thiết kế phần cứng, tích hợp phần mềm và kiểm thử hệ thống.

Mặc dù đã cố gắng hoàn thiện đề tài một cách nghiêm túc, nhưng do thời gian thực hiện và kinh nghiệm thực tế còn hạn chế, báo cáo không thể tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý của quý thầy cô để đề tài được hoàn thiện hơn trong các hướng nghiên cứu và triển khai tiếp theo.

TÓM TẮT

Đồ án tập trung nghiên cứu, thiết kế và xây dựng hệ thống phát hiện, cảnh báo xâm nhập dựa trên trí tuệ nhân tạo và thị giác máy tính, triển khai trên nền tảng thiết bị nhúng Raspberry Pi 4. Xuất phát từ hạn chế của các hệ thống camera giám sát truyền thống, vốn chủ yếu ghi hình thụ động và phụ thuộc nhiều vào người quan sát, đề tài hướng đến việc xây dựng một hệ thống có khả năng tự động phân tích hình ảnh, nhận diện hành vi bất thường và hỗ trợ cảnh báo trong thời gian gần thực.

Hệ thống được thiết kế theo hướng kết hợp giữa mô hình phát hiện đối tượng và mô hình phân loại hành động. Trong đó, hình ảnh thu nhận từ camera được xử lý để phát hiện sự xuất hiện của người trong khu vực giám sát. Sau đó, vùng ảnh chứa người được tách ra và đưa vào mô hình phân loại nhằm xác định trạng thái bình thường hoặc trạng thái có nguy cơ xâm nhập. Bên cạnh phần mềm xử lý AI, hệ thống còn được tích hợp các khối phần cứng phục vụ cho việc thu nhận hình ảnh, xử lý dữ liệu, phát cảnh báo tại chỗ và hỗ trợ giám sát từ xa thông qua giao diện Web Dashboard.

Quá trình xây dựng hệ thống bao gồm các bước thu thập và tổ chức dữ liệu, huấn luyện mô hình, đánh giá kết quả nhận dạng, chuyển đổi mô hình phục vụ triển khai trên thiết bị nhúng, đồng thời xây dựng chương trình xử lý video theo thời gian gần thực. Các kết quả thực nghiệm cho thấy hệ thống có khả năng phát hiện hành vi xâm nhập trong điều kiện thử nghiệm, đưa ra cảnh báo khi phát hiện tình huống bất thường và hiển thị trạng thái giám sát cho người dùng. Điều này cho thấy tính khả thi của việc ứng dụng các mô hình học sâu nhẹ trên nền tảng phần cứng chi phí thấp cho bài toán giám sát an ninh.

Mặc dù đạt được các kết quả ban đầu khả quan, hệ thống vẫn còn chịu ảnh hưởng bởi một số yếu tố như điều kiện ánh sáng, góc đặt camera, chất lượng hình ảnh đầu vào và giới hạn hiệu năng của thiết bị nhúng. Trong các hướng phát triển tiếp theo, hệ thống có thể được cải thiện bằng cách mở rộng dữ liệu thực tế, tối ưu mô hình nhận dạng, đánh giá trên nhiều kịch bản video khác nhau và kết hợp thêm các cảm biến ngoại vi nhằm nâng cao độ ổn định, độ tin cậy và khả năng ứng dụng trong môi trường thực tế.

Mục lục

LỜI CẢM ƠN	iv
TÓM TẮT	v
DANH MỤC HÌNH ẢNH	x
DANH MỤC BẢNG BIỂU	xii
DANH MỤC TỪ VIẾT TẮT	xiii
1 TỔNG QUAN ĐỀ TÀI	1
1.1 LÝ DO CHỌN ĐỀ TÀI	1
1.2 MỤC TIÊU CỦA ĐỀ TÀI	1
1.2.1 Mục tiêu tổng quát	1
1.2.2 Mục tiêu cụ thể	1
1.3 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU	2
1.3.1 Đối tượng nghiên cứu	2
1.3.2 Phạm vi nghiên cứu	2
1.4 PHƯƠNG PHÁP NGHIÊN CỨU	2
1.5 BỐ CỤC BÁO CÁO	3
2 CƠ SỞ LÝ THUYẾT	4
2.1 TỔNG QUAN HỆ THỐNG GIÁM SÁT THÔNG MINH	4
2.1.1 Camera giám sát truyền thống	4
2.1.2 Camera giám sát ứng dụng AI	4
2.1.3 Bài toán phát hiện hành vi leo rào	5
2.2 CƠ SỞ LÝ THUYẾT VỀ TRÍ TUỆ NHÂN TẠO VÀ HỌC SÂU	6
2.2.1 Trí tuệ nhân tạo và Machine Learning	6
2.2.2 Deep Learning và các hàm kích hoạt	6
2.3 CƠ SỞ LÝ THUYẾT VỀ YOLOV8 TRONG PHÁT HIỆN NGƯỜI	7
2.3.1 Bài toán Object Detection	7
2.3.2 Bounding box và crop vùng người	8
2.3.3 Confidence threshold và IoU threshold	8
2.4 CƠ SỞ LÝ THUYẾT VỀ MOBILENETV2	10
2.4.1 Mạng CNN trong phân loại ảnh	10
2.4.2 Kiến trúc MobileNetV2	10
2.4.3 Transfer Learning	11
2.4.4 Fine-tuning và Partial Freezing	12
2.5 CƠ SỞ LÝ THUYẾT VỀ TIỀN XỬ LÝ ẢNH VÀ TĂNG CƯỜNG DỮ LIỆU	12
2.5.1 Resize, crop và normalize ảnh	12
2.5.2 Data Augmentation	13
2.6 HÀM MẤT MẤT, TỐI ƯU HÓA VÀ LỊCH HỌC	14

2.6.1	Cross Entropy Loss	14
2.6.2	AdamW Optimizer	15
2.6.3	OneCycleLR Scheduler	15
2.7	CƠ SỞ LÝ THUYẾT VỀ ĐÁNH GIÁ MÔ HÌNH	15
2.8	CƠ SỞ LÝ THUYẾT VỀ TRIỂN KHAI MÔ HÌNH TRÊN THIẾT BỊ NHÚNG	17
2.8.1	Edge AI	17
2.8.2	ONNX và ONNX Runtime	17
2.9	THƯ VIỆN OPENCV	18
2.10	GIAO THỨC RTSP	18
2.11	MEDIA MTX	19
2.12	GIAO THỨC HLS	19
2.13	CLOUDFLARE TUNNEL	19
2.14	GIAO THỨC HTTPS	20
3	PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	21
3.1	PHÂN TÍCH YÊU CẦU HỆ THỐNG	21
3.2	KIẾN TRÚC TỔNG THỂ CỦA HỆ THỐNG	22
3.2.1	Sơ đồ khối hệ thống	23
3.2.2	Vai trò của từng khối trong hệ thống	23
3.3	THIẾT KẾ PHẦN CỨNG HỆ THỐNG	24
3.3.1	Thiết kế khối xử lý trung tâm	24
3.3.2	Thiết kế khối thu nhận hình ảnh	25
3.3.3	Thiết kế khối báo động	26
3.3.4	Thiết kế khối reset hệ thống	28
3.3.5	Thiết kế khối relay	29
3.3.6	Thiết kế khối chiếu sáng	31
3.3.7	Thiết kế khối nguồn và board quản lý ngoại vi	33
3.4	THIẾT KẾ PHẦN MỀM HỆ THỐNG	40
3.4.1	Kiến trúc mô hình AI	41
3.4.2	Dữ liệu đầu vào	41
3.4.3	Thiết kế module phát hiện người bằng YOLOv8	41
3.4.4	Thiết kế module crop và padding vùng người	42
3.4.5	Thiết kế mô hình phân loại MobileNetV2	43
3.4.6	Thiết kế module Intrusion Frame Counter	46
3.4.7	Thiết kế khối output	46
3.4.8	Thiết kế giao diện quản lý	46
3.4.9	Thiết kế hệ thống truyền phát video và truy cập từ xa	47
3.5	THIẾT KẾ THUẬT TOÁN HOẠT ĐỘNG CỦA HỆ THỐNG	49
3.5.1	Lưu đồ giải thuật chương trình chính của hệ thống	49
3.5.2	Lưu đồ giải thuật xử lý AI phát hiện xâm nhập	50
3.5.3	Lưu đồ giải thuật module ghi hình 24/7 và tự động dọn dẹp bộ nhớ	52
3.5.4	Lưu đồ giải thuật vòng đời xử lý HTTP Request của Web Server	52
4	XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG	54
4.1	MÔI TRƯỜNG XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG	54
4.1.1	Môi trường phát triển phần mềm	54
4.1.2	Môi trường triển khai trên thiết bị nhúng	54
4.1.3	Thiết bị phần cứng sử dụng	55

4.2	XÂY DỰNG CHƯƠNG TRÌNH HUẤN LUYỆN MÔ HÌNH MOBILENETV2	55
4.2.1	Chuẩn bị bộ dữ liệu huấn luyện	55
4.2.2	Tiền xử lý và tăng cường dữ liệu	56
4.2.3	Xây dựng kiến trúc MobileNetV2	57
4.2.4	Cấu hình huấn luyện	57
4.2.5	Chiến lược fine-tuning và partial freezing	58
4.2.6	Đánh giá và lưu mô hình tốt nhất	59
4.3	CHUYỂN ĐỔI VÀ TỐI ƯU MÔ HÌNH PHỤC VỤ TRIỂN KHAI	59
4.3.1	Xuất mô hình từ PyTorch sang ONNX	59
4.3.2	So sánh mô hình ONNX Float32 và INT8	60
4.3.3	Chuẩn bị file triển khai	60
4.4	XÂY DỰNG CHƯƠNG TRÌNH NHẬN DIỆN VÀ CẢNH BÁO	61
4.4.1	Khởi tạo hệ thống	61
4.4.2	Đọc dữ liệu từ camera	62
4.4.3	Phát hiện người bằng YOLOv8	62
4.4.4	Crop vùng người và tiền xử lý ảnh	62
4.4.5	Phân loại hành động bằng MobileNetV2	62
4.4.6	Kích hoạt cảnh báo	63
4.4.7	Hiển thị kết quả lên web giám sát	63
4.5	XÂY DỰNG GIAO DIỆN WEB GIÁM SÁT VÀ ĐIỀU KHIỂN	63
4.5.1	Trang giao diện đăng nhập	63
4.5.2	Chức năng hiển thị trạng thái hệ thống	64
4.5.3	Chức năng điều khiển từ xa	65
4.5.4	Chức năng lưu trữ và xem lại dữ liệu	65
5	KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	67
5.1	MỤC TIÊU VÀ NỘI DUNG THỰC NGHIỆM	67
5.2	MÔI TRƯỜNG DỮ LIỆU VÀ THỰC NGHIỆM	67
5.2.1	Cấu hình phần cứng thực nghiệm	67
5.2.2	Dữ liệu huấn luyện, validation và test	68
5.2.3	Dữ liệu kiểm thử và kịch bản kiểm thử	68
5.3	KẾT QUẢ HUẤN LUYỆN MÔ HÌNH MOBILENETV2	69
5.3.1	Kết quả loss và accuracy	69
5.3.2	Precision, Recall và F1-score	69
5.3.3	Phân tích confusion matrix	70
5.4	KẾT QUẢ TEST MÔ HÌNH	71
5.5	SO SÁNH VÀ ĐÁNH GIÁ VỚI CÁC NGHIÊN CỨU LIÊN QUAN	72
5.5.1	So sánh độ chính xác nhận dạng	73
5.5.2	Đánh giá khả năng triển khai Edge AI	73
5.5.3	Nhận xét chung	74
5.5.4	Đánh giá vai trò của Intrusion Frame Counter	74
5.6	KẾT QUẢ TRÊN GIAO DIỆN WEB GIÁM SÁT	75
5.6.1	Dashboard ở trạng thái bình thường	75
5.6.2	Dashboard khi phát hiện intrusion	75
5.6.3	Lịch sử sự kiện	76
5.6.4	Lịch sử sự kiện	77
5.7	KẾT QUẢ PHẢN HỒI PHẦN CỨNG VÀ ĐỘ TRỄ	77

5.7.1	Phản hồi ở trạng thái normal và intrusion	77
5.7.2	Độ trễ end-to-end	78
5.8	ĐÁNH GIÁ TỐC ĐỘ XỬ LÝ VÀ TÀI NGUYÊN	78
5.8.1	Tốc độ inference trên Raspberry Pi 4	78
5.8.2	Đánh giá phiên bản Float32	78
5.9	ĐÁNH GIÁ CHUNG	79
5.9.1	Ưu điểm	79
5.9.2	Hạn chế	79
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	80
6.1	KẾT LUẬN	80
6.2	HƯỚNG PHÁT TRIỂN	80
	TÀI LIỆU THAM KHẢO	81
	PHỤ LỤC A. MÃ NGUỒN VÀ DỮ LIỆU PHỤC VỤ HUẤN LUYỆN, KIỂM THỬ MÔ HÌNH AI	83
	PHỤ LỤC B. TRIỂN KHAI HỆ THỐNG	84

Danh sách hình vẽ

2.1	Sơ sánh tư thế lớp Normal và lớp Intrusion nhìn từ góc camera giám sát	5
2.2	Sơ đồ phân cấp lĩnh vực Trí tuệ nhân tạo	6
2.3	Confusion Matrix cho bài toán phân loại nhị phân	16
3.1	Sơ đồ đặc tả kỹ thuật của hệ thống giám sát xâm nhập ứng dụng trí tuệ nhân tạo	22
3.2	Sơ đồ khối của hệ thống	23
3.3	Hình ảnh thực tế và sơ đồ chân của Raspberry Pi 4	24
3.4	Sơ đồ bo mạch Raspberry Pi 4	25
3.5	Webcam USB Full HD	25
3.6	Sơ đồ kết nối camera USB với Raspberry Pi 4	26
3.7	Buzzer 1209 sử dụng trong khối báo động	27
3.8	Sơ đồ kết nối Raspberry Pi 4 với còi báo động	27
3.9	hình ảnh nút nhấn Tactile Switch B3F-4055	28
3.10	Sơ đồ mạch nút nhấn reset hệ thống kết nối raspberry pi 4	29
3.11	Hình ảnh thực tế cấu kiện Rơ-le điện cơ Omron G5NB-1A-E-5VDC sử dụng trong hệ thống.	30
3.12	Sơ đồ nguyên lý mạch kích Relay của hệ thống	31
3.13	Đèn pha LED 30W ngoài trời sử dụng trong khối chiếu sáng	32
3.14	Sơ đồ kết nối Raspberry Pi 4 với khối chiếu sáng thông qua relay	33
3.15	Module nguồn AC-DC HLK-20M05 sử dụng trong khối nguồn	34
3.16	Sơ đồ mạch tăng bảo vệ và lọc nhiễu đầu vào AC-DC HLK-20M05 sử dụng trong khối nguồn	35
3.17	Sơ đồ mạch lọc phối hợp làm mịn điện áp ngõ ra 5VDC	36
3.18	Sơ đồ nguyên lý tổng thể board quản lý nguồn và giao tiếp ngoại vi	37
3.19	Sơ đồ nguyên lý tổng thể board quản lý nguồn và giao tiếp ngoại vi	37
3.20	Bản vẽ thiết kế đường mạch Layout 2D của bo mạch trên phần mềm	39
3.21	Sơ đồ khối kiến trúc mô hình AI của hệ thống	41
3.22	Mình hoạ tác dụng của padding bounding box.	43
3.23	Kiến trúc classifier head của MobileNetV2 trong hệ thống.	45
3.24	Sơ đồ khối luồng truyền phát video của hệ thống	47
3.25	Sơ đồ khối luồng giao diện Web Dashboard của hệ thống	48
3.26	Lưu đồ giải thuật chương trình chính của hệ thống	49
3.27	Lưu đồ giải thuật xử lý AI phát hiện xâm nhập của hệ thống	51
3.28	Lưu đồ giải thuật mô-đun ghi hình 24/7 và tự động dọn dẹp bộ nhớ	52
3.29	Lưu đồ giải thuật vòng đời xử lý HTTP Request của Web Server	53
4.1	Mình họa các kỹ thuật data augmentation được áp dụng	57
4.2	Các file cần thiết khi triển khai mô hình lên Raspberry Pi 4	61
4.3	Giao diện trang đăng nhập	64
4.4	Giao diện Web giám sát trung tâm	64
4.5	Giao diện điều khiển từ xa	65

4.6	Giao diện lưu trữ và xem lại ảnh sự kiện xâm nhập	66
4.7	Giao diện lưu trữ và xem lại video giám sát	66
5.1	Hệ thống phần cứng đã lắp đặt thực tế.	68
5.2	Train loss, validation loss, train accuracy và validation accuracy theo epoch.	69
5.3	Precision, Recall và F1-score trung bình theo từng epoch.	69
5.4	Tổng hợp các chỉ số đánh giá của best model.	70
5.5	Confusion matrix của best model trên tập validation.	70
5.6	Ma trận nhầm lẫn của mô hình trên tập test: (a) số lượng mẫu, (b) tỷ lệ chuẩn hóa	71
5.7	Tổng hợp các chỉ số đánh giá mô hình trên tập test	72
5.8	Web Dashboard ở trạng thái giám sát bình thường.	75
5.9	Web Dashboard khi hệ thống xác nhận trạng thái intrusion.	75
5.10	Giao diện lịch sử ảnh sự kiện intrusion.	76
5.11	Giao diện lưu trữ và xem lại video sự kiện	77

Danh sách bảng

2.1	Các kỹ thuật Data Augmentation phổ biến thường được sử dụng	14
2.2	Các chỉ số đánh giá mô hình phân loại nhị phân	16
3.1	Thông số kỹ thuật của Raspberry Pi 4 Model B	24
3.2	Thông số kỹ thuật của webcam USB	26
3.3	Thông số kỹ thuật của buzzer điện tử	27
3.4	Thông số kỹ thuật của nút nhấn B3F-4055	29
3.5	Thông số kỹ thuật của rơ-le điện tử Omron G5NB-1A-E-5VDC	30
3.6	Thông số kỹ thuật linh kiện phụ trợ	31
3.7	Thông số kỹ thuật của đèn LED 30W	32
3.8	Bảng ước tính dòng tiêu thụ của các thành phần trong hệ thống	34
3.9	Thông số kỹ thuật của bộ nguồn 5VDC 4A	34
3.10	Thông số cấu hình lựa chọn của các linh kiện kỹ thuật	35
3.11	Thông số cấu hình lựa chọn của các linh kiện điện tử	36
4.1	Môi trường phần mềm và phần cứng của hệ thống	54
4.2	Danh sách thiết bị và linh kiện chính được sử dụng	55
4.3	Các siêu tham số huấn luyện mô hình MobileNetV2	58
4.4	So sánh mô hình MobileNetV2 ONNX Float32 và INT8	60
4.5	Ảnh xạ tín hiệu giữa chương trình và phần cứng	63
5.1	Phân bố dữ liệu huấn luyện, validation và test.	68
5.2	So sánh kết quả test của đề tài với các nghiên cứu liên quan.	73
5.3	Độ trễ end-to-end đo được của hệ thống.	78
5.4	Tốc độ xử lý của các mô hình trên Raspberry Pi 4	78

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Tên đầy đủ
1	AI	Artificial Intelligence
2	Edge AI	Edge Artificial Intelligence
3	CCTV	Closed-Circuit Television
4	CNN	Convolutional Neural Network
5	YOLO	You Only Look Once
6	YOLOv8	You Only Look Once version 8
7	MobileNetV2	Mobile Network Version 2
8	ONNX	Open Neural Network Exchange
9	ONNX Runtime	Open Neural Network Exchange Runtime
10	INT8	8-bit Integer
11	Float32	32-bit Floating Point
12	GPIO	General Purpose Input/Output
13	USB	Universal Serial Bus
14	FPS	Frames Per Second
15	CPU	Central Processing Unit
16	GPU	Graphics Processing Unit
17	RAM	Random Access Memory
18	OpenCV	Open Source Computer Vision Library
19	PyTorch	PyTorch Deep Learning Framework
20	Torchvision	PyTorch Computer Vision Library
21	ImageNet	ImageNet Dataset
22	AdamW	Adaptive Moment Estimation with Decoupled Weight Decay
23	OneCycleLR	One Cycle Learning Rate Scheduler
24	IoU	Intersection over Union
25	NMS	Non-Maximum Suppression
26	TP	True Positive
27	TN	True Negative
28	FP	False Positive
29	FN	False Negative
30	FNR	False Negative Rate
31	mAP	mean Average Precision
32	HLS	HTTP Live Streaming
33	HTTP	HyperText Transfer Protocol
34	HTML	HyperText Markup Language
35	CSS	Cascading Style Sheets
36	JSON	JavaScript Object Notation
37	API	Application Programming Interface
38	Flask	Flask Web Framework
39	FFmpeg	Fast Forward MPEG
40	LED	Light Emitting Diode
41	AC	Alternating Current
42	DC	Direct Current
43	PCB	Printed Circuit Board
44	MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
45	EMI	Electromagnetic Interference
46	RTSP	Real-Time Streaming Protocol
47	RFC	Request for Comments
48	IETF	Internet Engineering Task Force
49	RTP	Real-time Transport Protocol
50	TCP	Transmission Control Protocol

51	IP	Internet Protocol
52	MediaMTX	MediaMTX Media Server
53	RTMP	Real-Time Messaging Protocol
54	WebRTC	Web Real-Time Communication
55	SRT	Secure Reliable Transport
56	MPEG-TS	Moving Picture Experts Group Transport Stream
57	CDN	Content Delivery Network
58	M3U8	M3U8 Playlist File
59	Cloudflare Tunnel	Cloudflare Tunnel Service
60	NAT	Network Address Translation
61	WAF	Web Application Firewall
62	HTTPS	HyperText Transfer Protocol Secure
63	TLS	Transport Layer Security
64	MITM	Man-in-the-Middle
65	CUDA	Compute Unified Device Architecture
66	OpenCL	Open Computing Language
67	IoT	Internet of Things
68	F1-score	F1 Score
69	YOLOv8n	You Only Look Once version 8 nano
70	RGB	Red Green Blue
71	BGR	Blue Green Red
72	ReLU	Rectified Linear Unit
73	ReLU6	Rectified Linear Unit 6
74	BN	Batch Normalization
75	ARM	Advanced RISC Machine
76	VDC	Volts Direct Current
77	VAC	Volts Alternating Current
78	GND	Ground
79	SSR	Solid State Relay
80	NPN	Negative-Positive-Negative Transistor
81	USB-C	Universal Serial Bus Type-C

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. LÝ DO CHỌN ĐỀ TÀI

Trong bối cảnh xã hội hiện đại, nhu cầu đảm bảo an ninh tại các công trình dân dụng và công nghiệp ngày càng trở nên cấp thiết. Các khu vực như nhà ở, trường học, nhà xưởng, kho bãi, khu vực quân sự hay căn cứ hạ tầng trọng yếu đều đòi hỏi một hệ thống giám sát có khả năng phát hiện và phản ứng kịp thời trước các hành vi xâm nhập trái phép.

Tuy nhiên, các hệ thống camera giám sát truyền thống hiện nay chủ yếu dừng lại ở vai trò ghi hình thụ động. Toàn bộ trách nhiệm phát hiện bất thường vẫn phụ thuộc vào con người theo dõi màn hình liên tục một công việc đòi hỏi sự tập trung cao độ, dễ xảy ra bỏ sót khi khối lượng camera lớn, khi người vận hành mệt mỏi hoặc khi sự kiện diễn ra ngoài giờ làm việc. Thực tiễn cho thấy phần lớn các vụ xâm nhập chỉ được phát hiện sau khi sự việc đã xảy ra, thông qua việc truy xuất lại băng ghi hình lúc này hậu quả đã không thể tránh khỏi.

Trước thực trạng đó, sự phát triển của trí tuệ nhân tạo và thị giác máy tính mở ra một hướng tiếp cận mới có giá trị thực tiễn cao. Thay vì chờ con người quan sát, hệ thống có thể tự động phân tích từng khung hình từ camera, nhận diện hành vi bất thường và phát cảnh báo ngay lập tức mà không cần sự can thiệp thủ công. Đặc biệt, hành động leo rào một trong những phương thức xâm nhập phổ biến nhất có thể được mô hình hóa và nhận diện tự động thông qua phân tích tư thế và hành động của người trong ảnh.

Từ những lý do trên, đề tài hướng đến việc xây dựng một hệ thống có khả năng nhận diện hành động leo rào từ luồng video camera và phát cảnh báo kịp thời, ứng dụng các kỹ thuật học sâu hiện đại trên nền tảng thiết bị nhúng chi phí thấp, phù hợp với yêu cầu triển khai thực tế.

1.2. MỤC TIÊU CỦA ĐỀ TÀI

1.2.1. Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống phát hiện và cảnh báo xâm nhập dựa trên hành động leo hàng rào, sử dụng camera để thu nhận hình ảnh và mô hình trí tuệ nhân tạo để phân tích hành vi. Hệ thống được thiết kế theo định hướng AI Edge, trong đó quá trình xử lý và ra quyết định được thực hiện trực tiếp trên thiết bị biên nhằm đáp ứng yêu cầu xử lý gần thời gian thực, giảm độ trễ và nâng cao tính chủ động trong hoạt động giám sát an ninh. Bên cạnh đó, hệ thống tích hợp giao diện Web Dashboard cho phép người dùng theo dõi hình ảnh và trạng thái hoạt động của hệ thống từ xa thông qua mạng Internet, góp phần nâng cao khả năng quản lý và giám sát hệ thống một cách thuận tiện.

1.2.2. Mục tiêu cụ thể

Để đạt được mục tiêu tổng quát nêu trên, đề tài đặt ra các mục tiêu cụ thể như sau:

- Thứ 1. Thiết kế hệ thống camera giám sát khu vực hàng rào với góc nhìn phù hợp để quan sát toàn bộ vị trí có nguy cơ xâm nhập.
- Thứ 2. Xây dựng mô hình AI nhận diện hai trạng thái hành động của người: Normal (hành động bình thường, không có dấu hiệu xâm nhập) và Intrusion (hành động leo rào, có khả năng là hành vi xâm nhập).
- Thứ 3. Tích hợp YOLOv8 làm mô hình phát hiện người trong khung hình, cung cấp vùng ảnh crop chứa

người làm đầu vào cho giai đoạn phân loại hành động.

Thứ 4. Ứng dụng MobileNetV2 làm mô hình phân loại hành động dựa trên ảnh người đã được cắt (crop), bảo đảm mô hình có kích thước nhỏ, tốc độ suy luận nhanh và phù hợp để triển khai trên các thiết bị AI Edge có tài nguyên tính toán hạn chế.

Thứ 5. Thiết kế và tích hợp hệ thống phần cứng – phần mềm, xây dựng giao diện Web Dashboard phục vụ giám sát và quản lý hệ thống từ xa thông qua mạng Internet.

Thứ 6. Triển khai, thực nghiệm và đánh giá hiệu năng của hệ thống trong điều kiện thực tế về độ chính xác, độ trễ xử lý và tính ổn định trong quá trình giám sát.

1.3. ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

1.3.1. Đối tượng nghiên cứu

Đề tài tập trung nghiên cứu các đối tượng sau:

Về hành vi con người: hành động bình thường của người trong khu vực giám sát (đi lại, đứng, quan sát...) và hành động leo rào hoặc vượt qua hàng rào có dấu hiệu xâm nhập. Đây là hai lớp hành động cốt lõi mà hệ thống cần phân biệt.

Về mô hình AI: mô hình phát hiện người YOLOv8 và mô hình phân loại hành động MobileNetV2 bao gồm kiến trúc, cơ chế hoạt động, quy trình huấn luyện, fine-tuning và tối ưu hóa cho triển khai nhúng.

Về phần cứng: hệ thống gồm Raspberry Pi 4, webcam USB 1080p, mô-đun cảnh báo bằng còi, cùng đèn chiếu sáng hỗ trợ giám sát trong điều kiện thiếu sáng.

1.3.2. Phạm vi nghiên cứu

Đề tài được giới hạn trong các phạm vi cụ thể như sau:

Về bài toán: hệ thống chỉ tập trung phát hiện hành vi leo rào là dấu hiệu xâm nhập. Bài toán phân loại gồm hai lớp: Normal và Intrusion.

Về dữ liệu: dữ liệu đầu vào là video hoặc luồng hình ảnh từ camera. Hệ thống sử dụng ảnh đã được crop vùng người trước, không phân tích toàn bộ khung cảnh gốc. Tập dữ liệu huấn luyện được xây dựng tổng hợp với đa dạng tư thế, môi trường và điều kiện ánh sáng.

Về mô hình: hệ thống sử dụng pipeline hai giai đoạn YOLOv8 phát hiện người trước, MobileNetV2 phân loại hành động sau. Mô hình tập trung nhận diện người từ trước, không thực hiện nhận diện khuôn mặt hay định danh cá nhân.

Về hành vi: hệ thống chưa tập trung xử lý các hành vi phức tạp khác như phá khóa, cắt rào hay bò qua rào những hành vi này có thể được mở rộng trong các nghiên cứu tiếp theo.

1.4. PHƯƠNG PHÁP NGHIÊN CỨU

Đề tài được thực hiện theo hướng kết hợp giữa nghiên cứu lý thuyết và thực nghiệm, cụ thể như sau:

Về nghiên cứu lý thuyết: tìm hiểu và tổng hợp các cơ sở lý thuyết liên quan đến thị giác máy tính, object detection và image classification; nghiên cứu kiến trúc YOLOv8 và MobileNetV2, bao gồm nguyên lý hoạt động, ưu điểm và giới hạn của từng mô hình trong bài toán phân loại hành động.

Về xây dựng dữ liệu: xây dựng tập dữ liệu ảnh gồm hai lớp Normal và Intrusion, đảm bảo tính đa dạng về tư thế, góc camera, môi trường nền và điều kiện ánh sáng; thực hiện phân chia tập train/validation theo nguyên tắc không chồng lấp để đảm bảo đánh giá khách quan.

Về huấn luyện và tối ưu hóa mô hình: huấn luyện MobileNetV2 trên tập dữ liệu đã chuẩn bị với chiến lược Transfer Learning và Fine-tuning; áp dụng Data Augmentation, Intrusion Frame Counter, AdamW và OneCycleLR Scheduler để tối ưu hóa quá trình học; export mô hình sang ONNX để triển khai hiệu quả trên Raspberry Pi.

Về kiểm thử và đánh giá: tích hợp YOLOv8 với MobileNetV2 trong chương trình kiểm thử trên video thực tế; đánh giá hệ thống thông qua các chỉ số định lượng gồm Accuracy, Precision, Recall, F1-score, Confusion Matrix và tốc độ xử lý FPS thực tế trên Raspberry Pi.

1.5. BỐ CỤC BÁO CÁO

Báo cáo được tổ chức thành 6 chương với nội dung như sau:

Chương 1 - Tổng quan đề tài: trình bày lý do chọn đề tài, mục tiêu nghiên cứu, đối tượng và phạm vi nghiên cứu, phương pháp thực hiện.

Chương 2 - Cơ sở lý thuyết: trình bày các nền tảng lý thuyết liên quan gồm bài toán phát hiện xâm nhập bằng thị giác máy tính, pipeline AI hai giai đoạn, kiến trúc YOLOv8 và MobileNetV2, kỹ thuật Transfer Learning và Fine-tuning, tiền xử lý ảnh và Data Augmentation, hàm mất mát và thuật toán tối ưu hóa, các chỉ số đánh giá mô hình, và cơ sở lý thuyết về triển khai Edge AI với ONNX

Chương 3 - Phân tích và thiết kế hệ thống: trình bày yêu cầu chức năng của hệ thống, kiến trúc tổng thể phần mềm, thiết kế các module thành phần bao gồm module phát hiện người, module phân loại hành động và module cảnh báo.

Chương 4 - Xây dựng và triển khai hệ thống: trình bày môi trường phát triển, quy trình xây dựng chương trình huấn luyện mô hình MobileNetV2, quy trình export và tối ưu hóa mô hình sang ONNX FLOAT32, xây dựng chương trình kiểm thử trên video và triển khai thực tế trên Raspberry Pi.

Chương 5 - Kết quả thực nghiệm và đánh giá: trình bày dữ liệu thực nghiệm, kết quả huấn luyện MobileNetV2 (biểu đồ loss/accuracy, Precision/Recall/F1, Confusion Matrix), kết quả kiểm thử trên video thực tế, đánh giá tốc độ xử lý, phân tích ưu điểm và hạn chế của hệ thống.

Chương 6 - Kết luận và hướng phát triển trình bày kết luận cuối cùng của đề tài và nêu ra hướng phát triển cho hệ thống trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. TỔNG QUAN HỆ THỐNG GIÁM SÁT THÔNG MINH

Hệ thống giám sát an ninh đã trải qua quá trình phát triển từ các giải pháp thủ công hoàn toàn phụ thuộc vào con người, sang các hệ thống camera ghi hình thụ động, và hiện nay tiến đến thế hệ giám sát thông minh ứng dụng trí tuệ nhân tạo. Sự chuyển dịch này không chỉ nâng cao hiệu quả phát hiện mà còn mở ra khả năng tự động hóa toàn bộ quy trình giám sát, giảm thiểu sự phụ thuộc vào nguồn nhân lực và khắc phục các hạn chế vốn có của phương pháp truyền thống.

2.1.1. Camera giám sát truyền thống

Camera giám sát truyền thống (Closed-Circuit Television — CCTV) là hệ thống thu hình và ghi lại hình ảnh liên tục tại khu vực cần giám sát. Dữ liệu video được lưu trữ trên đầu ghi (DVR/NVR) và có thể được truy xuất khi cần thiết để xem lại sự kiện đã xảy ra.

Về nguyên lý hoạt động, hệ thống camera truyền thống chỉ đảm nhiệm vai trò ghi hình thụ động. Mọi quyết định liên quan đến phát hiện bất thường, nhận diện sự kiện hay phát cảnh báo đều phụ thuộc hoàn toàn vào người vận hành theo dõi màn hình trực tiếp. Khi số lượng camera lớn điều phổ biến tại các nhà máy, trường học hay khu công nghiệp một người vận hành phải đồng thời theo dõi nhiều luồng hình ảnh khác nhau, dẫn đến giới hạn nghiêm trọng về khả năng chú ý và tốc độ phản ứng.

Hệ thống camera truyền thống bộc lộ ba hạn chế cốt lõi trong bài toán giám sát an ninh:

- **Thứ nhất, sự phụ thuộc hoàn toàn vào con người:** chất lượng giám sát dao động theo trạng thái tâm lý và thể chất của người vận hành một môi, mất tập trung hay vắng mặt đều có thể dẫn đến bỏ sót sự kiện quan trọng.
- **Thứ hai, không có khả năng phản ứng thời gian thực:** ngay cả khi người vận hành phát hiện sự kiện bất thường, thời gian từ lúc nhận ra đến lúc đưa ra phản hồi vẫn tốn thêm nhiều giây quý giá.
- **Thứ ba, giá trị chủ yếu ở truy xuất sau sự kiện:** phần lớn thời gian, footage từ camera chỉ được xem lại sau khi sự việc đã xảy ra lúc này camera đóng vai trò bằng chứng hơn là công cụ ngăn chặn.

2.1.2. Camera giám sát ứng dụng AI

Sự phát triển của học sâu (Deep Learning) và thị giác máy tính đã tạo ra thế hệ hệ thống giám sát thông minh nơi camera không còn chỉ ghi hình mà còn “hiểu” được nội dung hình ảnh theo thời gian thực. Hệ thống giám sát ứng dụng AI có khả năng tự động phân tích từng khung hình, phát hiện đối tượng, nhận diện hành vi và đưa ra cảnh báo mà không cần sự can thiệp của con người trong vòng lặp xử lý.

Kiến trúc tổng quát của một hệ thống camera giám sát AI bao gồm các lớp chức năng:

- **Lớp thu nhận dữ liệu:** camera thu hình và cung cấp luồng video liên tục. Dữ liệu hình ảnh được đưa vào pipeline xử lý theo từng khung hình hoặc theo nhóm khung hình tùy yêu cầu về độ trễ.
- **Lớp phát hiện đối tượng:** mô hình Object Detection YOLOv8 xác định vị trí và lớp của các đối tượng trong khung hình ví dụ phát hiện người, xe, động vật. Đây là bước tiền xử lý quan trọng giúp thu hẹp vùng phân tích cho các bước tiếp theo.
- **Lớp phân tích hành vi:** từ các đối tượng đã được phát hiện, mô hình phân loại hành động (Action Classification) hoặc nhận diện hành vi (Behavior Recognition) phân tích tư thế, chuyển động và ngữ

cảnh để xác định hành vi đang diễn ra có bình thường hay không.

- **Lớp ra quyết định và cảnh báo:** dựa trên kết quả phân tích, hệ thống đưa ra quyết định và kích hoạt cảnh báo tương ứng âm thanh, đèn, thông báo qua ứng dụng hoặc tín hiệu đến trung tâm giám sát.

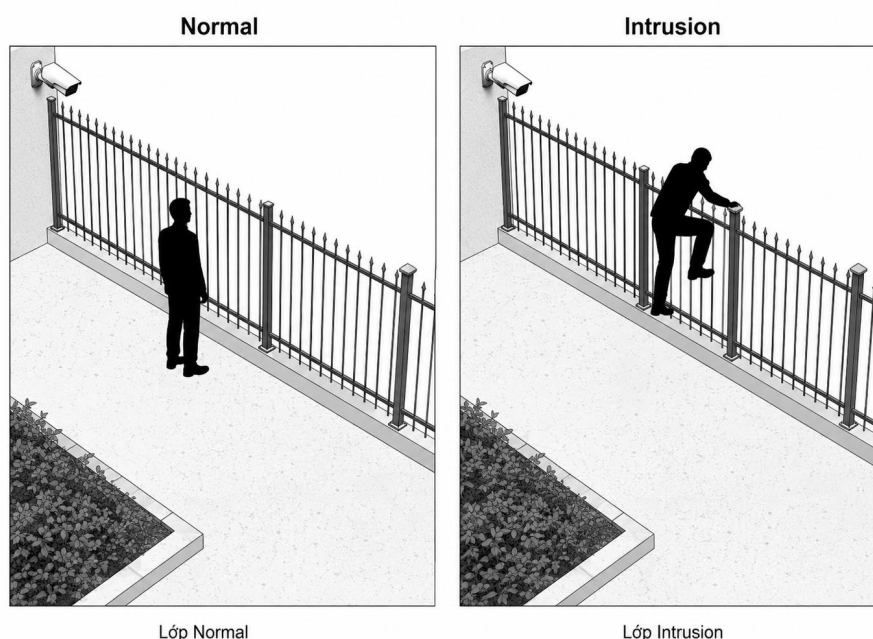
So với hệ thống truyền thống, giám sát ứng dụng AI mang lại những cải tiến đáng kể: khả năng phân tích đồng thời nhiều luồng camera mà không bị giới hạn bởi sự chú ý của con người; tốc độ phát hiện nhanh trong phạm vi vài frame (dưới 1 giây) so với hàng chục giây khi phụ thuộc vào người quan sát; hoạt động liên tục 24/7 với hiệu suất ổn định bất kể thời gian hay điều kiện ánh sáng; và khả năng phân biệt các hành vi cụ thể thay vì chỉ ghi nhận sự hiện diện của đối tượng.[1]

2.1.3. Bài toán phát hiện hành vi leo rào

Trong số các hành vi cần giám sát tại khu vực hàng rào và tường rào, hành động leo rào được xác định là dấu hiệu xâm nhập quan trọng nhất và cũng là bài toán thị giác máy tính có tính đặc thù riêng.

Leo rào là hành động một người tác động vật lý lên hàng rào hoặc tường bảo vệ nhằm vượt qua ranh giới được kiểm soát mà không đi qua lối vào hợp lệ. Hành động này thường diễn ra chậm và có chủ đích, với các đặc trưng tư thế thị giác nhận dạng được. Những đặc trưng cơ thể nghiêng về phía hàng rào, tay bám vào cấu trúc hàng rào, trọng tâm cơ thể dịch chuyển lên cao và chân rời khỏi mặt đất. Những đặc trưng này tạo nên sự khác biệt rõ ràng về mặt thị giác so với các hành động bình thường như đi bộ, đứng hay ngồi gần hàng rào.

Tuy nhiên, bài toán phát hiện hành vi leo rào không đơn giản như phân biệt hai trạng thái hoàn toàn khác nhau. Thách thức chính nằm ở sự tồn tại của các trường hợp ranh giới (hard negative cases) những hành động bình thường có tư thế bề ngoài gần giống leo rào nhưng thực chất không phải xâm nhập, ví dụ: người đứng sát hàng rào và giơ tay lên cao, người sửa chữa hàng rào từ bên trong, người nhặt vật rơi gần chân hàng rào hay trẻ em leo thang gần tường. Một hệ thống phân loại kém có thể nhầm các trường hợp này thành intrusion, gây ra tỉ lệ cảnh báo sai cao và làm giảm độ tin cậy của hệ thống.[12], [13]



Hình 2.1. So sánh tư thế lớp Normal và lớp Intrusion nhìn từ góc camera giám sát

Chính vì vậy, bài toán được định nghĩa là bài toán phân loại nhị phân với hai lớp:

- **Normal:** hành động bình thường, không có dấu hiệu xâm nhập. Lớp này bao gồm cả các hard negative cases

những tư thế gần giống leo rào nhưng không phải nhằm giúp mô hình học được ranh giới quyết định tinh tế hơn.

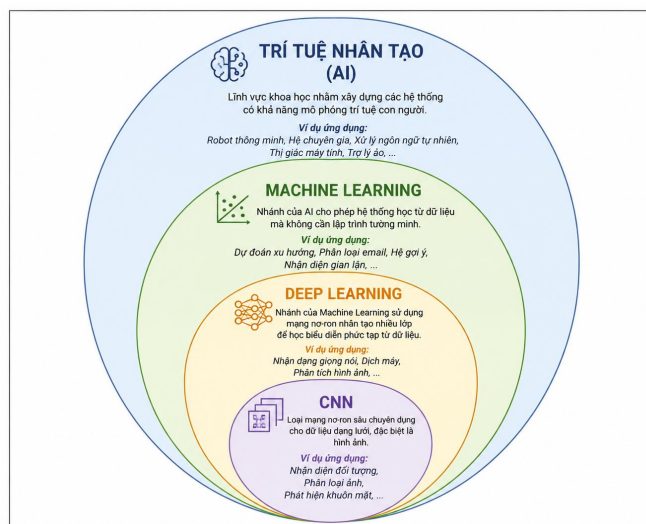
- **Intrusion:** hành động leo rào có chủ đích, thể hiện ý định vượt qua ranh giới bảo vệ.

Việc xác định rõ ràng hai lớp này và đặc biệt chú trọng đến thiết kế tập dữ liệu cho lớp *Normal* là yếu tố quyết định đến khả năng hệ thống hoạt động đáng tin cậy trong thực tế, nơi các tình huống ranh giới xảy ra thường xuyên hơn các trường hợp lý tưởng trong phòng thí nghiệm.

2.2. CƠ SỞ LÝ THUYẾT VỀ TRÍ TUỆ NHÂN TẠO VÀ HỌC SÂU

2.2.1. Trí tuệ nhân tạo và Machine Learning

Trí tuệ nhân tạo (Artificial Intelligence — AI) là lĩnh vực khoa học máy tính nghiên cứu các hệ thống có khả năng thực hiện những nhiệm vụ đòi hỏi trí thông minh như nhận thức, suy luận và ra quyết định, trong đó Machine Learning (học máy) là nhánh con cho phép hệ thống tự học từ dữ liệu để xây dựng mô hình dự đoán thay vì được lập trình bằng các quy tắc cứng nhắc. Toàn bộ các hệ thống AI thực tế hiện nay, bao gồm cả hệ thống trong đề tài, đều thuộc nhóm AI hẹp (Narrow AI) — chỉ thực hiện tốt một nhiệm vụ cụ thể trong một miền xác định, khác với AI tổng quát (General AI) vẫn còn là mục tiêu nghiên cứu dài hạn chưa đạt được. Trong lĩnh vực giám sát thông minh, AI đóng vai trò chuyển đổi hệ thống camera từ công cụ ghi hình thụ động thành hệ thống phân tích chủ động, có khả năng tự động nhận diện đối tượng, phân loại hành động và đưa ra cảnh báo mà không cần sự can thiệp liên tục của con người. Đối với các bài toán xử lý ảnh có độ phức tạp cao như phát hiện người hay phân loại hành động, các phương pháp Machine Learning truyền thống thường gặp khó khăn do cần kỹ thuật trích xuất đặc trưng thủ công, đây chính là động lực thúc đẩy sự phát triển của Deep Learning như một giải pháp tự động học đặc trưng từ dữ liệu thô.[19], [20]



Hình 2.2. Sơ đồ phân cấp lĩnh vực Trí tuệ nhân tạo

2.2.2. Deep Learning và các hàm kích hoạt

Deep Learning (học sâu) là nhánh đặc biệt của Machine Learning sử dụng mạng nơ-ron nhân tạo nhiều lớp để tự động học các biểu diễn đặc trưng phân cấp từ dữ liệu thô, không cần giai đoạn trích xuất đặc trưng thủ công như Machine Learning truyền thống. Tại mỗi nơ-ron, sau khi tổ hợp tuyến tính giữa đầu vào và trọng số, một hàm kích hoạt phi tuyến σ được áp dụng nhằm tăng khả năng biểu diễn của mạng. Các hàm kích hoạt phổ biến trong Deep Learning gồm:

$$\text{ReLU}(z) = \max(0, z) \quad (2.1)$$

$$\text{ReLU6}(z) = \min(\max(0, z), 6) \quad (2.2)$$

$$\text{Sigmoid}(z) = \frac{1}{1 + e^{-z}} \quad (2.3)$$

$$\text{Softmax}(z_k) = \frac{e^{z_k}}{\sum_j e^{z_j}} \quad (2.4)$$

Trong đó ReLU thường được sử dụng trong các mạng nơ-ron sâu do tính đơn giản và hiệu quả tính toán cao. ReLU6 là biến thể giới hạn đầu ra trong khoảng từ 0 đến 6, được sử dụng trong MobileNetV2 nhằm hỗ trợ tốt hơn cho việc triển khai trên thiết bị có tài nguyên hạn chế. Sigmoid thường dùng cho các bài toán phân loại nhị phân, trong khi Softmax thường được dùng ở lớp đầu ra của bài toán phân loại nhiều lớp.[9]

Nhờ khả năng học đặc trưng phân cấp tự động từ cấp thấp (cạnh, biên) đến cấp trung (góc, bộ phận cơ thể) và cấp cao (tư thế toàn thân), Deep Learning đặc biệt phù hợp với bài toán nhận diện hành vi từ ảnh, nơi các đặc trưng như tư thế leo rào khó mô tả đầy đủ bằng quy tắc thủ công. Trong đề tài, hai kiến trúc Deep Learning được ứng dụng gồm YOLOv8, dùng cho bài toán phát hiện người theo thời gian thực, và MobileNetV2, mạng CNN nhẹ dùng để phân loại hành động thành hai trạng thái bình thường hoặc có nguy cơ xâm nhập sau khi vùng ảnh chứa người được cắt ra từ kết quả của YOLOv8.[8], [20]

2.3. CƠ SỞ LÝ THUYẾT VỀ YOLOV8 TRONG PHÁT HIỆN NGƯỜI

2.3.1. Bài toán Object Detection

Object Detection (phát hiện đối tượng) là một trong những bài toán nền tảng của thị giác máy tính, với nhiệm vụ đồng thời giải quyết hai câu hỏi: đối tượng trong ảnh thuộc lớp nào và vị trí của nó ở đâu. Đây là bài toán phức tạp hơn so với Image Classification thuần túy, bởi mô hình không chỉ cần nhận diện đối tượng mà còn phải xác định vị trí không gian của từng đối tượng đó trong ảnh đầu vào.

Về mặt hình thức, cho một ảnh đầu vào I có kích thước $W \times H$, bài toán Object Detection yêu cầu mô hình dự đoán một tập hợp các phát hiện:

$$\mathcal{D}_{det} = \{ (b_i, c_i, s_i) \mid i = 1, 2, \dots, N \} \quad (2.5)$$

Trong đó:

- $b_i = (x_{1i}, y_{1i}, x_{2i}, y_{2i})$ là bounding box của đối tượng thứ i ;
- c_i là nhãn lớp của đối tượng thứ i ;
- $s_i \in [0, 1]$ là điểm tin cậy, phản ánh mức độ chắc chắn của mô hình đối với dự đoán;
- N là tổng số đối tượng được phát hiện trong ảnh.

Kết quả đầu ra của mô hình Object Detection gồm ba thành phần chính: tọa độ bounding box xác định vùng chứa đối tượng, nhãn lớp cho biết đối tượng thuộc lớp nào và điểm tin cậy đánh giá mức độ chắc chắn của dự đoán. Trong hệ thống của đề tài, YOLOv8 chỉ cần phát hiện một lớp duy nhất là person, nhờ đó mô hình tập trung vào nhiệm vụ định vị người trong khung hình camera giám sát. Sau khi người được phát hiện, vùng ảnh chứa người sẽ được cắt ra và đưa sang mô hình MobileNetV2 để phân loại hành động bình thường hoặc có nguy cơ xâm nhập.[21]

YOLOv8 (*You Only Look Once version 8*) là một kiến trúc thuộc dòng YOLO, được phát triển bởi Ultralytics. Điểm đặc trưng của các mô hình YOLO là xử lý toàn bộ ảnh đầu vào chỉ trong một lần lan truyền

tiến qua mạng nơ-ron. Cách tiếp cận này khác với các phương pháp phát hiện đối tượng hai giai đoạn, vốn cần bước đề xuất vùng ảnh trước khi phân loại và hiệu chỉnh vị trí đối tượng.

Nhờ cơ chế xử lý một giai đoạn, YOLOv8 có tốc độ inference nhanh và phù hợp với các ứng dụng yêu cầu xử lý gần thời gian thực. Đối với hệ thống giám sát xâm nhập, đặc điểm này rất quan trọng vì camera liên tục cung cấp các khung hình mới. Nếu mô hình phát hiện người xử lý quá chậm, toàn bộ pipeline sẽ bị tăng độ trễ, làm giảm khả năng cảnh báo kịp thời khi xuất hiện hành vi bất thường.[2]

2.3.2. Bounding box và crop vùng người

Bounding box là một hình chữ nhật được xác định bởi bốn tọa độ trong hệ tọa độ ảnh, dùng để biểu diễn vùng không gian chứa đối tượng được phát hiện. Trong hệ thống này, bounding box được biểu diễn theo định dạng hai điểm góc:

$$\text{Bounding box} = (x_1, y_1, x_2, y_2) \quad (2.6)$$

Trong đó:

- (x_1, y_1) : tọa độ góc trên bên trái của bounding box,
- (x_2, y_2) : tọa độ góc dưới bên phải của bounding box,
- Chiều rộng: $W_{\text{box}} = x_2 - x_1$,
- Chiều cao: $H_{\text{box}} = y_2 - y_1$.

Diện tích vùng bounding box được tính theo công thức:

$$A_{\text{box}} = (x_2 - x_1)(y_2 - y_1) \quad (2.7)$$

Sau khi YOLOv8 xác định được bounding box của người trong khung hình, hệ thống thực hiện thao tác crop tức là cắt vùng ảnh pixel tương ứng với tọa độ bounding box ra khỏi ảnh gốc. Trong thực tế triển khai, để tránh trường hợp YOLOv8 crop quá sát khiến mất đi một phần tay hoặc chân của người đặc biệt quan trọng khi phân tích tư thế leo rào hệ thống áp dụng thêm một lượng padding quanh bounding box gốc:

$$x'_1 = \max(0, x_1 - \text{pad}) \quad (2.8)$$

$$y'_1 = \max(0, y_1 - \text{pad}) \quad (2.9)$$

$$x'_2 = \min(W_{\text{frame}}, x_2 + \text{pad}) \quad (2.10)$$

$$y'_2 = \min(H_{\text{frame}}, y_2 + \text{pad}) \quad (2.11)$$

Trong đó pad là giá trị mở rộng tính bằng pixel; W_{frame} và H_{frame} lần lượt là chiều rộng và chiều cao của khung hình gốc. Việc giới hạn tọa độ trong phạm vi $[0, W_{\text{frame}}]$ và $[0, H_{\text{frame}}]$ đảm bảo bounding box mở rộng không vượt ra ngoài biên ảnh. Vùng crop thu được sau đó được resize về kích thước chuẩn và đưa vào MobileNetV2 để phân loại hành động.[2]

2.3.3. Confidence threshold và IoU threshold

Trong quá trình phát hiện đối tượng, một ảnh đầu vào thường sinh ra rất nhiều bounding box ứng viên từ các ô lưới (*grid cell*) khác nhau của mô hình. Để lọc ra các phát hiện có chất lượng, hai ngưỡng quan trọng được sử dụng là Confidence threshold và IoU threshold.

2.3.3.1. Confidence threshold

Confidence score là giá trị đo lường mức độ chắc chắn của mô hình khi cho rằng một bounding box chứa đối tượng thuộc lớp cụ thể. Giá trị này được định nghĩa như sau:

$$s = P(\text{object}) \times \text{IoU}(\text{pred}, \text{ground_truth}) \quad (2.12)$$

Trong đó $P(\text{object})$ là xác suất có đối tượng trong vùng bounding box, và $\text{IoU}(\text{pred}, \text{ground_truth})$ là độ trùng lặp giữa bounding box dự đoán và bounding box thực tế. Confidence score nhận giá trị trong khoảng $[0, 1]$, giá trị càng cao nghĩa là mô hình càng chắc chắn về phát hiện đó.

Confidence threshold (ký hiệu CONF_THRESH) là ngưỡng tối thiểu mà một bounding box phải đạt được để được chấp nhận:

$$\text{Chấp nhận detection khi: } s \geq \text{CONF_THRESH} \quad (2.13)$$

Trong hệ thống này, giá trị được sử dụng là CONF_THRESH = 0,4, nghĩa là chỉ những bounding box có confidence score từ 0,4 trở lên mới được giữ lại để xử lý tiếp. Việc đặt ngưỡng quá thấp sẽ dẫn đến nhiều phát hiện sai (*false positive*), trong khi ngưỡng quá cao có thể bỏ sót người trong điều kiện ánh sáng yếu hoặc góc camera khó. Giá trị 0,4 là sự cân bằng phù hợp cho môi trường giám sát thực tế.[2]

2.3.3.2. IoU threshold và Non-Maximum Suppression

Intersection over Union (IoU) là chỉ số đo lường mức độ chồng lấp giữa hai bounding box. Cho hai bounding box A và B , IoU được tính theo công thức:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (2.14)$$

Trong đó:

- $|A \cap B|$ là diện tích phần giao nhau (intersection) giữa hai bounding box,
- $|A \cup B|$ là diện tích phần hợp (union) của hai bounding box,
- IoU nhận giá trị trong khoảng $[0, 1]$: giá trị 0 nghĩa là hai bounding box hoàn toàn không chồng lấp, giá trị 1 nghĩa là hai bounding box trùng khít hoàn toàn.

Diện tích giao nhau giữa hai bounding box $A = (x1_A, y1_A, x2_A, y2_A)$ và $B = (x1_B, y1_B, x2_B, y2_B)$ được tính như sau:

$$x_{\text{inter1}} = \max(x1_A, x1_B) \quad (2.15)$$

$$y_{\text{inter1}} = \max(y1_A, y1_B) \quad (2.16)$$

$$x_{\text{inter2}} = \min(x2_A, x2_B) \quad (2.17)$$

$$y_{\text{inter2}} = \min(y2_A, y2_B) \quad (2.18)$$

$$|A \cap B| = \max(0, x_{\text{inter2}} - x_{\text{inter1}}) \times \max(0, y_{\text{inter2}} - y_{\text{inter1}}) \quad (2.19)$$

$$|A \cup B| = A_A + A_B - |A \cap B| \quad (2.20)$$

Trong đó A_A và A_B lần lượt là diện tích của bounding box A và B .

IoU threshold (ký hiệu IOU_THRESH) được sử dụng trong thuật toán Non-Maximum Suppression (NMS) một bước hậu xử lý quan trọng nhằm loại bỏ các bounding box trùng lặp. Trong hệ thống này, IOU_THRESH = 0,45, có nghĩa là nếu hai bounding box có IoU vượt quá 0,45, chỉ bounding box có confidence score cao hơn được

giữ lại, bounding box còn lại bị loại bỏ. Cơ chế này ngăn chặn tình trạng một người bị phát hiện nhiều lần bởi các bounding box chồng lấp nhau, đảm bảo mỗi người trong khung hình chỉ được crop và phân loại một lần duy nhất.

2.4. CƠ SỞ LÝ THUYẾT VỀ MOBILENETV2

2.4.1. Mạng CNN trong phân loại ảnh

Mạng nơ-ron tích chập (Convolutional Neural Network — CNN) là kiến trúc học sâu được sử dụng phổ biến nhất trong các bài toán xử lý ảnh, bao gồm phân loại ảnh, phát hiện đối tượng và phân đoạn ngữ nghĩa. Khác với mạng nơ-ron truyền thống kết nối đầy đủ (fully connected), CNN khai thác cấu trúc không gian của ảnh thông qua các phép tích chập cục bộ giữa ảnh đầu vào và một bộ lọc (filter/kernel), cho phép học các đặc trưng thị giác có tính phân cấp từ thấp đến cao. Sau mỗi lớp tích chập, lớp Batch Normalization được áp dụng để chuẩn hóa phân phối đầu ra theo từng mini-batch, giúp giảm hiện tượng Internal Covariate Shift, cho phép sử dụng learning rate lớn hơn và quá trình huấn luyện hội tụ ổn định hơn; tiếp theo, hàm kích hoạt phi tuyến ReLU được áp dụng để tăng khả năng biểu diễn của mô hình.

Nhờ cơ chế xếp chồng nhiều lớp tích chập, CNN có khả năng học đặc trưng theo cấu trúc phân cấp: các lớp đầu học các đặc trưng cấp thấp như cạnh, góc và gradient màu sắc; các lớp giữa kết hợp thành các mẫu phức tạp hơn như bộ phận cơ thể người; các lớp cuối tổng hợp thành đặc trưng cấp cao phản ánh ngữ nghĩa hành động, chẳng hạn như tư thế leo rào với cơ thể nghiêng và tay bám vào hàng rào. Chính cơ chế học đặc trưng phân cấp này làm cho CNN trở thành lựa chọn phù hợp cho bài toán phân loại hành động dựa trên ảnh crop người.[8]

2.4.2. Kiến trúc MobileNetV2

MobileNetV2 là kiến trúc CNN nhẹ được Google giới thiệu năm 2018, được thiết kế đặc biệt cho các ứng dụng trên thiết bị di động và nhúng với tài nguyên tính toán hạn chế. So với các mạng CNN lớn như VGG16 hay ResNet50, MobileNetV2 có số lượng tham số ít hơn nhiều lần trong khi vẫn đạt được độ chính xác cạnh tranh trên các tập dữ liệu chuẩn.[9]

2.4.2.1. Depthwise Separable Convolution

Nền tảng của MobileNetV2 là phép tích chập tách biệt theo chiều sâu (Depthwise Separable Convolution), được phân tách thành hai bước:

Bước 1 — Depthwise Convolution: áp dụng một bộ lọc riêng biệt cho từng kênh đầu vào độc lập:

$$Y_{dw}[i, j, c] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X[i \cdot s + m, j \cdot s + n, c] \cdot K_{dw}[m, n, c] \quad (2.21)$$

Bước 2 — Pointwise Convolution (tích chập 1×1): kết hợp thông tin từ tất cả các kênh:

$$Y_{pw}[i, j, c_{out}] = \sum_{c=0}^{C_{in}-1} Y_{dw}[i, j, c] \cdot K_{pw}[c, c_{out}] + b[c_{out}] \quad (2.22)$$

Chi phí tính toán so sánh giữa tích chập tiêu chuẩn và Depthwise Separable Convolution:

$$\begin{aligned} \text{Tiêu chuẩn: } & k^2 \cdot C_{in} \cdot C_{out} \cdot H_{out} \cdot W_{out} \\ \text{DS Conv: } & k^2 \cdot C_{in} \cdot H_{out} \cdot W_{out} + C_{in} \cdot C_{out} \cdot H_{out} \cdot W_{out} \\ \text{Tỉ lệ giảm: } & \frac{1}{C_{out}} + \frac{1}{k^2} \end{aligned} \quad (2.23)$$

Với kernel $k = 3$ và C_{out} lớn (thường 64–320), DS Conv giảm chi phí tính toán xuống còn khoảng 8–9

lần so với tích chập tiêu chuẩn.

2.4.2.2. *Inverted Residual Block với Linear Bottleneck*

Thành phần cốt lõi của MobileNetV2 là khối Inverted Residual (còn gọi là Bottleneck Block), bao gồm ba bước:

Bước 1 — Expansion:

$$X_{\text{exp}} = \text{BN}(\text{Conv}_{1 \times 1}(X, C_{\text{in}} \cdot t)) \xrightarrow{\text{ReLU6}}$$

Bước 2 — Depthwise Conv 3×3 :

$$X_{\text{dw}} = \text{BN}(\text{DWConv}_{3 \times 3}(X_{\text{exp}})) \xrightarrow{\text{ReLU6}} \quad (2.24)$$

Bước 3 — Projection (Linear):

$$X_{\text{out}} = \text{BN}(\text{Conv}_{1 \times 1}(X_{\text{dw}}, C_{\text{out}})) \quad (\text{không ReLU})$$

Kết nối tắt (shortcut connection):

$$Y = \begin{cases} X_{\text{out}} + X & \text{khi } s = 1 \text{ và } C_{\text{in}} = C_{\text{out}} \\ X_{\text{out}} & \text{khi } s = 2 \text{ hoặc } C_{\text{in}} \neq C_{\text{out}} \end{cases} \quad (2.25)$$

Lý do không dùng ReLU ở bước projection (Linear Bottleneck): các tác giả MobileNetV2 chứng minh rằng khi số kênh nhỏ, hàm kích hoạt phi tuyến có thể làm mất thông tin do đặt nhiều giá trị âm về 0. Việc giữ tuyến tính ở lớp chiếu giúp bảo toàn thông tin trong không gian đặc trưng chiều thấp.

2.4.2.3. *Cấu trúc tổng thể MobileNetV2*

Kiến trúc MobileNetV2 đầy đủ bao gồm: một lớp tích chập đầu vào với stride = 2, tiếp theo là 17 khối Inverted Residual với hệ số mở rộng $t = 6$, và cuối cùng là một lớp tích chập 1×1 nâng số kênh lên 1280 trước khi qua Global Average Pooling và lớp phân loại. Toàn bộ mạng có khoảng 3,4 triệu tham số, nhỏ hơn nhiều lần so với VGG16 (≈ 138 triệu) hay ResNet50 (≈ 25 triệu), trong khi đạt độ chính xác top-1 khoảng 72% trên ImageNet.

2.4.3. Transfer Learning

Transfer Learning (học chuyển giao) là kỹ thuật tận dụng kiến thức đã học từ một bài toán nguồn (source task) để giải quyết một bài toán đích (target task) có liên quan. Trong thị giác máy tính, Transfer Learning thường được thực hiện bằng cách sử dụng mô hình đã được huấn luyện trước (*pretrained model*) trên tập dữ liệu lớn như ImageNet làm điểm khởi đầu, thay vì huấn luyện mô hình từ đầu với trọng số ngẫu nhiên.

2.4.3.1. *Nguyên lý của Transfer Learning*

Về mặt lý thuyết, các lớp tích chập đầu của mạng CNN pretrained đã học được các đặc trưng thị giác tổng quát như cạnh, góc, texture, màu sắc và hình dạng cơ bản. Đây là những đặc trưng có giá trị cho nhiều bài toán nhận diện hình ảnh khác nhau. Vì vậy, khi chuyển sang một bài toán mới, mô hình có thể tái sử dụng các đặc trưng đã học thay vì phải học lại hoàn toàn từ đầu.

Ý tưởng tổng quát của Transfer Learning có thể biểu diễn như sau:

$$\theta_T = \text{Transfer}(\theta_S, \mathcal{D}_T) \quad (2.26)$$

Trong đó θ_S là tập tham số đã học từ bài toán nguồn, $\mathcal{D}_T$ là dữ liệu của bài toán đích và θ_T là tập tham số

sau khi được điều chỉnh cho bài toán đích.[22]

2.4.3.2. Fine-tuning mô hình pretrained

Trong thực tế, Transfer Learning thường được thực hiện bằng cách giữ lại phần backbone của mô hình pretrained để trích xuất đặc trưng, sau đó thay đổi lớp phân loại cuối cho phù hợp với số lớp của bài toán mới. Quá trình tiếp tục huấn luyện mô hình trên dữ liệu đích được gọi là fine-tuning.

Bài toán tối ưu trong giai đoạn fine-tuning có thể viết như sau:

$$\theta_T^* = \arg \min_{\theta_T} \frac{1}{N_T} \sum_{i=1}^{N_T} \mathcal{L}(f_{\theta_T}(\mathbf{x}_i^T), y_i^T) \quad (2.27)$$

Trong đó N_T là số lượng mẫu trong tập dữ liệu đích, $\mathbf{x}_i^T$ là dữ liệu đầu vào, y_i^T là nhãn tương ứng, f_{θ_T} là mô hình sau khi chuyển giao và $\mathcal{L}$ là hàm mất mát.

So với huấn luyện từ đầu, Transfer Learning giúp mô hình hội tụ nhanh hơn, giảm yêu cầu về dữ liệu huấn luyện và hạn chế nguy cơ overfitting khi tập dữ liệu đích có kích thước không quá lớn. Tuy nhiên, hiệu quả của kỹ thuật này phụ thuộc vào mức độ liên quan giữa bài toán nguồn và bài toán đích. Nếu hai bài toán quá khác nhau, kiến thức chuyển giao có thể không phù hợp và làm giảm hiệu quả của mô hình.[3]

2.4.4. Fine-tuning và Partial Freezing

Fine-tuning là kỹ thuật điều chỉnh một phần hoặc toàn bộ trọng số của mô hình pretrained để thích nghi với tập dữ liệu và bài toán đích. Khác với Transfer Learning thuần túy chỉ huấn luyện lớp classifier mới, Fine-tuning cho phép các lớp backbone cũng được cập nhật nhưng với learning rate nhỏ hơn và chỉ sau một giai đoạn warm-up nhất định.[22]

Partial Freezing là chiến lược chỉ đóng băng một phần các lớp đầu của backbone trong giai đoạn đầu huấn luyện. Về mặt toán học, đối với tham số θ_i thuộc lớp đóng băng:

$$\Delta\theta_i = \begin{cases} 0 & \text{nếu lớp } i \text{ bị đóng băng} \\ -\eta \frac{\partial L}{\partial \theta_i} & \text{nếu lớp } i \text{ được mở băng} \end{cases} \quad (2.28)$$

Trong đó η là learning rate và L là hàm mất mát. Quá trình huấn luyện gồm hai giai đoạn:

- **Giai đoạn 1 (Frozen backbone):** Phần lớn backbone bị đóng băng; chỉ classifier head được cập nhật. Mục đích là để head mới ổn định trước, tránh gradient lớn làm hỏng pretrained weights của backbone.
- **Giai đoạn 2 (Unfreezing):** Backbone được mở băng hoàn toàn; cả backbone lẫn head đều được cập nhật với learning rate nhỏ hơn theo chiến lược Differential Learning Rate được trình bày chi tiết ở mục 2.6.2.3, cho phép tinh chỉnh đặc trưng phù hợp với ảnh leo rào từ camera giám sát.

2.5. CƠ SỞ LÝ THUYẾT VỀ TIỀN XỬ LÝ ẢNH VÀ TĂNG CƯỜNG DỮ LIỆU

2.5.1. Resize, crop và normalize ảnh

Trước khi đưa vào mô hình học sâu, ảnh đầu vào cần được chuẩn hóa qua một chuỗi các bước tiền xử lý nhằm đảm bảo tính nhất quán về kích thước, định dạng và phân phối giá trị pixel. Đây là bước bắt buộc vì các mô hình CNN yêu cầu đầu vào có kích thước cố định, đồng thời việc chuẩn hóa giá trị pixel giúp quá trình tối ưu hóa

hội tụ nhanh và ổn định hơn.

2.5.1.1. *Resize ảnh*

Resize là bước đưa ảnh về kích thước cố định mà mô hình yêu cầu. Cho ảnh đầu vào có kích thước $W_{in} \times H_{in}$, sau khi resize về kích thước đích $W_{out} \times H_{out}$, tỉ lệ biến đổi theo hai chiều là:

$$s_w = \frac{W_{out}}{W_{in}}, \quad s_h = \frac{H_{out}}{H_{in}} \quad (2.29)$$

Trong thực tế, kích thước ảnh đầu vào thường được lựa chọn tùy theo kiến trúc mô hình và yêu cầu tính toán của hệ thống:

- **Kích thước lớn:** giúp giữ lại nhiều chi tiết hình ảnh hơn, hỗ trợ mô hình học các đặc trưng nhỏ hoặc tư thế phức tạp.
- **Kích thước nhỏ:** giúp giảm số phép tính, tăng tốc độ inference và phù hợp hơn với các thiết bị có tài nguyên hạn chế.

2.5.1.2. *Normalize ảnh*

Normalize là bước biến đổi giá trị pixel từ khoảng $[0, 255]$ về phân phối có trung bình gần 0 và độ lệch chuẩn gần 1. Công thức normalize theo từng kênh màu:

$$x_{norm} = \frac{x - \mu}{\sigma} \quad (2.30)$$

Trong đó x là giá trị pixel gốc sau khi đã chuyển về khoảng $[0, 255]$ bằng cách chia cho 255; còn μ và σ là bộ tham số chuẩn thường được lấy từ tập dữ liệu dùng để huấn luyện trước mô hình:

$$\begin{aligned} \mu &= [0,485; 0,456; 0,406] \quad (\text{trung bình RGB}) \\ \sigma &= [0,229; 0,224; 0,225] \quad (\text{độ lệch chuẩn RGB}) \end{aligned} \quad (2.31)$$

Việc normalize theo cùng bộ tham số với giai đoạn pretrain giúp phân phối đầu vào không bị lệch so với điều kiện mà mô hình đã học, từ đó giúp mô hình pretrained hoạt động ổn định hơn khi áp dụng Transfer Learning hoặc fine-tuning.[9]

2.5.1.3. *Chuyển ảnh sang tensor*

Trong các mô hình học sâu, ảnh đầu vào cần được chuyển sang dạng tensor để có thể được xử lý bởi mô hình. Ban đầu, ảnh thường được biểu diễn dưới dạng ma trận ba chiều $H \times W \times C$, trong đó H là chiều cao, W là chiều rộng và C là số kênh màu. Khi sử dụng PyTorch, dữ liệu ảnh thường được sắp xếp theo định dạng $C \times H \times W$, tức là số kênh màu được đặt ở chiều đầu tiên.

Việc chuyển ảnh sang tensor giúp dữ liệu có cấu trúc phù hợp với đầu vào của mạng nơ-ron tích chập. Sau khi chuyển đổi, nhiều ảnh có thể được ghép lại thành một batch để phục vụ quá trình huấn luyện hoặc suy luận của mô hình.[4]

2.5.2. Data Augmentation

Data Augmentation (tăng cường dữ liệu) là kỹ thuật mở rộng tập dữ liệu huấn luyện bằng cách áp dụng các phép biến đổi ngẫu nhiên lên ảnh gốc, giúp mô hình học được các đặc trưng bất biến (*invariant*) với các biến thể thực tế của dữ liệu chẳng hạn như thay đổi góc nhìn, điều kiện ánh sáng, khoảng cách hay nhiễu hình ảnh mà

không cần thu thập thêm dữ liệu thực tế. Đây là kỹ thuật đặc biệt quan trọng trong các bài toán thị giác máy tính, nơi dữ liệu thực tế thường khan hiếm và điều kiện thu hình đa dạng, khó kiểm soát.[23]

Trong đề tài, các kỹ thuật tăng cường dữ liệu chuẩn của thư viện torchvision được kết hợp áp dụng trong quá trình huấn luyện, được tóm tắt trong **Bảng 2.1**.

Bảng 2.1. Các kỹ thuật Data Augmentation phổ biến thường được sử dụng

Kỹ thuật	Mục đích
RandomResizedCrop	Cắt ngẫu nhiên một vùng ảnh với tỉ lệ diện tích và khung hình thay đổi, sau đó resize về kích thước đích, mô phỏng sự thay đổi khoảng cách giữa camera và đối tượng.
RandomHorizontalFlip	Lật ảnh ngẫu nhiên theo trục dọc với một xác suất cho trước, giúp mô hình không phụ thuộc vào một hướng xuất hiện cố định của đối tượng.
ColorJitter	Biến đổi ngẫu nhiên độ sáng, độ tương phản, độ bão hòa và sắc độ màu của ảnh, giúp mô hình thích nghi tốt hơn với điều kiện ánh sáng và chất lượng ảnh khác nhau trong thực tế.
RandomRotation	Xoay ảnh ngẫu nhiên một góc nhỏ lấy mẫu từ phân phối đều, mô phỏng sự thay đổi góc đặt camera hoặc tư thế đối tượng.
RandomPerspective	Áp dụng phép biến đổi phối cảnh ngẫu nhiên, mô phỏng sự thay đổi góc nhìn của camera trong môi trường thực tế.
GaussianBlur	Làm mờ ảnh bằng bộ lọc Gaussian, mô phỏng hiện tượng ảnh bị mất nét do camera lấy nét không chính xác hoặc đối tượng di chuyển nhanh.

Việc kết hợp đồng thời nhiều kỹ thuật tăng cường dữ liệu nêu trên tạo ra một không gian biến đổi phong phú, giúp mô hình học được các đặc trưng bất biến với nhiều điều kiện thực tế khác nhau, từ đó giảm thiểu hiện tượng overfitting và nâng cao khả năng tổng quát hóa khi triển khai trong môi trường thực tế.

2.6. HÀM MẤT MÁT, TỐI ƯU HÓA VÀ LỊCH HỌC

2.6.1. Cross Entropy Loss

Cross Entropy Loss (hàm mất mát entropy chéo) là hàm mất mát được sử dụng phổ biến nhất cho các bài toán phân loại, đo lường sự khác biệt giữa phân phối xác suất dự đoán của mô hình (thông qua hàm Softmax) và phân phối xác suất thực tế, hay còn gọi là nhãn ground truth. Với bài toán phân loại K lớp, vector logits $\mathbf{z} = [z_1, \dots, z_K]$ và nhãn thực tế y ở dạng one-hot, hàm mất mát cho một mẫu được tính như sau:

$$\mathcal{L}_{CE} = -\log p_y, \quad p_y = \frac{\exp(z_y)}{\sum_{j=1}^K \exp(z_j)} \quad (2.32)$$

Trong đó p_y là xác suất dự đoán cho lớp đúng theo Softmax. Cross Entropy Loss có thể áp dụng linh hoạt cho cả phân loại nhị phân và phân loại đa lớp, giá trị hàm mất mát càng nhỏ đồng nghĩa dự đoán của mô hình càng gần với nhãn thực tế, do đó hàm này được sử dụng rộng rãi trong các mô hình học sâu cho bài toán phân loại ảnh.[21]

2.6.2. AdamW Optimizer

AdamW (Adam with decoupled Weight Decay) là thuật toán tối ưu hóa phổ biến trong huấn luyện mô hình học sâu, kết hợp momentum bậc nhất và bậc hai của Adam với cơ chế weight decay được tách riêng khỏi quá trình cập nhật gradient thay vì ghép trực tiếp vào gradient như Adam gốc. Công thức cập nhật tham số của AdamW tại bước t như sau:

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} - \underbrace{\eta \lambda \theta_{t-1}}_{\text{weight decay thuần túy}} \quad (2.33)$$

Trong đó $\hat{m}_t$ và $\hat{v}_t$ lần lượt là ước lượng moment bậc nhất và bậc hai của gradient đã hiệu chỉnh bias, η là learning rate và λ là hệ số weight decay. Việc tách weight decay ra khỏi $\hat{v}_t$ giúp chính quy hóa hiệu quả hơn so với Adam gốc, đặc biệt có lợi khi fine-tuning các mô hình pretrained lớn như MobileNetV2 trong đề tài.

Differential Learning Rate và Gradient Clipping

Trong quá trình fine-tuning, các nhóm tham số được cấu hình với learning rate khác nhau:

- Backbone ở các lớp thấp: $\eta_1 = \eta_{\text{base}} \times \text{factor_low}$
- Backbone ở các lớp cao: $\eta_2 = \eta_{\text{base}} \times \text{factor_mid}$
- Classifier head: $\eta_3 = \eta_{\text{base}}$

Chiến lược này kết hợp với gradient clipping — giới hạn norm của gradient toàn cục để tránh bùng nổ gradient:

$$g_{\text{clipped}} = g \cdot \min\left(1, \frac{\text{max_norm}}{\|g\|_2}\right) \quad (2.34)$$

Gradient clipping đặc biệt quan trọng trong giai đoạn unfreezing khi gradient từ nhiều nhóm tham số có thể có độ lớn không đồng đều.[5]

2.6.3. OneCycleLR Scheduler

OneCycleLR là lịch học (learning rate scheduler) điều chỉnh learning rate theo một chu kỳ duy nhất trong suốt quá trình huấn luyện: tăng tuyến tính từ η_{min} lên η_{max} trong $p \cdot T$ bước đầu (warmup), sau đó giảm theo hàm cosine (annealing) trong $(1 - p) \cdot T$ bước còn lại:

$$\eta(t) = \eta_{\text{final}} + (\eta_{\text{max}} - \eta_{\text{final}}) \cdot \frac{1 + \cos(\pi t' / T')}{2}, \quad t' = t - pT, \quad T' = (1 - p)T \quad (2.35)$$

Giai đoạn warmup giúp ổn định quá trình huấn luyện khi classifier head chưa được điều chỉnh, tránh gradient lớn làm hỏng trọng số pretrained của backbone; trong khi giai đoạn cosine annealing giúp learning rate giảm mượt mà hơn so với giảm bậc thang (step decay), tránh các bước nhảy đột ngột gây bất ổn. Sự kết hợp giữa AdamW và OneCycleLR là cấu hình được sử dụng rộng rãi trong các bài toán fine-tuning mô hình pretrained, đã được kiểm chứng giúp mô hình hội tụ tốt với số epoch huấn luyện ít hơn so với các lịch học truyền thống.[24]

2.7. CƠ SỞ LÝ THUYẾT VỀ ĐÁNH GIÁ MÔ HÌNH

Việc đánh giá mô hình phân loại không thể chỉ dựa vào một chỉ số duy nhất. Trong bài toán phân loại nhị phân, cần sử dụng một bộ chỉ số đánh giá bổ trợ lẫn nhau để phản ánh toàn diện hiệu năng của mô hình, dựa trên nền tảng là ma trận nhầm lẫn (Confusion Matrix). Quy ước: lớp Positive là lớp hành vi cần phát hiện (intrusion), lớp Negative là lớp còn lại (normal). Bốn trường hợp kết quả dự đoán được định nghĩa: **True Positive (TP)** và

True Negative (TN) là các dự đoán đúng; **False Positive (FP)** là dự đoán sai thành Positive trong khi thực tế là Negative; **False Negative (FN)** là dự đoán sai thành Negative trong khi thực tế là Positive. Tổng số mẫu trong tập đánh giá:

$$N = TP + TN + FP + FN \quad (2.36)$$

Từ bốn giá trị TP, TN, FP, FN, các chỉ số đánh giá Accuracy, Precision, Recall và F1-score được dẫn xuất như tóm tắt trong **Bảng 2.2**.

Bảng 2.2. Các chỉ số đánh giá mô hình phân loại nhị phân

Chỉ số	Công thức	Ý nghĩa
Accuracy	$\frac{TP + TN}{N}$	Tỉ lệ dự đoán đúng tổng thể; có thể gây hiểu lầm khi dữ liệu hai lớp mất cân bằng.
Precision	$\frac{TP}{TP + FP}$	Trong các mẫu dự đoán Positive, tỉ lệ thực sự đúng; cao giúp giảm cảnh báo giả.
Recall	$\frac{TP}{TP + FN}$	Trong các mẫu thực sự Positive, tỉ lệ được phát hiện; cao giúp giảm bỏ sót xâm nhập.
F1-score	$\frac{2TP}{2TP + FP + FN}$	Trung bình điều hòa giữa Precision và Recall, chỉ cao khi cả hai chỉ số đều cao.

Giữa Precision và Recall luôn tồn tại sự đánh đổi (*trade-off*): hạ ngưỡng phân loại giúp tăng Recall nhưng làm tăng FP và giảm Precision, trong khi tăng ngưỡng giúp tăng Precision nhưng có thể bỏ sót nhiều mẫu Positive hơn. F1-score phản ánh điểm cân bằng giữa hai chỉ số này, đặc biệt phù hợp khi dữ liệu không cân bằng và cả hai loại lỗi FP, FN đều cần được kiểm soát.[6]

Confusion Matrix là công cụ trực quan hóa toàn diện nhất để đánh giá hiệu năng của mô hình, từ đó toàn bộ các chỉ số ở trên đều có thể được dẫn xuất trực tiếp. Với bài toán phân loại nhị phân, Confusion Matrix có dạng ma trận 2×2 như minh họa ở **Hình 2.3**.

		Dự đoán của mô hình	
		Normal	Intrusion
Thực tế	Normal	TN (đúng — không cảnh báo)	FP (cảnh báo nhầm)
	Intrusion	FN (bỏ sót xâm nhập)	TP (phát hiện đúng)

Hình 2.3. Confusion Matrix cho bài toán phân loại nhị phân

Trong các bài toán cảnh báo xâm nhập, hai loại lỗi cần đặc biệt chú ý: lỗi FP khiến hệ thống cảnh báo sai khi không có xâm nhập thực sự, làm giảm độ tin cậy và có thể gây tâm lý xem thường cảnh báo (*alarm fatigue*); lỗi FN khiến hệ thống bỏ sót hành vi xâm nhập thực tế, là lỗi nghiêm trọng hơn vì ảnh hưởng trực tiếp đến mục tiêu an ninh của hệ thống. Do đó, việc tối thiểu hóa FN, hay tương đương tối đa hóa Recall, thường được ưu tiên trong các bài toán cảnh báo rủi ro cao. Confusion Matrix sinh ra từ quá trình đánh giá thực nghiệm sẽ được phân tích chi

tiết trong Chương 5.

2.8. CƠ SỞ LÝ THUYẾT VỀ TRIỂN KHAI MÔ HÌNH TRÊN THIẾT BỊ NHÚNG

2.8.1. Edge AI

Edge AI (trí tuệ nhân tạo biên) là mô hình triển khai trong đó toàn bộ quá trình inference của mô hình AI được thực hiện trực tiếp trên thiết bị đầu cuối (edge device) tại nơi dữ liệu được thu thập, thay vì gửi dữ liệu lên máy chủ trung tâm để xử lý rồi nhận kết quả về. Trong các hệ thống thị giác máy tính thời gian thực, Edge AI giúp giảm độ trễ, tăng tính riêng tư và cho phép hệ thống hoạt động ổn định hơn trong điều kiện kết nối mạng không liên tục.

Mô hình triển khai truyền thống theo kiến trúc cloud-based có độ trễ (*latency*) được xác định bởi:

$$T_{\text{total}}^{\text{cloud}} = T_{\text{capture}} + T_{\text{compress}} + T_{\text{transmit}} + T_{\text{inference_server}} + T_{\text{return}} \quad (2.37)$$

Trong đó T_{transmit} và T_{return} phụ thuộc vào băng thông mạng, thường dao động từ vài chục đến vài trăm millisecond. Ngược lại, với kiến trúc Edge AI:

$$T_{\text{total}}^{\text{edge}} = T_{\text{capture}} + T_{\text{preprocess}} + T_{\text{inference_local}} \quad (2.38)$$

Toàn bộ quá trình diễn ra tại chỗ, loại bỏ hoàn toàn độ trễ mạng. Đây là yếu tố quan trọng đối với các hệ thống cảnh báo hoặc giám sát thời gian thực, nơi độ trễ xử lý có ảnh hưởng trực tiếp đến khả năng phản hồi của hệ thống.[25]

Ngoài giảm độ trễ, Edge AI mang lại ba lợi ích quan trọng khác:

- Tính riêng tư: dữ liệu hình ảnh camera có thể được xử lý trực tiếp trên thiết bị, giảm rủi ro rò rỉ thông tin qua đường truyền mạng.
- Hoạt động độc lập: hệ thống vẫn có thể hoạt động ngay cả khi mất kết nối internet, phù hợp với các địa điểm có hạ tầng mạng không ổn định.
- Cảnh báo thời gian thực: xử lý tại chỗ giúp giảm độ trễ end-to-end, từ đó hỗ trợ hệ thống phản hồi nhanh hơn khi xuất hiện sự kiện bất thường.

2.8.2. ONNX và ONNX Runtime

ONNX (Open Neural Network Exchange) là định dạng trung gian mở cho phép biểu diễn mô hình học sâu độc lập với framework huấn luyện. Mô hình được huấn luyện bằng các framework như PyTorch hoặc TensorFlow có thể được export sang ONNX và chạy trên nhiều nền tảng khác nhau thông qua ONNX Runtime mà không cần phụ thuộc trực tiếp vào môi trường huấn luyện ban đầu.

2.8.2.1. Quá trình export ONNX

Khi export mô hình sang ONNX, đồ thị tính toán (computation graph) được ghi lại dưới dạng đồ thị có hướng không chu trình (DAG — Directed Acyclic Graph):

$$G = (V, E) \quad (2.39)$$

Trong đó V là tập các nút (*nodes*) đại diện cho các phép toán như tích chập, cộng, nhân hoặc activation;

còn E là tập các cạnh (*edges*) biểu diễn luồng dữ liệu tensor giữa các phép toán. Định dạng ONNX chuẩn hóa tập hợp các operator được hỗ trợ, đảm bảo tính tương thích khi triển khai mô hình trên các nền tảng phần cứng và phần mềm khác nhau.[7]

2.8.2.2. ONNX Runtime

ONNX Runtime là engine thực thi được tối ưu hóa cho inference, tự động áp dụng các kỹ thuật tối ưu hóa đồ thị:

- Operator fusion: gộp nhiều phép toán liên tiếp, ví dụ Conv + BN + ReLU, thành một phép toán duy nhất, giảm số lần đọc/ghi bộ nhớ.
- Constant folding: tính trước các biểu thức hằng trong đồ thị, loại bỏ tính toán thừa lúc inference.
- Memory planning: tái sử dụng bộ nhớ cho các tensor trung gian, giảm mức sử dụng bộ nhớ cực đại trong quá trình chạy mô hình.

Trên các thiết bị nhúng, ONNX Runtime có thể chạy trên CPU hoặc các backend tăng tốc phần cứng tùy nền tảng. Nhờ các tối ưu hóa như SIMD (*Single Instruction Multiple Data*), tối ưu đồ thị và quản lý bộ nhớ, ONNX Runtime thường giúp cải thiện tốc độ inference so với việc triển khai trực tiếp bằng framework huấn luyện đầy đủ.

2.9. THƯ VIỆN OPENCV

OpenCV (Open Source Computer Vision Library) là một thư viện mã nguồn mở chuyên cung cấp các thuật toán và công cụ phục vụ cho lĩnh vực xử lý ảnh và thị giác máy tính (Computer Vision). Dự án được khởi xướng tại Intel bởi Gary Bradski vào năm 1999 và phát hành phiên bản đầu tiên vào năm 2000. OpenCV được phát triển theo giấy phép mã nguồn mở, cho phép sử dụng miễn phí trong cả nghiên cứu, giáo dục và thương mại.

Thư viện hỗ trợ đa nền tảng như Windows, Linux, macOS, Android và iOS, đồng thời tương thích với nhiều ngôn ngữ lập trình phổ biến như C++, Python, Java và MATLAB. Bên cạnh đó, OpenCV còn hỗ trợ tăng tốc xử lý thông qua các công nghệ CUDA và OpenCL trên các phần cứng tương thích, giúp cải thiện hiệu năng của các ứng dụng xử lý ảnh và thị giác máy tính.

OpenCV cung cấp nhiều chức năng quan trọng như xử lý ảnh và video, chuyển đổi không gian màu, lọc ảnh, phát hiện biên, nhận dạng đối tượng, phân đoạn ảnh, trích xuất đặc trưng, theo dõi đối tượng và hỗ trợ xây dựng các mô hình học máy. Nhờ khả năng xử lý mạnh mẽ và tính linh hoạt cao, OpenCV được ứng dụng rộng rãi trong các lĩnh vực như giám sát an ninh, robot, xe tự hành, y tế, công nghiệp và trí tuệ nhân tạo [14].

2.10. GIAO THỨC RTSP

RTSP (Real-Time Streaming Protocol) là giao thức điều khiển truyền phát dữ liệu đa phương tiện theo thời gian thực, được chuẩn hóa trong RFC 2326 bởi IETF (Internet Engineering Task Force). Giao thức này cho phép thiết lập, điều khiển và kết thúc các phiên truyền tải dữ liệu âm thanh hoặc video giữa máy chủ và thiết bị khách thông qua các lệnh như SETUP, PLAY, PAUSE và TEARDOWN. Khác với các giao thức truyền dữ liệu thông thường, RTSP không trực tiếp truyền tải nội dung video hoặc âm thanh mà chỉ đảm nhiệm vai trò quản lý phiên truyền. Dữ liệu đa phương tiện thường được truyền thông qua giao thức RTP (Real-time Transport Protocol) hoặc RTP over TCP, trong khi RTSP chịu trách nhiệm điều khiển quá trình phát và nhận dữ liệu.

Nhờ khả năng truyền dữ liệu với độ trễ thấp, RTSP được sử dụng rộng rãi trong các hệ thống camera IP, giám sát an ninh, hội nghị truyền hình và các ứng dụng phát trực tuyến. Giao thức này hỗ trợ nhiều chức năng như phát trực tiếp, tạm dừng, tiếp tục phát và kết thúc phiên truyền mà không cần thiết lập lại toàn bộ kết nối.[15]

Một địa chỉ RTSP thường có dạng:

rtsp://username:password@ip_address:554/path

Trong đó, username và password là thông tin xác thực người dùng, ip_address là địa chỉ IP của thiết bị phát, port là cổng dịch vụ (mặc định là 554) và path là đường dẫn đến luồng dữ liệu cần truy cập. Với khả năng truyền tải hình ảnh liên tục và ổn định, RTSP hiện là một trong những giao thức phổ biến nhất trong các hệ thống camera giám sát và xử lý video thời gian thực.

2.11. MEDIA MTX

MediaMTX (trước đây có tên là rtsp-simple-server) là một máy chủ truyền thông (Media Server) mã nguồn mở được phát triển để tiếp nhận, quản lý và phân phối các luồng đa phương tiện theo thời gian thực. MediaMTX hỗ trợ nhiều giao thức truyền phát phổ biến như RTSP, RTMP, HLS, WebRTC, SRT và MPEG-TS, cho phép tiếp nhận, chuyển tiếp và phát lại luồng video hoặc âm thanh đến nhiều thiết bị khác nhau thông qua mạng nội bộ hoặc Internet. MediaMTX hoạt động theo mô hình trung gian giữa nguồn phát và thiết bị nhận dữ liệu. Máy chủ có thể tiếp nhận luồng video từ camera IP, webcam hoặc các ứng dụng phát trực tiếp, sau đó phân phối lại đến nhiều thiết bị khách mà không yêu cầu nguồn phát phải tạo nhiều kết nối đồng thời. Cơ chế này giúp giảm tải cho thiết bị nguồn, nâng cao khả năng mở rộng và tăng tính ổn định của hệ thống.

Một ưu điểm nổi bật của MediaMTX là khả năng chuyển đổi giữa các giao thức truyền phát (Protocol Remuxing) mà không cần mã hóa lại nội dung video (Transcoding), từ đó giảm đáng kể tải nguyên xử lý của hệ thống. Ngoài ra, phần mềm còn hỗ trợ xác thực người dùng, ghi hình (Recording), cung cấp API quản lý và theo dõi trạng thái các luồng truyền theo thời gian thực.

Nhờ khả năng quản lý và phân phối luồng dữ liệu hiệu quả, MediaMTX được ứng dụng rộng rãi trong các hệ thống camera giám sát, truyền hình trực tiếp, Internet of Things (IoT), xử lý video bằng trí tuệ nhân tạo và các ứng dụng yêu cầu truyền tải dữ liệu đa phương tiện với độ trễ thấp.[16]

2.12. GIAO THỨC HLS

HLS (HTTP Live Streaming) là giao thức truyền phát nội dung đa phương tiện do Apple phát triển, được thiết kế để truyền tải video và âm thanh theo cơ chế gần thời gian thực (Near Real-Time Streaming) thông qua giao thức HTTP. Thay vì truyền liên tục một luồng dữ liệu duy nhất, HLS chia nội dung thành nhiều đoạn nhỏ (Segment) có thời lượng từ vài giây và tạo một tệp danh sách phát (Playlist) định dạng M3U8 để quản lý các đoạn dữ liệu này. Khi thiết bị khách truy cập luồng HLS, trình phát sẽ tải tệp M3U8 trước, sau đó lần lượt tải các đoạn video tương ứng để phát liên tục. Cơ chế này giúp giảm hiện tượng giật, lag khi mạng không ổn định và hỗ trợ khả năng thích ứng tốc độ truyền (Adaptive Bitrate Streaming), cho phép tự động điều chỉnh chất lượng video phù hợp với băng thông mạng hiện tại.

Do hoạt động dựa trên giao thức HTTP tiêu chuẩn, HLS có khả năng tương thích cao với trình duyệt web, thiết bị di động và mạng phân phối nội dung (CDN), đồng thời dễ dàng vượt qua tường lửa và bộ định tuyến hơn so với nhiều giao thức truyền phát khác. Tuy nhiên, do sử dụng cơ chế phân đoạn dữ liệu và bộ đệm phát, HLS thường tạo ra độ trễ từ vài giây so với nguồn phát trực tiếp.[17]

Một luồng HLS thường được truy cập thông qua tệp danh sách phát có phần mở rộng .m3u8, ví dụ:

`http://server_ip/live/stream/index.m3u8`

Trong đó, trình phát sẽ đọc tệp M3U8 để xác định vị trí các đoạn dữ liệu video và tải chúng theo thứ tự nhằm tạo thành luồng phát liên tục cho người dùng.

2.13. CLOUDFLARE TUNNEL

Cloudflare Tunnel là một dịch vụ do Cloudflare phát triển, cho phép công bố (Publish) các dịch vụ đang chạy trong mạng nội bộ lên Internet một cách an toàn mà không cần cấu hình Port Forwarding trên bộ định tuyến hoặc sử dụng địa chỉ IP công cộng. Dịch vụ này tạo ra một kết nối được mã hóa từ máy chủ nội bộ đến hạ tầng của

Cloudflare, sau đó chuyển tiếp các yêu cầu từ người dùng bên ngoài đến dịch vụ tương ứng

Cơ chế hoạt động của Cloudflare Tunnel dựa trên việc cài đặt một tác nhân (Agent) có tên cloudflared trên máy chủ nội bộ. Tác nhân này sẽ thiết lập một kết nối bảo mật từ mạng nội bộ đến hệ thống Cloudflare và duy trì kết nối liên tục. Khi người dùng truy cập tên miền đã được cấu hình, yêu cầu sẽ được định tuyến thông qua mạng Cloudflare rồi chuyển tiếp đến dịch vụ bên trong thông qua đường hầm (Tunnel) đã được thiết lập.

Do sử dụng kết nối outbound từ máy chủ nội bộ, Cloudflare Tunnel không yêu cầu cấu hình NAT Port Forwarding hoặc IP Public tĩnh, góp phần giảm nguy cơ bị tấn công trực tiếp từ Internet. Ngoài ra, dịch vụ còn hỗ trợ mã hóa lưu lượng truyền tải và dễ dàng tích hợp với các cơ chế bảo mật của Cloudflare như xác thực người dùng, tường lửa ứng dụng web (WAF) và kiểm soát truy cập.

Nhờ khả năng thiết lập kết nối an toàn và ổn định, Cloudflare Tunnel được sử dụng rộng rãi trong các hệ thống giám sát từ xa, dịch vụ web nội bộ, ứng dụng IoT và các hệ thống cần truy cập từ Internet mà vẫn đảm bảo tính bảo mật cao.[18]

2.14. GIAO THỨC HTTPS

HTTPS (HyperText Transfer Protocol Secure) là phiên bản bảo mật của giao thức HTTP, được sử dụng để truyền tải dữ liệu giữa máy khách (Client) và máy chủ (Server) thông qua mạng Internet. HTTPS sử dụng giao thức TLS (Transport Layer Security) để mã hóa dữ liệu trong quá trình truyền, giúp bảo vệ thông tin khỏi nguy cơ bị nghe lén, giả mạo hoặc chỉnh sửa bởi các bên thứ ba.

Khi một kết nối HTTPS được thiết lập, máy khách và máy chủ sẽ thực hiện quá trình bắt tay bảo mật nhằm xác thực danh tính của hai bên và tạo khóa mã hóa cho phiên làm việc. Sau đó, toàn bộ dữ liệu trao đổi giữa máy khách và máy chủ sẽ được mã hóa trước khi truyền đi, đảm bảo tính bí mật (Confidentiality), tính toàn vẹn dữ liệu (Integrity) và khả năng xác thực (Authentication) của thông tin.

Nhờ khả năng bảo vệ dữ liệu hiệu quả, HTTPS hiện là giao thức tiêu chuẩn cho hầu hết các website và ứng dụng trên Internet, đặc biệt là các hệ thống yêu cầu đăng nhập, thanh toán trực tuyến, giám sát từ xa và truyền tải dữ liệu nhạy cảm. Việc sử dụng HTTPS không chỉ nâng cao mức độ bảo mật mà còn tăng tính tin cậy của hệ thống và giảm nguy cơ bị tấn công trung gian.[26]

Một địa chỉ HTTPS thường có dạng:

`https://domain_name/path`

Trong đó, domain_name là tên miền của máy chủ và path là đường dẫn đến tài nguyên cần truy cập. So với HTTP thông thường, HTTPS cung cấp mức độ bảo mật cao hơn và hiện nay đã trở thành giao thức truyền tải dữ liệu phổ biến trên Internet.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. PHÂN TÍCH YÊU CẦU HỆ THỐNG

Hệ thống camera giám sát thông minh được xây dựng nhằm hỗ trợ theo dõi khu vực cần bảo vệ, phát hiện các hành vi xâm nhập và đưa ra cảnh báo kịp thời. Để đáp ứng mục tiêu này, hệ thống cần đảm bảo các yêu cầu về giám sát hình ảnh, phát hiện sự kiện bất thường, cảnh báo, lưu trữ dữ liệu và khả năng vận hành ổn định trong thời gian dài.

Trước hết, hệ thống cần cung cấp khả năng giám sát khu vực thông qua hình ảnh và video được cập nhật liên tục. Chất lượng hình ảnh cần đủ rõ ràng để phục vụ cho việc quan sát, theo dõi đối tượng và hỗ trợ quá trình phát hiện xâm nhập.

Bên cạnh đó, hệ thống cần hỗ trợ giám sát trực tuyến theo thời gian gần thực thông qua giao diện quản lý. Chức năng này giúp theo dõi trạng thái hiện tại của khu vực giám sát một cách liên tục và thuận tiện.

Một yêu cầu quan trọng là hệ thống phải có khả năng phát hiện các tình huống xâm nhập hoặc hoạt động bất thường xảy ra trong khu vực bảo vệ. Khi phát hiện đối tượng xuất hiện trong vùng giám sát, hệ thống cần thực hiện nhận diện và đánh giá nguy cơ nhằm hỗ trợ quá trình xử lý sự cố.

Khi phát hiện sự kiện bất thường, hệ thống cần tự động kích hoạt cơ chế cảnh báo để thu hút sự chú ý và hỗ trợ phản ứng kịp thời. Bên cạnh đó, hệ thống cũng cần cho phép tắt cảnh báo thủ công trong trường hợp còi báo động vẫn đang hoạt động sau khi sự cố đã được xử lý hoặc khi cần dừng cảnh báo để tránh ảnh hưởng đến quá trình vận hành.

Để đảm bảo hiệu quả giám sát trong nhiều điều kiện môi trường khác nhau, hệ thống cần hỗ trợ chiếu sáng khu vực giám sát khi cần thiết. Chức năng chiếu sáng phải cho phép điều khiển từ xa thông qua giao diện quản lý nhằm nâng cao khả năng quan sát vào ban đêm hoặc trong điều kiện thiếu sáng.

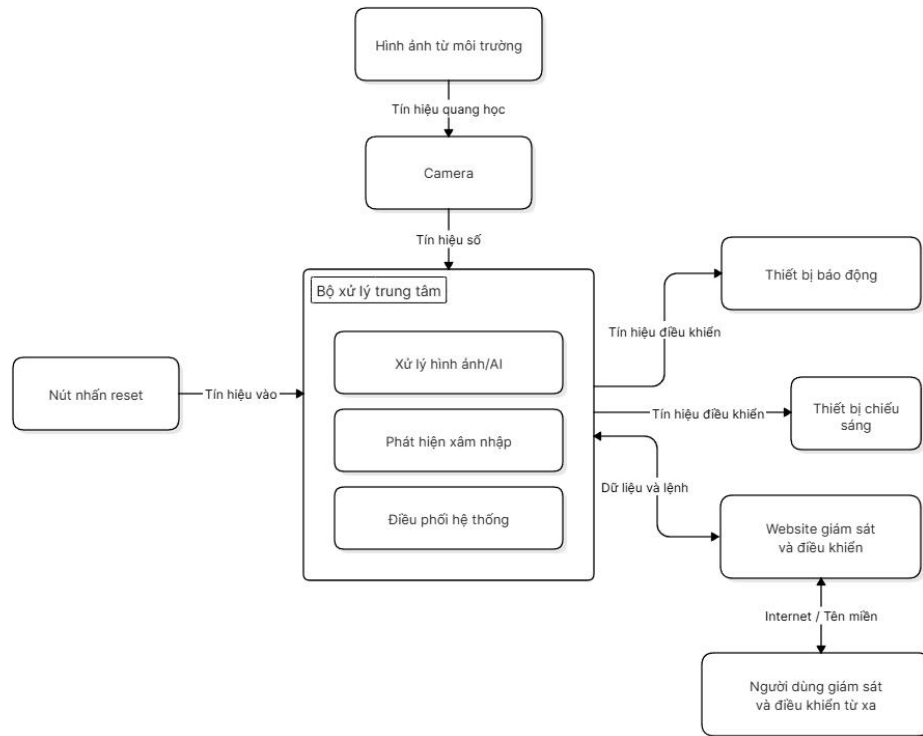
Về lưu trữ dữ liệu, hệ thống cần có khả năng ghi nhận và lưu trữ dữ liệu giám sát trong suốt quá trình hoạt động. Khi xảy ra sự kiện bất thường, hệ thống cần lưu lại hình ảnh liên quan kèm theo thông tin thời gian để phục vụ công tác kiểm tra và đối chiếu sau này. Hệ thống cũng cần hỗ trợ quản lý dữ liệu đã lưu, bao gồm các chức năng tra cứu, xem lại, phát lại, tải về và xóa dữ liệu khi cần thiết.

Ngoài ra, hệ thống cần hỗ trợ khả năng truy cập và giám sát từ xa thông qua Internet. Điều này cho phép thực hiện việc theo dõi và quản lý hệ thống ở mọi nơi có kết nối mạng.

Hệ thống cần được cung cấp nguồn điện ổn định nhằm đảm bảo các chức năng giám sát, xử lý, cảnh báo và lưu trữ dữ liệu hoạt động liên tục trong quá trình vận hành.

Hệ thống cần được trang bị nút nhấn Reset vật lý nhằm hỗ trợ khởi động lại thiết bị khi xảy ra sự cố hoặc lỗi trong quá trình vận hành, góp phần giảm thời gian gián đoạn và nâng cao độ tin cậy của hệ thống.

3.2. KIẾN TRÚC TỔNG THỂ CỦA HỆ THỐNG



Hình 3.1. Sơ đồ đặc tả kỹ thuật của hệ thống giám sát xâm nhập ứng dụng trí tuệ nhân tạo

Hệ thống camera giám sát thông minh được xây dựng nhằm theo dõi khu vực cần bảo vệ, phát hiện hành vi xâm nhập và đưa ra cảnh báo khi xuất hiện sự kiện bất thường. Cấu trúc hệ thống gồm các khối chức năng chính: camera, bộ xử lý trung tâm, thiết bị báo động, thiết bị chiếu sáng, website giám sát và nút nhấn reset.

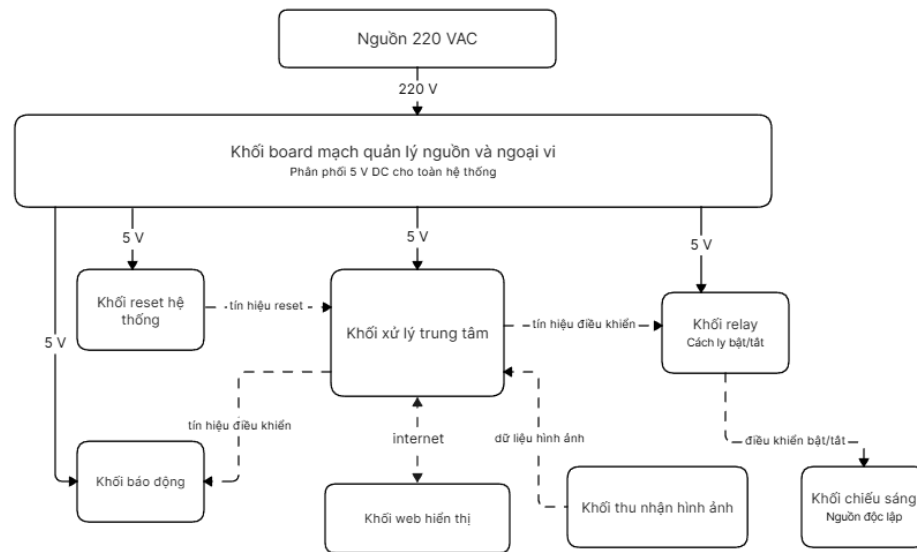
Camera có nhiệm vụ thu nhận hình ảnh từ môi trường và chuyển đổi thành dữ liệu số để truyền đến bộ xử lý trung tâm. Tại đây, dữ liệu hình ảnh được xử lý bằng các thuật toán thị giác máy tính và AI nhằm phát hiện sự xuất hiện của đối tượng, nhận biết hành vi xâm nhập và đưa ra quyết định điều khiển hệ thống.

Khi phát hiện sự kiện bất thường, bộ xử lý trung tâm sẽ gửi tín hiệu điều khiển đến thiết bị báo động để kích hoạt cảnh báo tại chỗ. Đồng thời, hệ thống có thể điều khiển thiết bị chiếu sáng nhằm hỗ trợ quan sát trong điều kiện thiếu sáng, nâng cao chất lượng hình ảnh và hiệu quả giám sát.

Bên cạnh đó, bộ xử lý trung tâm giao tiếp với website giám sát và điều khiển thông qua kết nối Internet. Website cho phép người dùng theo dõi hình ảnh trực tiếp, xem trạng thái hoạt động của hệ thống, nhận thông tin cảnh báo và thực hiện các thao tác điều khiển từ xa như bật/tắt đèn hoặc thiết bị báo động.

Ngoài ra, hệ thống được trang bị nút nhấn reset để hỗ trợ khởi động lại và khôi phục hoạt động khi cần thiết. Nhờ sự phối hợp giữa các khối chức năng, hệ thống có khả năng thu nhận hình ảnh, xử lý dữ liệu, phát hiện xâm nhập, kích hoạt cảnh báo và hỗ trợ giám sát từ xa một cách tự động, đáp ứng yêu cầu của một hệ thống camera giám sát thông minh trong thực tế.

3.2.1. Sơ đồ khối hệ thống



Hình 3.2. Sơ đồ khối của hệ thống

3.2.2. Vai trò của từng khối trong hệ thống

Khối thu nhận hình ảnh: Khối thu nhận và xử lý ảnh có nhiệm vụ tiếp nhận hình ảnh từ khu vực giám sát, chuyển đổi dữ liệu hình ảnh thành dạng phù hợp để đưa về khối xử lý trung tâm. Đây là khối đầu vào chính của hệ thống, cung cấp dữ liệu phục vụ cho quá trình giám sát và phát hiện xâm nhập.

Khối xử lý trung tâm: Khối xử lý trung tâm là khối điều phối chính của toàn bộ hệ thống. Khối này tiếp nhận dữ liệu hình ảnh, thực hiện xử lý, đánh giá trạng thái xâm nhập và đưa ra các tín hiệu điều khiển tương ứng. Ngoài ra, khối xử lý trung tâm còn giao tiếp với khối Website hiển thị để truyền dữ liệu giám sát và nhận lệnh điều khiển từ xa.

Khối báo động: Khối báo động có nhiệm vụ phát tín hiệu cảnh báo khi hệ thống phát hiện có dấu hiệu xâm nhập. Khối này nhận tín hiệu điều khiển từ khối xử lý trung tâm và thực hiện cảnh báo tại khu vực giám sát, giúp hỗ trợ quá trình xử lý sự cố kịp thời.

Khối relay: Khối relay có nhiệm vụ đóng vai trò là tầng chấp hành, nhận tín hiệu điều khiển từ khối xử lý trung tâm để thực hiện đóng hoặc ngắt nguồn cấp cho tải bên ngoài. Việc sử dụng relay giúp cách ly giữa mạch điều khiển điện áp thấp và mạch công suất, góp phần nâng cao độ an toàn và độ tin cậy của hệ thống.

Khối chiếu sáng: Khối chiếu sáng có nhiệm vụ cung cấp ánh sáng cho khu vực giám sát trong điều kiện thiếu sáng hoặc vào ban đêm, góp phần nâng cao chất lượng hình ảnh thu nhận và cải thiện hiệu quả giám sát của hệ thống. Khối chiếu sáng sử dụng nguồn cấp độc lập và được điều khiển đóng/ngắt thông qua khối relay, cho phép hệ thống chủ động kích hoạt hoặc tắt nguồn chiếu sáng theo trạng thái vận hành.

Khối reset hệ thống: Khối reset hệ thống có nhiệm vụ gửi tín hiệu reset về khối xử lý trung tâm khi cần khởi động lại hoặc phục hồi hoạt động của hệ thống. Chức năng này hỗ trợ quá trình vận hành, bảo trì và xử lý lỗi khi hệ thống gặp tình trạng treo hoặc hoạt động không ổn định.

Khối Web hiển thị: Khối Web hiển thị có nhiệm vụ cung cấp giao diện để theo dõi và điều khiển hệ thống. Khối này nhận dữ liệu giám sát từ khối xử lý trung tâm, hiển thị trạng thái hoạt động, hình ảnh và video giám sát, đồng thời cho phép gửi lệnh điều khiển ngược lại hệ thống khi cần thiết.

Khối board mạch quản lý nguồn và ngoại vi: Khối board mạch quản lý nguồn và ngoại vi có nhiệm vụ

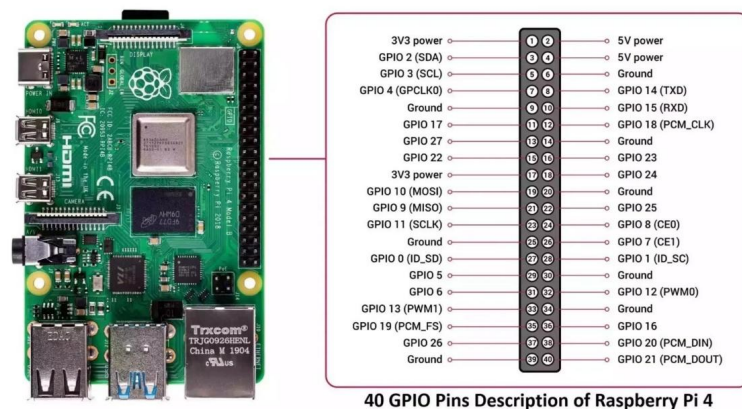
tiếp nhận nguồn điện xoay chiều 220 VAC, thực hiện chuyển đổi thành nguồn điện một chiều 5 VDC và phân phối đến các khối phần cứng chính của hệ thống như khối xử lý trung tâm, khối relay, khối báo động và khối reset hệ thống. Khối này đảm bảo điện áp và dòng điện ổn định cho các thiết bị hoạt động liên tục, góp phần hạn chế tình trạng sụt áp hoặc mất ổn định trong quá trình vận hành.

3.3. THIẾT KẾ PHẦN CỨNG HỆ THỐNG

3.3.1. Thiết kế khối xử lý trung tâm

Dựa trên yêu cầu của hệ thống, khối xử lý trung tâm cần thực hiện các nhiệm vụ thu nhận hình ảnh, phát hiện xâm nhập, lưu trữ dữ liệu, vận hành giao diện Web và điều khiển các thiết bị ngoại vi. Do phải xử lý đồng thời nhiều tác vụ như xử lý ảnh, truyền dữ liệu và giám sát trực tuyến, khối này cần có hiệu năng đủ cao và hoạt động ổn định trong thời gian dài.

Các vi điều khiển như Arduino Uno, Arduino Mega, ESP8266 và ESP32 phù hợp cho các ứng dụng điều khiển cơ bản nhưng còn hạn chế về khả năng xử lý hình ảnh và chạy nhiều tác vụ đồng thời. Vì vậy, Raspberry Pi 4 được lựa chọn làm khối xử lý trung tâm nhờ hỗ trợ hệ điều hành Linux, bộ nhớ lớn, kết nối mạng và hiệu năng xử lý phù hợp với yêu cầu của hệ thống.[9]

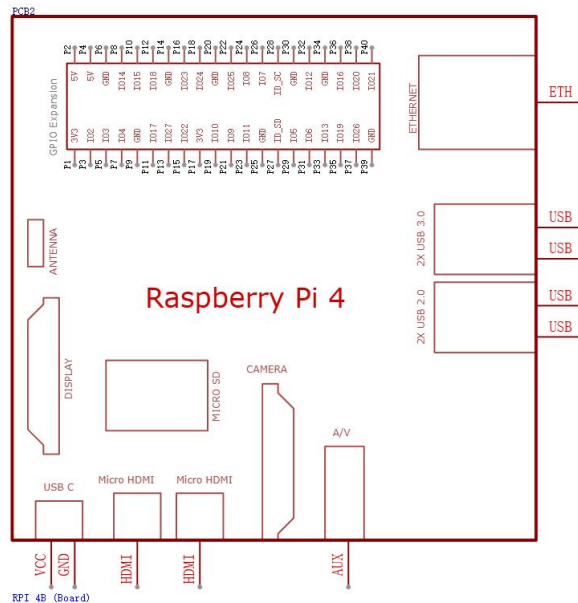


Hình 3.3. Hình ảnh thực tế và sơ đồ chân của Raspberry Pi 4

Bảng 3.1. Thông số kỹ thuật của Raspberry Pi 4 Model B

Thông số kỹ thuật	Mô tả
Bộ xử lý	Broadcom BCM2711
Kiến trúc CPU	4 nhân ARM Cortex-A72, 64-bit
Xung nhịp CPU	1.5 GHz
Bộ nhớ RAM	2GB / 4GB / 8GB tùy phiên bản
Kết nối mạng	Gigabit Ethernet, Wi-Fi
Giao tiếp USB	USB 2.0 và USB 3.0
GPIO	40 chân GPIO
Lưu trữ	Thẻ nhớ microSD 64 GB
Nguồn cấp	DC 5V qua cổng USB-C
Dòng khuyến nghị	Khoảng 3A
Hệ điều hành hỗ trợ	Linux-based OS

Với những đặc điểm trên (Bảng 3.1), nhóm quyết định chọn máy tính nhúng Raspberry Pi 4 Model B làm thiết bị xử lý trung tâm.



Hình 3.4. Sơ đồ bo mạch Raspberry Pi 4

Mô tả cách kết nối của Raspberry pi 4 với các khối (Hình 3.4):

Raspberry Pi 4 được kết nối với các khối chức năng thông qua nhiều chuẩn giao tiếp khác nhau. Camera sử dụng cổng USB để truyền dữ liệu hình ảnh số về Raspberry Pi 4. Các khối báo động, chiếu sáng và reset hệ thống được kết nối qua các chân GPIO để thực hiện điều khiển hoặc nhận tín hiệu vào. Đối với khối Web hiển thị, Raspberry Pi 4 sử dụng kết nối mạng để trao đổi dữ liệu giám sát và lệnh điều khiển. Nhờ các giao tiếp này, Raspberry Pi 4 đảm nhiệm vai trò trung tâm trong việc liên kết và điều phối toàn bộ hệ thống.

3.3.2. Thiết kế khối thu nhận hình ảnh

Dựa trên yêu cầu của hệ thống, khối thu nhận hình ảnh cần cung cấp dữ liệu video liên tục với chất lượng đủ rõ nét để phục vụ giám sát trực tuyến và phát hiện xâm nhập. Ngoài ra, thiết bị thu nhận ảnh cần dễ dàng kết nối với khối xử lý trung tâm, hoạt động ổn định và đáp ứng yêu cầu vận hành trong thời gian dài.

Để đáp ứng các yêu cầu trên, nhóm lựa chọn sử dụng camera dạng webcam giao tiếp qua cổng USB với độ phân giải tối đa 1920×1080. Chuẩn giao tiếp USB giúp việc kết nối với Raspberry Pi 4 đơn giản, không cần phần cứng trung gian và được hỗ trợ trực tiếp bởi hệ điều hành. Độ phân giải Full HD cung cấp chất lượng hình ảnh đủ tốt cho quá trình quan sát và phát hiện đối tượng trong khu vực giám sát.

Trong quá trình vận hành, độ phân giải xử lý có thể được điều chỉnh phù hợp nhằm cân bằng giữa chất lượng hình ảnh, tốc độ xử lý và khả năng hoạt động ổn định của hệ thống.



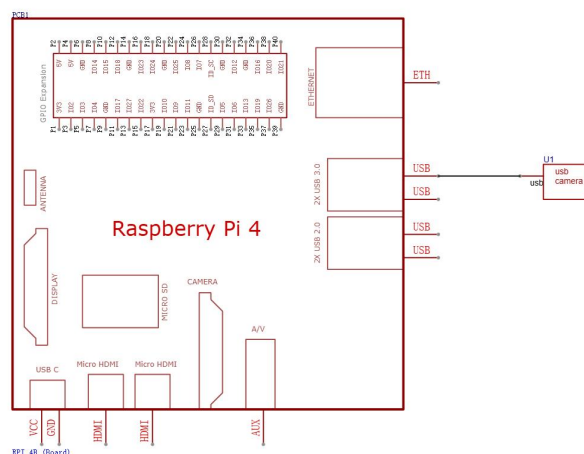
Hình 3.5. Webcam USB Full HD

Thông số kỹ thuật

Bảng 3.2. Thông số kỹ thuật của webcam USB

Thông số kỹ thuật	Mô tả
Loại camera	Webcam USB
Cảm biến hình ảnh	CMOS
Kích thước cảm biến	1/2.9 inch
Độ phân giải	Full HD 1920 x 1080 pixel
Điểm ảnh hiệu quả	2.0 Megapixel
Tiêu cự ống kính	3.6 mm
Góc nhìn	75.7 độ
Tốc độ khung hình	25 FPS / 30 FPS
Chuẩn giao tiếp	USB
Định dạng video	H.264 / MJPEG
Điện áp hoạt động	DC 5V
Dòng điện hoạt động	160 mA $\pm$ 6 mA
Nhiệt độ làm việc	-20°C đến 60°C

Với những đặc điểm trên (**Bảng 3.2**), nhóm quyết định chọn Webcam USB làm khối xử lý ảnh chính của hệ thống.

**Hình 3.6.** Sơ đồ kết nối camera USB với Raspberry Pi 4

Mô tả kết nối của camera USB với Raspberry Pi 4 (Hình 3.6):

Giao tiếp USB được sử dụng để truyền dữ liệu hình ảnh số từ camera về Raspberry Pi 4.

Bên cạnh chức năng truyền dữ liệu, cổng USB cũng cung cấp nguồn điện hoạt động cho camera. Nhờ đó, camera không cần sử dụng thêm nguồn cấp riêng trong quá trình vận hành.

Sau khi hệ thống khởi động, Raspberry Pi 4 nhận diện camera như một thiết bị video đầu vào. Từ đó, hệ thống tiến hành đọc các khung hình liên tục để phục vụ quá trình giám sát, xử lý ảnh và phát hiện xâm nhập.

3.3.3. Thiết kế khối báo động

Khối báo động có nhiệm vụ phát tín hiệu âm thanh khi hệ thống phát hiện trạng thái xâm nhập hoặc tình huống bất thường trong khu vực giám sát. Tín hiệu cảnh báo cần đủ rõ để có thể nhận biết tại chỗ, đồng thời mạch điều khiển phải đơn giản, dễ kết nối với khối xử lý trung tâm và phù hợp với đường nguồn phân phối trên bo mạch trung tâm.

Dựa trên yêu cầu của hệ thống, thiết bị báo động cần có điện áp hoạt động thấp, dòng tiêu thụ nhỏ, kích

thước gọn, dễ lắp đặt trên mạch và có mức âm lượng đủ lớn. Một số phương án có thể sử dụng cho khối báo động gồm loa thường, còi báo động công suất lớn hoặc buzzer điện tử. Loa thường cần thêm mạch khuếch đại và tín hiệu âm thanh điều khiển phức tạp hơn. Còi công suất lớn có âm lượng cao nhưng thường yêu cầu nguồn riêng và mạch công suất lớn hơn. Trong khi đó, buzzer điện tử có cấu tạo đơn giản, dễ điều khiển bằng tín hiệu đóng/ngắt và phù hợp với mô hình hệ thống nhỏ gọn.

Vì vậy, hệ thống lựa chọn buzzer điện tử 1209 làm thiết bị báo động. Buzzer này sử dụng điện áp định mức 5VDC, dòng tiêu thụ khoảng 22mA, âm lượng đạt khoảng 93dB và có kích thước nhỏ 12 x 9mm. Với các thông số này, buzzer đáp ứng được yêu cầu cảnh báo tại chỗ, đồng thời dễ tích hợp vào mạch phần cứng của hệ thống.



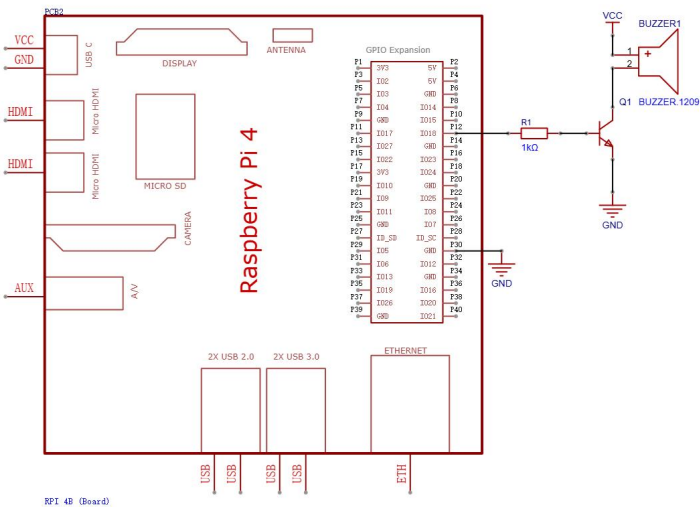
Hình 3.7. Buzzer 1209 sử dụng trong khối báo động

Thông số kỹ thuật

Bảng 3.3. Thông số kỹ thuật của buzzer điện tử

Thông số kỹ thuật	Mô tả
Loại	Buzzer điện tử
Kiểu gắn	Xuyên lỗ
Điện áp định mức	5VDC
Dòng điện hoạt động	22mA
Cường độ âm thanh	93dB
Tần số âm thanh	2300Hz
Kích thước	12 x 9mm
Nhiệt độ làm việc	-20°C đến +70°C
Đường kính	10 - 19mm

Với những đặc điểm trên (**Bảng 3.3**) nhóm quyết định chọn Buzzer 1209 cho đề tài.



Hình 3.8. Sơ đồ kết nối Raspberry Pi 4 với còi báo động

Mô tả cách kết nối của khối báo động (Hình 3.8):

Khối báo động được điều khiển bởi Raspberry Pi 4 thông qua chân GPIO 17. Do chân GPIO không đủ khả năng cấp dòng trực tiếp cho buzzer, mạch sử dụng thêm Transistor làm khóa đóng ngắt trung gian. Tín hiệu từ GPIO 17 được đưa qua điện trở 1 k Ω đến cực nền của Transistor để điều khiển trạng thái hoạt động của buzzer sử dụng nguồn 5 VDC.

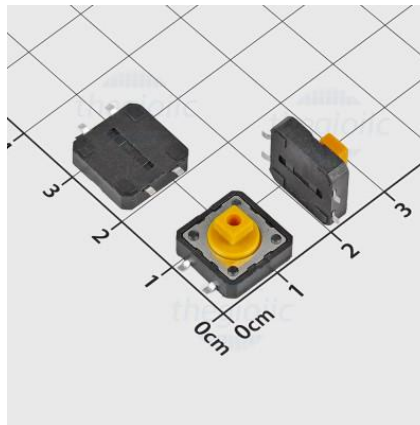
Khi GPIO 17 xuất mức logic cao, Transistor dẫn và cấp nguồn cho buzzer phát âm thanh cảnh báo. Ngược lại, khi GPIO 17 ở mức logic thấp, Transistor ngắt dẫn và buzzer dừng hoạt động. Cách thiết kế này giúp bảo vệ chân GPIO của Raspberry Pi 4 và đảm bảo khối báo động hoạt động ổn định.

3.3.4. Thiết kế khối reset hệ thống

Khởi reset hệ thống được thiết kế nhằm hỗ trợ khôi phục hoạt động của thiết bị khi xảy ra các sự cố như treo chương trình, lỗi phần mềm hoặc hệ thống không còn phản hồi trong quá trình vận hành. Khi gặp các tình huống này, người dùng có thể nhấn nút reset để thực hiện quá trình khởi động lại (reboot) Raspberry Pi 4, giúp hệ thống hoạt động trở lại mà không cần phải tháo hoặc ngắt nguồn cấp điện. Giải pháp này giúp quá trình vận hành thuận tiện hơn, đồng thời giảm nguy cơ ảnh hưởng đến dữ liệu và hệ điều hành do việc tắt nguồn đột ngột.

Hai loại nút nhấn cơ khí thường được sử dụng là nút nhấn giữ (Latching Switch) và nút nhấn nhả (Tactile Momentary Switch). Nút nhấn giữ có khả năng duy trì trạng thái sau mỗi lần tác động nên thường được sử dụng trong các ứng dụng đóng/ngắt nguồn hoặc lựa chọn chế độ hoạt động. Tuy nhiên, loại nút này không phù hợp với các hệ thống cần nhận biết sự kiện nhấn tức thời.

Ngược lại, nút nhấn nhả chỉ duy trì trạng thái đóng mạch trong thời gian người dùng tác động lực và tự động trở về trạng thái ban đầu khi thả tay. Nhờ khả năng tạo xung tín hiệu ngắn, rõ ràng cho mỗi lần nhấn, loại nút này được sử dụng rộng rãi trong các hệ thống điều khiển số. Do đó, nút nhấn nhả được lựa chọn để thực hiện chức năng reset trong hệ thống thiết kế.

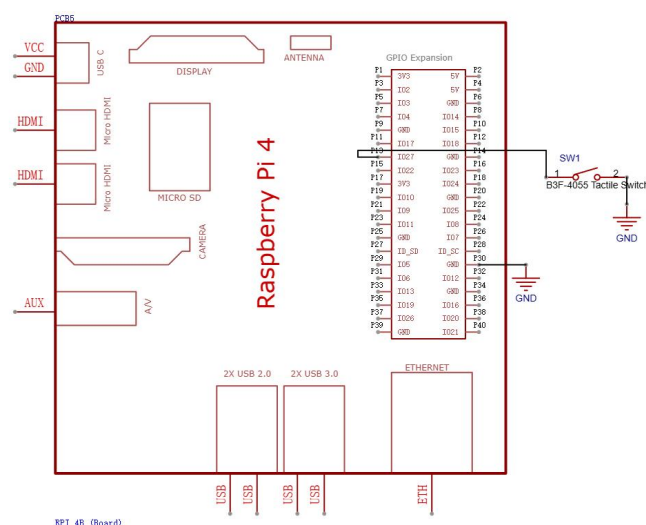


Hình 3.9. hình ảnh nút nhấn Tactile Switch B3F-4055

Bảng 3.4. Thông số kỹ thuật của nút nhấn B3F-4055

Thông số kỹ thuật	Mô tả
Tên linh kiện	B3F-4055 Tactile Switch
Số chân	4 chân
Kích thước thân	12 mm × 12 mm
Chiều cao nút nhấn	7.3 mm
Điện áp định mức	12 VDC
Dòng điện định mức	500 mA
Nhiệt độ hoạt động	-35°C đến +85°C

Từ các đặc điểm trên (**Bảng 3.4**) nhóm quyết định lựa chọn cấu kiện Nút nhấn nhả 4 chân xuyên lỗ làm nút nhấn thực thi cho khối này.

**Hình 3.10.** Sơ đồ mạch nút nhấn reset hệ thống kết nối raspberry pi 4

Mô tả cách kết nối (Hình 3.10):

Về mặt thiết kế phần cứng, khối nút nhấn được xây dựng theo hướng tối giản hóa linh kiện ngoại vi nhằm giảm độ phức tạp của mạch. Một đầu của nút nhấn SW1 được kết nối trực tiếp với chân GPIO27 của Raspberry Pi 4, trong khi đầu còn lại được nối xuống GND mà không cần sử dụng điện trở kéo hay tụ điện bên ngoài.

Để đảm bảo tín hiệu đầu vào luôn ổn định, chân GPIO27 được cấu hình kích hoạt điện trở kéo lên nội bộ (Internal Pull-up Resistor) tích hợp sẵn bên trong Raspberry Pi 4. Nhờ đó, khi nút nhấn ở trạng thái không tác động, chân GPIO luôn được duy trì ở mức logic HIGH (3,3V), tránh hiện tượng tín hiệu bị thả nổi và ảnh hưởng bởi nhiễu môi trường.

Trong quá trình hoạt động, khi người dùng nhấn SW1, tiếp điểm cơ khí đóng lại tạo đường dẫn xuống GND, kéo điện áp tại chân GPIO27 về mức 0V. Raspberry Pi 4 nhận diện sự kiện sườn xuống (Falling Edge) và kích hoạt tiến trình khởi động lại hệ thống theo chương trình đã được thiết lập.

3.3.5. Thiết kế khối relay

Khối relay đóng vai trò là tầng chấp hành trung gian, thực hiện chức năng đóng cắt nguồn cho các thiết bị tải công suất cao. Do tín hiệu điều khiển từ bộ xử lý trung tâm chỉ có điện áp và dòng điện nhỏ nên không thể điều khiển trực tiếp tải, vì vậy relay được sử dụng để đảm bảo khả năng đóng cắt và cách ly giữa mạch điều khiển với mạch công suất.

Các giải pháp đóng ngắt tải phổ biến hiện nay gồm transistor MOSFET, rơ-le bán dẫn (SSR) và rơ-le điện cơ. MOSFET có tốc độ đóng ngắt cao, hoạt động êm và tuổi thọ lớn, tuy nhiên không tạo được sự cách ly điện hoàn toàn giữa mạch điều khiển và mạch tải, đồng thời có thể phát sinh nhiệt khi làm việc với tải lớn. SSR sử dụng công nghệ bán dẫn kết hợp cách ly quang, có độ bền cao, không gây tiếng ồn cơ khí nhưng giá thành tương đối cao và vẫn tồn tại dòng rò khi ở trạng thái ngắt.

Rơ-le điện cơ là giải pháp được sử dụng rộng rãi nhờ khả năng cách ly điện hoàn toàn giữa mạch điều khiển điện áp thấp và mạch tải điện áp cao. Ngoài ra, relay điện cơ có khả năng chịu dòng khởi động lớn, kết cấu đơn giản, dễ thay thế và chi phí thấp, phù hợp với các hệ thống điều khiển tải chiếu sáng ngoài trời.

Sau khi đánh giá các phương án, hệ thống lựa chọn rơ-le điện cơ Omron G5NB-1A-E-5VDC làm phần tử đóng cắt chính. Relay có khả năng chịu tải tiếp điểm đến 5A, đáp ứng tốt yêu cầu điều khiển các tải điện xoay chiều hoặc tải điện một chiều có công suất phù hợp.



Hình 3.11. Hình ảnh thực tế cấu kiện Rơ-le điện cơ Omron G5NB-1A-E-5VDC sử dụng trong hệ thống.

Bảng 3.5. Thông số kỹ thuật của rơ-le điện từ Omron G5NB-1A-E-5VDC

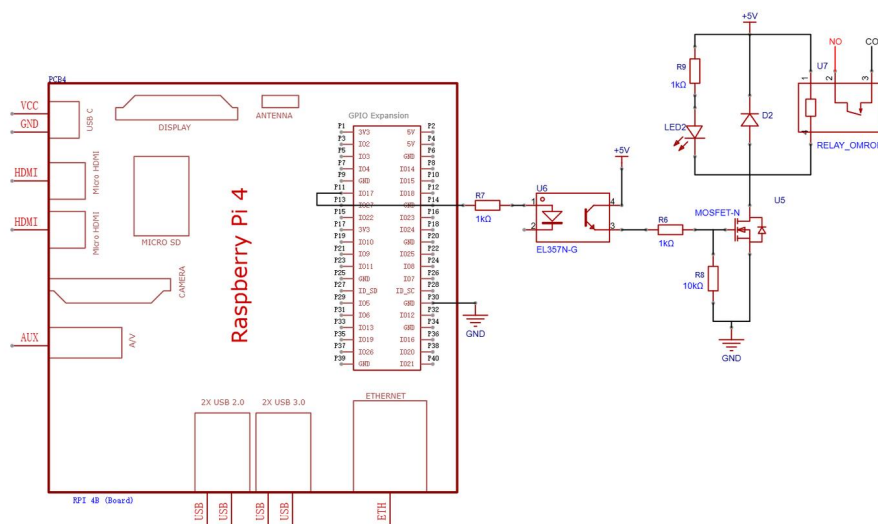
Thông số kỹ thuật	Mô tả
Điện áp cuộn dây kích	5 VDC
Số cực	1 cực
Số chân	4 chân
Dòng điện tải tối đa	5 A
Công suất đóng cắt tối đa DC	90 W
Công suất đóng cắt tối đa AC	1,25 kVA
Chiều dài	20,5 mm
Chiều rộng	7,2 mm
Chiều cao	15,3 mm

Từ các đặc điểm trên (**Bảng 3.5**), nhóm quyết định lựa chọn relay G5NB-1A-E-5VDC cho khối này.

Để đảm bảo khả năng cách ly, khuếch đại tín hiệu điều khiển và bảo vệ linh kiện công suất, mạch relay được bổ sung các linh kiện phụ trợ gồm bộ cách ly quang EL357N-G, MOSFET kênh N, diode dập xung ngược, LED chỉ thị trạng thái và các điện trở hạn dòng, điện trở kéo xuống. Các linh kiện này giúp bảo vệ GPIO của Raspberry Pi 4, tăng độ ổn định của tín hiệu điều khiển và nâng cao độ tin cậy cho toàn bộ hệ thống.

Bảng 3.6. Thông số kỹ thuật linh kiện phụ trợ

Tên linh kiện	Thông số
Optocoupler	EL357N-G (SOP-4)
MOSFET	N-Channel MOSFET
Diode dập xung	1N4148, 100V / 0.3A
LED chỉ thị	LED đỏ 5 mm
R7, R6, R9	1 k Ω
R8	10 k Ω



Hình 3.12. Sơ đồ nguyên lý mạch kích Relay của hệ thống

Mô tả cách kết nối (Hình 3.12):

Sơ đồ nguyên lý khối relay được thiết kế với Raspberry Pi 4 là bộ điều khiển trung tâm phát tín hiệu điều khiển từ chân GPIO đến mạch cách ly quang EL357N thông qua điện trở hạn dòng. Tín hiệu sau khi được cách ly sẽ điều khiển MOSFET đóng vai trò tăng công suất để kích cuộn dây relay hoạt động. Song song với cuộn dây relay là diode dập xung ngược nhằm bảo vệ linh kiện khi relay ngắt, đồng thời LED chỉ thị kết hợp điện trở hạn dòng được sử dụng để hiển thị trạng thái kích hoạt của relay. Tiếp điểm thường hở (NO) và chân chung (COM) của relay được kết nối với tải chiếu sáng, cho phép thực hiện chức năng đóng hoặc ngắt nguồn cấp cho tải theo tín hiệu điều khiển từ Raspberry Pi 4, đồng thời đảm bảo cách ly giữa mạch điều khiển điện áp thấp và mạch tải công suất.

3.3.6. Thiết kế khối chiếu sáng

Khối chiếu sáng được sử dụng nhằm hỗ trợ camera thu nhận hình ảnh trong điều kiện thiếu sáng hoặc vào ban đêm. Do hệ thống được triển khai ngoài trời, thiết bị chiếu sáng cần có độ sáng đủ lớn và khả năng hoạt động ổn định trong môi trường thực tế.

Các phương án chiếu sáng có thể sử dụng gồm đèn LED 5V, đèn LED 12V, đèn hồng ngoại và đèn LED AC 220V. Đèn LED 5V có ưu điểm dễ cấp nguồn nhưng công suất thấp, chỉ phù hợp với phạm vi nhỏ. Đèn LED 12V cho độ sáng cao hơn nhưng cần nguồn cấp riêng. Đèn hồng ngoại hỗ trợ quan sát ban đêm nhưng hiệu quả phụ thuộc vào khả năng hỗ trợ của camera. Trong khi đó, đèn LED AC 220V có độ sáng cao, tận dụng trực tiếp nguồn điện lưới và phù hợp với các vị trí lắp đặt ngoài trời.

Sau khi đánh giá các phương án, nhóm lựa chọn đèn LED 30W sử dụng nguồn AC 220V cho khối chiếu sáng. Loại đèn này có khả năng hoạt động ổn định, phạm vi chiếu sáng rộng và đáp ứng tốt yêu cầu hỗ trợ camera

trong điều kiện thiếu sáng.

Đèn được điều khiển thông qua relay. Raspberry Pi 4 xuất tín hiệu điều khiển relay để đóng hoặc ngắt nguồn 220V AC cấp cho đèn. Giải pháp này cho phép bật/tắt đèn từ xa, đồng thời cách ly hoàn toàn mạch điều khiển điện áp thấp với mạch công suất điện áp cao, đảm bảo an toàn cho hệ thống.



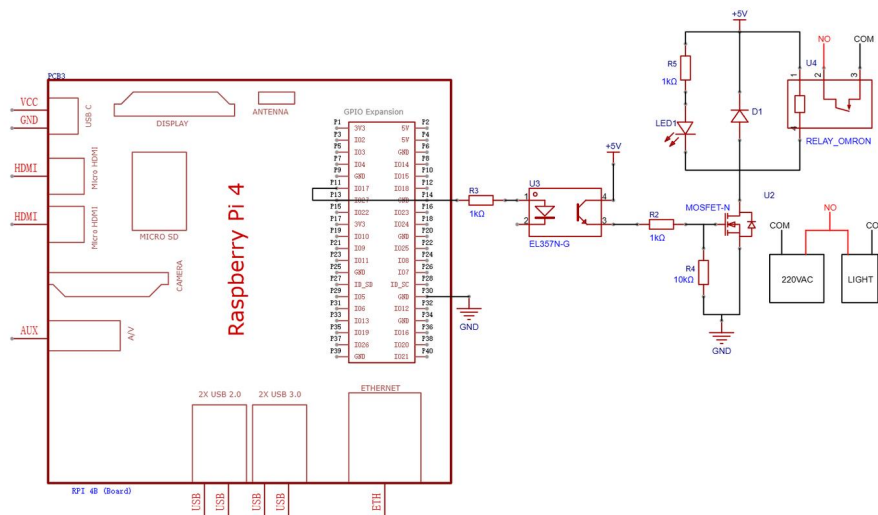
Hình 3.13. Đèn pha LED 30W ngoài trời sử dụng trong khối chiếu sáng

Thông số kỹ thuật

Bảng 3.7. Thông số kỹ thuật của đèn LED 30W

Thông số kỹ thuật	Mô tả
Model	FL 5054 30W
Công suất	30W
Điện áp	220V/50Hz
Quang thông	2700 lm
Kích thước	196 × 172 × 52 mm
Khả năng chống nước	IP66

Từ các thông số kỹ thuật của thiết bị chiếu sáng đã lựa chọn, nhóm tiến hành thiết kế sơ đồ nguyên lý kết nối giữa Raspberry Pi 4 và khối chiếu sáng. Mạch sử dụng tầng cách ly quang kết hợp với MOSFET và relay điện từ để thực hiện chức năng đóng/ngắt nguồn cấp cho đèn LED, đảm bảo khả năng điều khiển ổn định và cách ly an toàn giữa mạch điều khiển điện áp thấp với tải công suất



độ ổn định khi hoạt động liên tục đều được xem xét nhằm đảm bảo hệ thống vận hành ổn định, kể cả khi nhiều tải được kích hoạt đồng thời.

3.3.7.1. Phân tích nhu cầu tải và lựa chọn giải pháp nguồn cấp

Khi thiết kế khối nguồn, nhóm đã xét đến các yếu tố như điện áp đầu vào, điện áp đầu ra, dòng tải tối đa, khả năng ổn áp, tổn hao công suất và độ ổn định khi hệ thống hoạt động liên tục. Ngoài ra, mạch nguồn cần có khả năng cấp dòng dự phòng để đảm bảo hệ thống vẫn hoạt động ổn định khi các khối như buzzer hoặc relay được kích hoạt đồng thời.

Thông số chi tiết về dòng tiêu thụ của hệ thống

Bảng 3.8. Bảng ước tính dòng tiêu thụ của các thành phần trong hệ thống

Thành phần	Điện áp hoạt động	Dòng tiêu thụ
Raspberry Pi 4	5V	3000mA
Webcam USB	5V	300mA
Buzzer 1209	5V	22mA
Relay 5V	5V	40mA
Opto cách ly EL357N	5V	khoảng 2–5mA
LED báo trạng thái	5V	khoảng 2–5mA
Mạch reset hệ thống	5V	khoảng 1–5mA
Tổng dòng tiêu thụ ước tính	5V	khoảng 3370–3380mA

Từ bảng tính toán (**Bảng 3.8**), tổng dòng tiêu thụ cực đại của hệ thống vào khoảng 3.38A. Trong đó, Raspberry Pi 4 là thành phần tiêu thụ công suất lớn nhất, còn các thiết bị ngoại vi như webcam USB, relay, buzzer, optocoupler và LED báo trạng thái chỉ chiếm một phần nhỏ của tổng tải.

Để đáp ứng yêu cầu cấp nguồn cho toàn bộ hệ thống, nhóm lựa chọn module nguồn AC-DC HLK-20M05 với điện áp đầu ra 5VDC và dòng đầu ra cực đại 4A, tương đương công suất đầu ra 20W. So với tổng dòng tiêu thụ tính toán 3.38A, nguồn vẫn còn khoảng 0.62A dự phòng, đủ để bù cho các sai số tính toán, sự dao động tải ngắn hạn và đảm bảo hệ thống hoạt động ổn định khi các khối chức năng được kích hoạt đồng thời.



Hình 3.15. Module nguồn AC-DC HLK-20M05 sử dụng trong khối nguồn

Bảng 3.9. Thông số kỹ thuật của bộ nguồn 5VDC 4A

Thông số kỹ thuật	Mô tả
Điện áp đầu vào	100 ~ 240VAC
Công suất đầu ra	20W
Điện áp đầu ra	5VDC
Dòng đầu ra tối đa	4A
Nhiệt độ hoạt động	-25°C đến +60°C
Độ ẩm hoạt động	5% ~ 95% RH
Kích thước (D × R × C)	57 × 33 × 22.5 mm
Khối lượng	81g
Điện áp cách ly	3000VAC

Từ các đặc điểm đã phân tích (**Bảng 3.9**), nhóm lựa chọn module nguồn AC-DC HLK-20M05 cho hệ thống. Mặc dù module đã tích hợp sẵn mạch chuyển đổi và các cơ chế bảo vệ cơ bản, nhưng để nâng cao độ an toàn và chất lượng nguồn cấp, nhóm bổ sung thêm các mạch bảo vệ và lọc nhiễu.

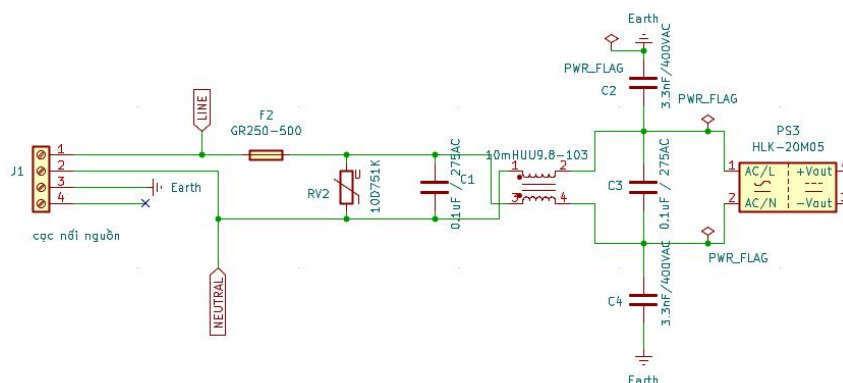
Cụ thể, tầng đầu vào AC được trang bị cầu chì, Varistor chống quá áp và mạch lọc EMI nhằm hạn chế ảnh hưởng của nhiễu và các xung điện áp từ lưới điện. Ở đầu ra DC, các tụ lọc được sử dụng để giảm gợn sóng điện áp, ổn định nguồn cấp và đảm bảo hệ thống hoạt động tin cậy.

3.3.7.2. Thiết kế Tầng bảo vệ và lọc nhiễu đầu vào AC

Sơ đồ mạch điện tầng bảo vệ và lọc nhiễu đầu vào AC được thiết kế nhằm mục đích xử lý các sự cố về quá dòng, quá áp đột biến và lọc nhiễu EMI từ lưới điện trước khi cấp vào chân Input của module nguồn chính.

Bảng 3.10. Thông số cấu hình lựa chọn của các linh kiện kỹ thuật

Tên linh kiện kỹ thuật	Thông số cấu hình lựa chọn
Cọc nối nguồn	4-pin, Pitch 5.08mm (L–N–E)
Cầu Chì Tự Phục Hồi	250V–500mA
Tụ Chống Sét Varistor10D751K	750V
Tụ Keo Vàng X2	275VAC
Cuộn Cảm UU9.8-103	10mH
Tụ gốm cao Y2	3.3 nF–400VAC



Hình 3.16. Sơ đồ mạch tầng bảo vệ và lọc nhiễu đầu vào AC-DC HLK-20M05 sử dụng trong khối nguồn

Điện áp lưới 220 VAC được đưa vào bo mạch thông qua cọc kết nối nguồn (J1). Để đảm bảo an toàn và giảm ảnh hưởng của nhiễu điện từ, mạch đầu vào AC được trang bị các linh kiện bảo vệ và lọc nhiễu như sau:

- **Cọc kết nối nguồn (J1):** Tiếp nhận nguồn điện lưới gồm dây Line, Neutral và Earth, đồng thời đảm bảo kết nối cơ khí chắc chắn và an toàn điện.
- **Cầu chì bảo vệ (F2):** Được mắc nối tiếp trên đường nguồn vào nhằm ngắt mạch khi xảy ra quá dòng hoặc ngắn mạch, bảo vệ các khối phía sau khỏi hư hỏng.
- **Varistor (RV2):** Mắc song song với nguồn vào, có nhiệm vụ hấp thụ các xung quá áp đột biến từ lưới điện như sét lan truyền hoặc nhiễu đóng cắt tải.
- **Tụ an toàn X2 (C1, C3):** Được mắc giữa hai đường Line và Neutral để giảm nhiễu vi sai và hạn chế nhiễu phát xạ từ bộ nguồn.

- **Cuộn cảm lọc nhiễu đồng pha (Common Mode Choke):** Giảm nhiễu đồng pha trên đường nguồn AC, góp phần đáp ứng yêu cầu tương thích điện từ (EMI).
- **Tụ an toàn Y2 (C2, C4):** Kết hợp với cuộn cảm lọc nhiễu, tạo đường dẫn cho các thành phần nhiễu cao tần xuống tiếp địa, giúp cải thiện chất lượng nguồn cấp.

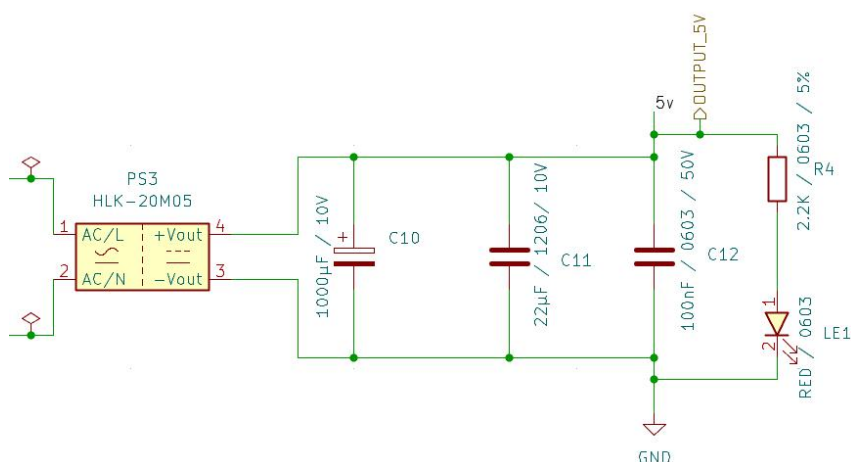
Nhờ sự kết hợp của các linh kiện trên, tầng đầu vào AC có khả năng bảo vệ hệ thống trước các sự cố điện áp và giảm đáng kể nhiễu từ lưới điện trước khi cấp nguồn cho module AC-DC.

3.3.7.3. Thiết kế Tầng lọc nguồn mịn đầu ra DC

Mạch điện tầng lọc nguồn DC đầu ra được thiết kế nhằm san phẳng gợn sóng điện áp tần số cao phát sinh trong quá trình chuyển đổi công suất của module. Hệ thống lọc đầu ra sử dụng giải pháp phối hợp song song giữa các tụ hóa dung lượng lớn và tụ gốm bypass để tối ưu hóa trở kháng nguồn ở các dải tần số khác nhau, đảm bảo điện áp ổn định cấp cho vi xử lý Raspberry Pi 4 và các ngoại vi khác.

Bảng 3.11. Thông số cấu hình lựa chọn của các linh kiện điện tử

Tên linh kiện kỹ thuật	Thông số cấu hình lựa chọn
Tụ hóa nhôm	100 μ F / 10V
Tụ gốm đa lớp	22 μ F / 10V
Tụ gốm dán	100nF / 50V
Điện trở dán màng oxit	2.2k Ω $\pm$ 1%
Đèn LED	2.2V



Hình 3.17. Sơ đồ mạch lọc phối hợp làm mịn điện áp ngõ ra 5VDC

Điện áp 5 VDC từ ngõ ra của module HLK-20M05 được đưa qua tầng lọc và hiển thị trạng thái nguồn nhằm đảm bảo chất lượng nguồn cấp cho hệ thống.

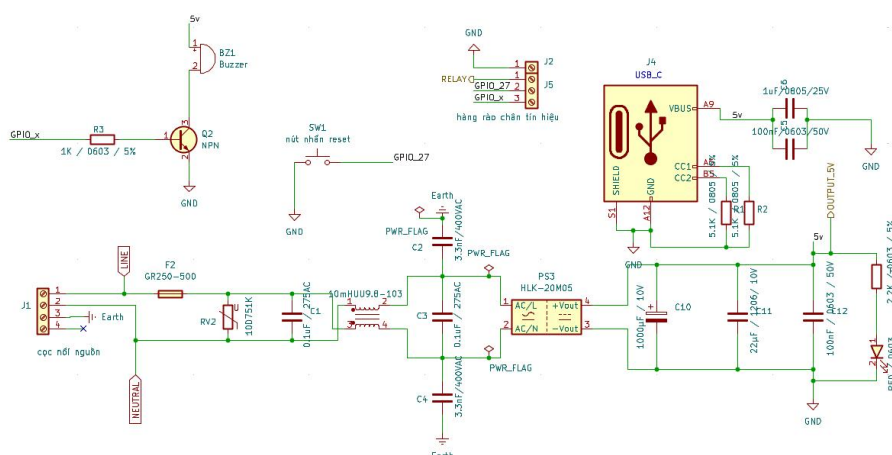
- **Tụ hóa lọc nguồn C10:** Được mắc song song tại đầu ra nguồn, có nhiệm vụ giảm gợn sóng điện áp và cung cấp dòng tức thời khi tải thay đổi đột ngột.
- **Mạng tụ decoupling C11 và C12:** Gồm tụ 22 μ F và 100 nF mắc song song giữa đường 5 V và GND. Các tụ này giúp lọc nhiễu ở nhiều dải tần khác nhau, ổn định điện áp nguồn và nâng cao chất lượng nguồn cấp cho các khối xử lý.
- **LED báo nguồn LE1 và điện trở hạn dòng R4:** Được sử dụng để hiển thị trạng thái hoạt động của nguồn 5 V, giúp dễ dàng kiểm tra tình trạng cấp nguồn của hệ thống.

Nhờ các linh kiện lọc và hiển thị trạng thái nguồn, điện áp đầu ra được ổn định hơn trước khi cung cấp cho các khối chức năng trong hệ thống.

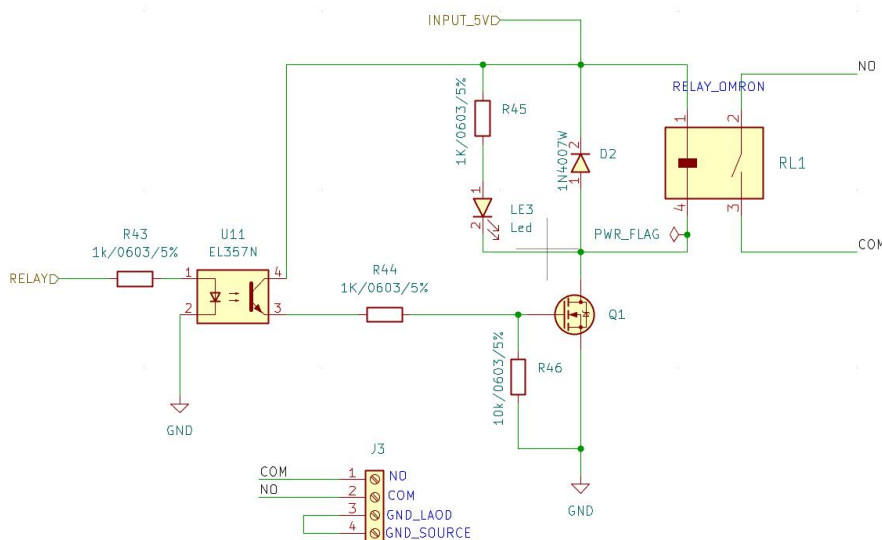
3.3.7.4. Tầng mạch giao tiếp và điều khiển ngoại vi

Trong hệ thống, các chân GPIO của Raspberry Pi 4 hoạt động ở mức logic 3,3 V và không thể điều khiển trực tiếp các tải công suất như relay hoặc buzzer. Do đó, tầng mạch giao tiếp ngoại vi được thiết kế nhằm thực hiện chức năng đệm công suất, cách ly tín hiệu điều khiển và bảo vệ GPIO khỏi các nhiễu điện từ hoặc xung điện áp phát sinh trong quá trình đóng cắt tải. Ngoài ra, tầng mạch này còn tích hợp các chức năng điều khiển relay, điều khiển còi báo động và tiếp nhận tín hiệu từ nút nhấn reset.

Sau khi hoàn thiện thiết kế và tích hợp các khối chức năng, bao gồm khối bảo vệ đầu vào AC, khối chuyển đổi nguồn AC-DC, khối lọc và ổn định điện áp đầu ra DC, cùng khối giao tiếp điều khiển ngoại vi, sơ đồ nguyên lý tổng thể của bo mạch quản lý nguồn được xây dựng và thể hiện như trong hình dưới đây :



Hình 3.18. Sơ đồ nguyên lý tổng thể board quản lý nguồn và giao tiếp ngoại vi



Hình 3.19. Sơ đồ nguyên lý tổng thể board quản lý nguồn và giao tiếp ngoại vi

Từ sơ đồ nguyên lý (**Hình 3.18** và **Hình 3.19**), tầng giao tiếp ngoại vi bao gồm các khối chức năng chính như sau:

- **Mạch đóng ngắt relay cách ly quang:** Tín hiệu điều khiển từ Raspberry Pi được đưa qua Optocoupler EL357N để cách ly giữa mạch điều khiển và mạch công suất. Khi có tín hiệu kích, MOSFET điều khiển relay đóng hoặc ngắt tải thông qua các tiếp điểm COM–NO. Diode D2 được sử dụng để bảo vệ mạch khỏi xung điện áp ngược sinh ra từ cuộn dây relay.
- **Mạch kích khối báo động:** Tín hiệu GPIO từ Raspberry Pi được đưa qua điện trở hạn dòng đến transistor Q2 để điều khiển buzzer 5 V. Khi GPIO ở mức cao, transistor dẫn và kích hoạt buzzer phát cảnh báo; khi GPIO ở mức thấp, buzzer ngừng hoạt động.
- **Mạch nút nhấn Reset:** Nút nhấn SW1 được kết nối với chân GPIO27, cho phép khởi động lại hệ thống khi cần thiết. Mạch sử dụng điện trở kéo nội của Raspberry Pi và xử lý chống dội phím bằng phần mềm để đảm bảo tín hiệu ổn định.
- **Cổng xuất nguồn USB Type-C:** Cổng USB Type-C J4 được sử dụng để cấp nguồn 5 VDC cho Raspberry Pi 4. Các tụ lọc nguồn được bố trí nhằm giảm nhiễu và ổn định điện áp, trong khi các điện trở cầu hình CC1 và CC2 đảm bảo cổng hoạt động đúng theo chuẩn USB Type-C.

Các khối trên giúp Raspberry Pi 4 thực hiện điều khiển thiết bị ngoại vi, phát cảnh báo, khởi động lại hệ thống và cấp nguồn ổn định cho quá trình vận hành.

3.3.7.5. Thiết kế Layout và chế tạo bo mạch thực tế

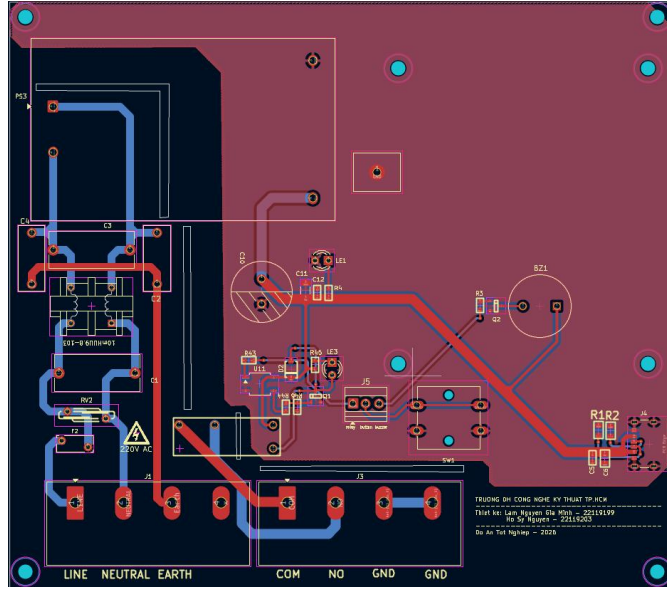
Sau khi hoàn thiện sơ đồ nguyên lý, nhóm tiến hành thiết kế layout PCB trên vật liệu FR-4 hai lớp, độ dày 1.6 mm. Quá trình bố trí linh kiện và đi dây được thực hiện theo các nguyên tắc thiết kế phần cứng nhằm đảm bảo khả năng chịu dòng, an toàn điện và giảm nhiễu cho hệ thống.

Tối ưu độ rộng đường mạch theo dòng tải: Các đường mạch được thiết kế với kích thước phù hợp theo chức năng sử dụng. Đường nguồn 220 VAC sử dụng bề rộng 1.5 mm để đảm bảo an toàn và khả năng chịu dòng. Đường nguồn chính 5 VDC cấp cho Raspberry Pi 4 được mở rộng lên 2.0 mm nhằm giảm sụt áp và tổn thất trên đường dẫn. Các nhánh nguồn phụ sử dụng bề rộng 0.6 mm, trong khi các đường tín hiệu logic được thiết kế trong khoảng 15–20 mil (0.38–0.5 mm), để tối ưu không gian bố trí.

Phân vùng cách ly điện áp: Bo mạch được chia thành hai khu vực riêng biệt gồm phần điện áp cao 220 VAC và phần điện áp thấp 5 VDC. Đồng thời, một rãnh cách ly cơ học (isolation slot) được bố trí dưới module nguồn AC-DC nhằm tăng khoảng cách cách điện và nâng cao độ an toàn cho hệ thống.

Phủ đồng mặt phẳng GND: Các vùng trống trên PCB được đổ đồng và kết nối với mass chung nhằm giảm nhiễu điện từ (EMI), cải thiện chất lượng tín hiệu và tăng độ ổn định trong quá trình hoạt động.

Sau khi hoàn thành các bước thiết kế, sơ đồ nguyên lý được chuyển sang môi trường thiết kế vật lý trên phần mềm KiCad để xây dựng mô hình PCB hoàn chỉnh. Kết quả thiết kế được trình bày trong các hình vẽ tiếp theo.



Hình 3.20. Bản vẽ thiết kế đường mạch Layout 2D của bo mạch trên phần mềm

3.3.7.6. Kiểm nghiệm tính khả thi và độ bền vận hành của nguồn cấp

Để khẳng định board mạch thiết kế có thể vận hành ổn định và liên tục trong thực tế, nhóm tiến hành hai bước kiểm nghiệm độc lập: cân bằng công suất ở cấp độ hệ thống và kiểm tra mật độ dòng điện ở cấp độ đường mạch PCB.

Cân bằng công suất toàn hệ thống

Tổng công suất tiêu thụ cực đại của hệ thống được xác định theo công thức:

$$P_{max} = P_{TT} + P_{NV} \quad (3.1)$$

Trong đó:

- P_{max} : Tổng công suất tiêu thụ cực đại của toàn hệ thống.
- P_{TT} : Công suất tiêu thụ của khối xử lý trung tâm.
- P_{NV} : Công suất tiêu thụ của các linh kiện ngoại vi.

Khối xử lý trung tâm Raspberry Pi 4 vận hành ở mức điện áp 5 V, dòng tiêu thụ đỉnh khi xử lý các tác vụ nặng là $I_{Pi} = 3,0 A$. Khi đó, công suất tiêu thụ của khối xử lý trung tâm được tính như sau:

$$P_{TT} = 5 V \times 3,0 A = 15 W \quad (3.2)$$

Các linh kiện ngoại vi bao gồm cuộn dây kích rơ-le với dòng tiêu thụ $I_{relay} = 40 mA$, còi báo BZ1 với dòng tiêu thụ $I_{buzzer} = 22 mA$ và LED chỉ thị D3 với dòng tiêu thụ $I_{led} = 5 mA$. Công suất tiêu thụ của các linh kiện ngoại vi được xác định như sau:

$$P_{NV} = 5 V \times (0,04 + 0,022 + 0,005) A = 0,335 W \quad (3.3)$$

Từ đó, tổng công suất tiêu thụ cực đại của toàn hệ thống là:

$$P_{max} = 15 + 0,335 = 15,335 W \quad (3.4)$$

Module nguồn HLK-20M05 được nhóm lựa chọn có công suất định mức $P_{nguồn} = 20 W$, tương ứng với ngõ ra

5 V – 4 A. Hệ số tải thực tế của nguồn được tính theo công thức:

$$\eta = \frac{P_{max}}{P_{nguồn}} = \frac{15,335}{20} \approx 76,675\% \quad (3.5)$$

Trong đó:

- η : Hệ số tải thực tế của nguồn.
- $P_{nguồn}$: Công suất định mức của module nguồn.

Hệ số tải này nhỏ hơn ngưỡng 80%, phù hợp với nguyên tắc thiết kế nguồn có dự phòng trong các hệ thống điện tử hoạt động liên tục. Do đó, nguồn vẫn còn khoảng 23% công suất dự phòng để đáp ứng các xung tải tức thời, hạn chế nguy cơ sụt áp hoặc quá nhiệt trong quá trình vận hành.

Đánh giá khả năng chịu tải của đường mạch PCB

Việc đánh giá khả năng chịu tải của đường mạch PCB được thực hiện dựa theo tiêu chuẩn IPC-2152, với độ dày lớp đồng 1 oz xấp xỉ 0,035 mm.

Đường nguồn chính 5V có chiều rộng thiết kế 2,0 mm. Theo tiêu chuẩn IPC-2152, đường mạch có chiều rộng 2,0 mm với lớp đồng 1 oz có khả năng tải liên tục đến khoảng 4,5 A với mức tăng nhiệt bề mặt $\Delta T \leq 10^\circ C$.

Tại ngõ ra kết nối USB-C, do ràng buộc về khoảng cách vật lý giữa các chân A9 và B9, nhóm không thể duy trì chiều rộng đường mạch 2,0 mm. Vì vậy, đường nguồn được tách thành hai đường song song, mỗi đường rộng 0,9 mm dẫn vào một chân riêng biệt. Tổng khả năng dẫn dòng của cặp đường song song này tương đương một đường đơn rộng khoảng 1,8 mm, vẫn đảm bảo chịu tải liên tục trên 3,0 A theo tiêu chuẩn IPC-2152.

Dòng điện thực tế lớn nhất trên đường nguồn chính là 3,0 A, tương đương khoảng 67% khả năng chịu tải thiết kế. Điều này cho thấy đường mạch vẫn còn biên độ an toàn, hạn chế hiện tượng sụt áp cục bộ do điện trở ký sinh của vết đồng.

Đường nguồn ngoại vi có chiều rộng thiết kế 0,6 mm. Các đường cấp điện cho còi BZ1, nút nhấn và mạch kích cách ly quang có dòng tải thực tế không vượt quá 50 mA. Trong khi đó, đường mạch rộng 0,6 mm có khả năng chịu tải định mức lên đến khoảng 1,5 A. Mức dự phòng lớn này đảm bảo độ bền cơ học lâu dài của vết đồng và loại trừ nguy cơ quá nhiệt hoặc bong tróc đường mạch trong quá trình sử dụng.

Phân tích thiết kế cách ly vùng cao áp

Nhóm bố trí rãnh xẻ cơ học (*isolation slot*) xuyên qua lớp nền FR-4 tại ranh giới giữa phân khu 220 V AC và phân khu 5 V DC nhằm tăng khoảng cách phóng điện bề mặt. Giải pháp này giúp hạn chế dòng rò phát sinh do độ ẩm hoặc bụi tích tụ theo thời gian, đặc biệt quan trọng trong môi trường lắp đặt thực tế.

Bên cạnh đó, việc tách biệt rõ ràng giữa vùng cao áp và vùng thấp áp còn góp phần bảo vệ các linh kiện bán dẫn nhạy cảm như optocoupler EL357N, Raspberry Pi và các mạch GPIO khỏi nguy cơ hư hỏng do nhiễu điện hoặc sự cố điện áp cao.

3.4. THIẾT KẾ PHẦN MỀM HỆ THỐNG

Trong hệ thống phát hiện và cảnh báo xâm nhập qua hành động leo rào, phần mềm có nhiệm vụ thu nhận và phân tích hình ảnh từ camera để đưa ra quyết định cảnh báo. Sau khi nhận khung hình, hệ thống thực hiện các bước phát hiện người, cắt vùng ảnh chứa đối tượng, phân loại hành động và kích hoạt cảnh báo khi phát hiện xâm nhập.

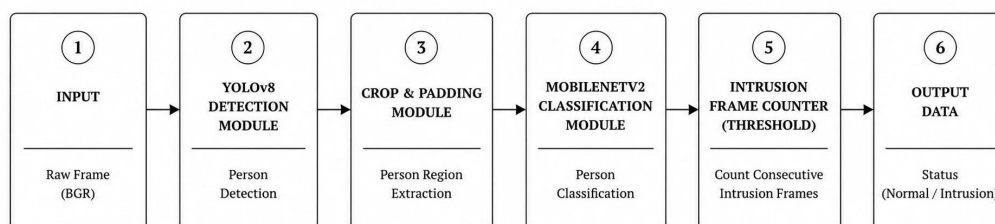
Để thực hiện các chức năng trên, hệ thống được xây dựng theo kiến trúc hai giai đoạn, kết hợp giữa YOLOv8 và MobileNetV2. Trong đó, YOLOv8 thực hiện phát hiện người và xác định vị trí đối tượng, còn MobileNetV2 thực hiện phân loại hành động thành hai trạng thái: Normal và Intrusion. Cách tiếp cận này giúp tập trung xử lý vào vùng chứa đối tượng, nâng cao hiệu quả nhận diện và giảm khối lượng tính toán.

Hệ thống lựa chọn YOLOv8n cho giai đoạn phát hiện do có kích thước nhỏ, tốc độ xử lý nhanh và phù hợp với khả năng tính toán của Raspberry Pi 4. Đối với giai đoạn phân loại, MobileNetV2 được lựa chọn nhờ kiến

trúc gọn nhẹ, chi phí tính toán thấp và vẫn đảm bảo độ chính xác cho bài toán phân loại hành vi. Sự kết hợp giữa hai mô hình giúp cân bằng giữa độ chính xác, tốc độ xử lý và khả năng triển khai trên thiết bị nhúng.

3.4.1. Kiến trúc mô hình AI

Hệ thống AI được xây dựng theo kiến trúc xử lý tuần tự (pipeline), trong đó dữ liệu hình ảnh đầu vào được xử lý qua nhiều module chuyên biệt để thực hiện nhiệm vụ phát hiện và phân loại hành vi xâm nhập. Mỗi module đảm nhận một chức năng riêng, đồng thời dữ liệu đầu ra của module trước sẽ trở thành dữ liệu đầu vào cho module kế tiếp, tạo thành một chuỗi xử lý liên tục và khép kín



Hình 3.21. Sơ đồ khối kiến trúc mô hình AI của hệ thống

Kiến trúc của mô hình AI bao gồm sáu khối chức năng chính: Input, YOLOv8 Detection Module, Crop Padding Module, MobileNetV2 Classification Module, Intrusion Frame Counter và Output Data. Quá trình xử lý bắt đầu từ việc thu nhận khung hình từ camera, sau đó sử dụng mô hình YOLOv8 để phát hiện đối tượng người trong ảnh. Vùng chứa đối tượng được cắt và chuẩn hóa kích thước thông qua module Crop Padding trước khi được đưa vào mô hình MobileNetV2 để phân loại trạng thái

Để nâng cao tính ổn định của hệ thống và hạn chế các cảnh báo giả do nhiễu hoặc sai số của từng khung hình đơn lẻ, kết quả phân loại tiếp tục được xử lý bởi Intrusion Frame Counter. Module này thực hiện việc đếm số lượng khung hình liên tiếp được nhận diện là Intrusion và chỉ đưa ra kết quả cảnh báo khi đạt ngưỡng xác định trước. Cuối cùng, hệ thống xuất ra Status của đối tượng dưới hai trạng thái Normal hoặc Intrusion, làm cơ sở cho các chức năng cảnh báo và giám sát của hệ thống.

3.4.2. Dữ liệu đầu vào

Dữ liệu đầu vào của hệ thống là chuỗi khung hình (frame) được thu nhận liên tục từ camera webcam độ phân giải Full HD 1080p (1920×1080) kết nối với Raspberry Pi 4 thông qua giao tiếp USB. Camera có nhiệm vụ giám sát khu vực cần bảo vệ và truyền hình ảnh theo thời gian thực đến hệ thống xử lý AI.

Trong quá trình hoạt động, Raspberry Pi 4 sử dụng thư viện OpenCV để đọc từng khung hình từ webcam. Mỗi khung hình sau khi được thu nhận sẽ được lưu dưới định dạng ảnh BGR (Blue–Green–Red), là định dạng mặc định của OpenCV và phù hợp với các bước xử lý tiếp theo trong pipeline AI.

Luồng dữ liệu đầu vào này là cơ sở cho toàn bộ quá trình nhận diện của hệ thống. Các khung hình sau khi được thu nhận sẽ được chuyển trực tiếp đến YOLOv8 Detection Module để thực hiện phát hiện đối tượng người, trước khi tiếp tục trải qua các bước cắt vùng ảnh, chuẩn hóa dữ liệu và phân loại trạng thái xâm nhập ở các module tiếp theo.

3.4.3. Thiết kế module phát hiện người bằng YOLOv8

Module phát hiện người có nhiệm vụ xác định vị trí người trong khung hình và cung cấp dữ liệu đầu vào cho bước phân loại hành động. Độ chính xác của module này ảnh hưởng trực tiếp đến chất lượng ảnh crop và kết quả nhận diện hành vi xâm nhập.

Trong hệ thống, mô hình YOLOv8n được chuyển đổi sang định dạng ONNX và thực thi bằng ONNX Runtime trên Raspberry Pi 4 nhằm tối ưu tốc độ suy luận. Mặc dù YOLOv8 có khả năng phát hiện nhiều lớp đối tượng, hệ thống chỉ giữ lại kết quả của lớp person để tập trung vào bài toán giám sát xâm nhập và giảm khối lượng xử lý cho các bước phía sau.

Hai ngưỡng lọc quan trọng được thiết lập cho bước phát hiện gồm:

- $CONF_THRESH = 0.4$: ngưỡng confidence tối thiểu để chấp nhận một detection. Các bounding box có confidence thấp hơn ngưỡng này sẽ bị loại bỏ nhằm tránh đưa các vùng ảnh nhiễu hoặc không chắc chắn vào mô hình phân loại.
- $IOU_THRESH = 0.45$: ngưỡng IoU được sử dụng trong quá trình Non-Maximum Suppression (NMS) để loại bỏ các bounding box bị trùng lặp. Giá trị này giúp hệ thống chỉ giữ lại một bounding box đại diện cho mỗi người khi có nhiều box chồng chéo nhau.

Như vậy, module phát hiện người bằng YOLOv8 có nhiệm vụ xác định chính xác vị trí người trong từng frame video. Kết quả đầu ra của module này là các bounding box chứa người, đóng vai trò làm dữ liệu đầu vào cho bước crop ảnh và phân loại hành động bằng MobileNetV2.

3.4.4. Thiết kế module crop và padding vùng người

3.4.4.1. Crop vùng người

Sau khi YOLOv8 trả về danh sách bounding box, hệ thống thực hiện cắt vùng ảnh tương ứng từ khung hình gốc. Mỗi bounding box được biểu diễn dưới dạng tọa độ (x_1, y_1, x_2, y_2) , trong đó (x_1, y_1) là góc trên bên trái và (x_2, y_2) là góc dưới bên phải của vùng chứa người.

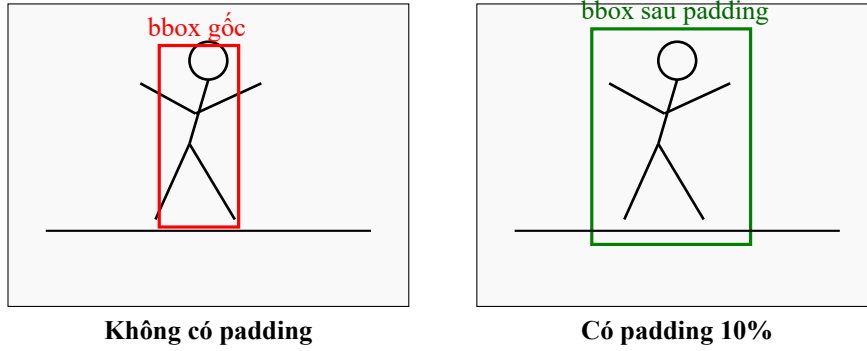
Trước khi cắt, các tọa độ được kiểm tra và giới hạn trong phạm vi kích thước khung hình thực tế để tránh lỗi truy xuất ngoài biên ảnh:

$$\begin{aligned}x_1 &= \max(0, x_1), & y_1 &= \max(0, y_1), \\x_2 &= \min(width, x_2), & y_2 &= \min(height, y_2).\end{aligned}$$

Ảnh crop thu được sau bước này chứa toàn bộ vùng người mà YOLOv8 phát hiện, và sẽ được đưa tiếp vào bước mở rộng padding trước khi đưa vào classifier.

3.4.4.2. Padding bounding box

Một vấn đề thực tế thường gặp khi dùng bounding box từ detector là vùng crop có thể bị cắt sát quá mức, làm mất đi các bộ phận quan trọng ở rìa như đầu, tay, chân của người, đặc biệt trong tư thế leo rào khi tay và chân thường vươn ra ngoài vùng thân chính. Để khắc phục vấn đề này, hệ thống áp dụng thêm một bước mở rộng padding quanh bounding box trước khi thực hiện crop.



Hình 3.22. Minh họa tác dụng của padding bounding box.

Cụ thể, với mỗi bounding box có chiều rộng bw và chiều cao bh , lượng mở rộng được tính theo tỷ lệ phần trăm so với kích thước của chính bounding box đó:

$$pad_x = bw \times \text{BOX_PADDING},$$

$$pad_y = bh \times \text{BOX_PADDING}.$$

Sau đó, tọa độ mới được tính như sau:

$$x_{1,new} = \max(0, x_1 - pad_x),$$

$$y_{1,new} = \max(0, y_1 - pad_y),$$

$$x_{2,new} = \min(width, x_2 + pad_x),$$

$$y_{2,new} = \min(height, y_2 + pad_y).$$

Với $\text{BOX_PADDING} = 0.10$, mỗi cạnh của bounding box được mở rộng thêm 10% so với kích thước gốc của vùng người. Tỷ lệ này được lựa chọn sau thực nghiệm dựa trên sự cân bằng giữa hai yêu cầu đối lập:

- Mở rộng đủ nhiều để đảm bảo giữ lại toàn bộ cơ thể người trong vùng crop, tránh mất thông tin ở rìa đặc biệt trong tư thế leo rào với tay vươn cao.
- Không mở rộng quá nhiều để tránh đưa quá nhiều nền cảnh vào ảnh crop, vì nền cảnh không mang thông tin về hành động và có thể gây nhiễu cho quá trình phân loại của MobileNetV2.

Sau khi mở rộng, tọa độ được giới hạn trong phạm vi kích thước khung hình gốc để đảm bảo không truy xuất vùng ảnh ngoài biên.

3.4.5. Thiết kế mô hình phân loại MobileNetV2

MobileNetV2 là thành phần trung tâm của hệ thống, đảm nhận nhiệm vụ phân loại hành động của người từ ảnh crop. Việc thiết kế mô hình cần cân bằng giữa độ chính xác phân loại và tốc độ inference, đặc biệt trong bối cảnh triển khai trên thiết bị nhúng có tài nguyên hạn chế như Raspberry Pi 4.

3.4.5.1. Đầu vào của mô hình

Dữ liệu dùng để huấn luyện mô hình MobileNetV2 được tổ chức theo cấu trúc thư mục chuẩn của thư viện torchvision, sử dụng lớp ImageFolder để tự động gán nhãn dựa trên tên thư mục con. Cụ thể, dataset được chia thành hai tập train và val, mỗi tập gồm hai lớp phân loại là *intrusion* và *normal*:

```
dataset/
  train/
```

```

intrusion/      ← ảnh crop người đang leo rào
normal/         ← ảnh crop người đi lại bình thường
val/
intrusion/
normal/

```

Đầu vào của mô hình MobileNetV2 là ảnh màu RGB chứa vùng crop người, được trích xuất từ kết quả phát hiện của YOLOv8. Ảnh đầu vào trải qua các bước tiền xử lý gồm crop, resize, chuyển sang tensor và chuẩn hoá trước khi đưa vào mô hình.

Kích thước ảnh đầu vào được thiết kế khác nhau giữa hai giai đoạn:

- Trong huấn luyện, ảnh được resize về 320×320 pixel để cung cấp đủ chi tiết về tư thế và hành động của người, hỗ trợ mô hình học các đặc trưng phân biệt giữa hai lớp Intrusion và Normal.
- Trong inference và export ONNX, ảnh được resize về 224×224 pixel — kích thước chuẩn của MobileNetV2 nhằm giảm chi phí tính toán, phù hợp với khả năng xử lý của Raspberry Pi 4.

Trước khi đưa vào mô hình, ảnh được chuyển sang tensor và chuẩn hoá theo từng kênh màu RGB bằng giá trị trung bình và độ lệch chuẩn:

$$x' = \frac{x - \mu}{\sigma} \quad (3.6)$$

Trong đó, x là giá trị pixel sau khi được đưa về khoảng $[0, 1]$, μ là giá trị trung bình và σ là độ lệch chuẩn tương ứng với từng kênh màu. Cụ thể, hệ thống sử dụng:

$$\mu = [0.485, 0.456, 0.406] \quad (3.7)$$

$$\sigma = [0.229, 0.224, 0.225] \quad (3.8)$$

Đây là các giá trị thống kê của tập dữ liệu ImageNet, phù hợp với mô hình MobileNetV2 đã được huấn luyện trước. Việc chuẩn hoá ảnh theo cùng phân phối dữ liệu với giai đoạn huấn luyện ban đầu giúp mô hình khai thác tốt hơn các đặc trưng đã học, đồng thời làm cho quá trình huấn luyện và suy luận ổn định hơn.

3.4.5.2. Đầu ra của mô hình

Đầu ra của mô hình là một vector gồm hai giá trị, tương ứng với hai lớp cần phân loại trong bài toán:

$$\mathbf{y} = [y_{normal}, y_{intrusion}] \quad (3.9)$$

Trong đó, y_{normal} biểu diễn mức độ tin cậy ảnh đầu vào thuộc lớp *Normal*, còn $y_{intrusion}$ biểu diễn mức độ tin cậy ảnh đầu vào thuộc lớp *Intrusion*. Sau khi đi qua hàm *softmax*, hai giá trị này được chuyển thành xác suất:

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^2 e^{z_j}} \quad (3.10)$$

Trong đó, z_i là giá trị đầu ra trước softmax của lớp thứ i , còn P_i là xác suất dự đoán tương ứng sau softmax. Tổng xác suất của hai lớp luôn bằng 1:

$$P_{normal} + P_{intrusion} = 1 \quad (3.11)$$

Quyết định phân loại cuối cùng được xác định bằng cách chọn lớp có xác suất lớn nhất:

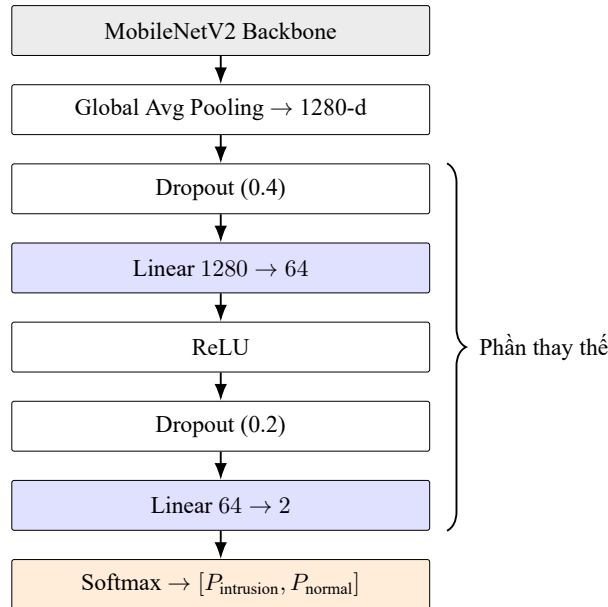
$$\hat{y} = \arg \max(P_{normal}, P_{intrusion}) \quad (3.12)$$

Nếu xác suất của lớp *Intrusion* lớn hơn xác suất của lớp *Normal*, ảnh crop được xem là có dấu hiệu xâm nhập. Ngược lại, nếu xác suất của lớp *Normal* lớn hơn, hệ thống xem hành động trong ảnh là bình thường. Tuy nhiên, trong quá trình xử lý video thực tế, kết quả của từng frame không được dùng độc lập để kích hoạt cảnh báo ngay lập tức. Thay vào đó chỉ phát cảnh báo khi lớp *Intrusion* xuất hiện liên tiếp trong một số khung hình nhất định. Cách làm này giúp giảm cảnh báo sai do các frame nhiễu, tư thế trung gian hoặc lỗi phân loại tạm thời của mô hình.

$$[P(\text{intrusion}), P(\text{normal})]$$

3.4.5.3. Thiết kế classifier head

Phần classifier head mặc định của MobileNetV2 được thay thế hoàn toàn bằng một cấu trúc gọn hơn, được thiết kế riêng cho bài toán phân loại nhị phân của đề tài, như minh họa trong **Hình 3.23**.



Hình 3.23. Kiến trúc classifier head của MobileNetV2 trong hệ thống.

Thiết kế này được lựa chọn dựa trên các lý do sau:

Thứ nhất, bài toán phân loại nhị phân chỉ có hai lớp nên không đòi hỏi một classifier head quá rộng. Một lớp ẩn duy nhất với 64 nơ-ron đã đủ capacity để học ranh giới quyết định giữa hai lớp normal và intrusion, trong khi giữ số tham số ở mức tối thiểu.

Thứ hai, head nhỏ giúp giảm thời gian inference đáng kể so với head mặc định rộng hơn, điều này đặc biệt quan trọng khi chạy trên Raspberry Pi 4 không có GPU.

Thứ ba, hai lớp Dropout với tỷ lệ 0.4 và 0.2 đóng vai trò điều chuẩn (regularization) giúp mô hình tổng quát hoá tốt hơn, hạn chế hiện tượng overfitting trong trường hợp dataset có kích thước hạn chế.

Về chiến lược đóng băng tham số (partial freezing), trong giai đoạn đầu huấn luyện, các lớp features[0] đến features[7] của backbone được đóng băng hoàn toàn, tức là trọng số được giữ nguyên từ pretrained và không cập nhật. Chỉ có features[8] trở về sau và toàn bộ classifier head mới được huấn luyện. Từ epoch thứ hai trở đi, toàn bộ backbone được mở băng để fine-tune sâu hơn. Chiến lược này giúp quá trình hội tụ ổn định hơn ở giai đoạn đầu, tận dụng tốt các đặc trưng cấp thấp đã được học từ ImageNet như cạnh, góc, kết cấu bề mặt, đồng thời cho phép mô hình thích nghi dần với đặc trưng riêng của bài toán leo rào.

3.4.6. Thiết kế module Intrusion Frame Counter

Sau khi mô hình MobileNetV2 thực hiện phân loại từng khung hình thành hai trạng thái Normal hoặc Intrusion, kết quả này chưa được sử dụng ngay để đưa ra cảnh báo. Nguyên nhân là do trong quá trình hoạt động thực tế, mô hình AI có thể xuất hiện các dự đoán sai trên một số khung hình riêng lẻ do ảnh hưởng của nhiễu, thay đổi ánh sáng, chuyển động nhanh hoặc góc quan sát không thuận lợi. Nếu hệ thống cảnh báo ngay khi phát hiện một khung hình Intrusion, tỷ lệ cảnh báo giả (False Positive) sẽ tăng lên đáng kể.

Để khắc phục vấn đề này, hệ thống sử dụng Intrusion Frame Counter làm lớp xử lý hậu kỳ (Post-processing). Module này có nhiệm vụ theo dõi kết quả phân loại của các khung hình liên tiếp và thực hiện đếm số lượng khung hình được nhận diện là Intrusion.

Mỗi khi MobileNetV2 trả về kết quả Intrusion, bộ đếm sẽ tăng thêm một đơn vị. Ngược lại, nếu kết quả là Normal, bộ đếm sẽ được đặt lại về giá trị ban đầu. Khi số lượng khung hình Intrusion liên tiếp đạt đến một ngưỡng được thiết lập trước (Threshold), hệ thống sẽ xác nhận rằng sự kiện xâm nhập thực sự đã xảy ra và chuyển trạng thái đầu ra thành Intrusion để kích hoạt các chức năng cảnh báo.

Cơ chế này hoạt động như một bộ lọc theo thời gian (Temporal Filtering), giúp loại bỏ các dự đoán sai xuất hiện ngẫu nhiên trên từng khung hình riêng lẻ, đồng thời chỉ xác nhận xâm nhập khi hành vi được duy trì liên tục trong một khoảng thời gian nhất định. Nhờ đó, hệ thống đạt được độ ổn định cao hơn, giảm số lượng cảnh báo giả nhưng vẫn đảm bảo khả năng phát hiện nhanh các đối tượng xâm nhập thực tế.

3.4.7. Thiết kế khối output

Sau khi trải qua các bước phát hiện đối tượng, cắt vùng ảnh, phân loại và xử lý ổn định theo thời gian bằng Intrusion Frame Counter, hệ thống sẽ sinh ra dữ liệu đầu ra (Output Data) để phục vụ các chức năng giám sát và cảnh báo.

Dữ liệu đầu ra của mô hình AI được biểu diễn dưới dạng Status, phản ánh trạng thái hiện tại của đối tượng trong vùng giám sát. Kết quả này bao gồm hai giá trị chính:

- **Normal:** Đối tượng được xác định là không có hành vi xâm nhập hoặc số lượng khung hình liên tiếp được phân loại là Intrusion chưa đạt ngưỡng xác nhận của hệ thống.
- **Intrusion:** Hệ thống xác nhận có hành vi xâm nhập sau khi số lượng khung hình liên tiếp được phân loại là Intrusion đạt hoặc vượt ngưỡng đã thiết lập trong module Intrusion Frame Counter.

Kết quả Status được cập nhật liên tục theo từng chu kỳ xử lý của hệ thống, bảo đảm phản ánh chính xác trạng thái hiện tại của khu vực giám sát theo thời gian thực. Dữ liệu này đóng vai trò là đầu ra cuối cùng của pipeline AI và là cơ sở để các module chức năng khác của hệ thống thực hiện các tác vụ như kích hoạt cảnh báo, hiển thị trạng thái trên giao diện quản lý hoặc lưu trữ thông tin sự kiện khi cần thiết.

Việc chuẩn hóa dữ liệu đầu ra thành hai trạng thái Normal và Intrusion giúp đơn giản hóa quá trình giao tiếp giữa mô hình AI và các thành phần khác trong hệ thống, đồng thời nâng cao khả năng mở rộng và tích hợp với các chức năng giám sát trong tương lai.

3.4.8. Thiết kế giao diện quản lý

3.4.8.1. Thiết kế chức năng hệ thống phần mềm

Hệ thống phần mềm được xây dựng theo mô hình phân lớp, gồm hai thành phần chính: khối xử lý AI chạy trên Raspberry Pi 4 và giao diện quản lý trên nền tảng Web PWA.

Chức năng giám sát và phát hiện xâm nhập: Hệ thống thu nhận hình ảnh từ camera và sử dụng mô hình YOLOv8 kết hợp MobileNetV2 để phát hiện người và phân loại hành động. Để giảm cảnh báo sai, cơ chế xác nhận theo nhiều khung hình liên tiếp được áp dụng trước khi kích hoạt báo động.

Chức năng truyền phát và lưu trữ video: Video giám sát được truyền lên giao diện Web thông qua giao thức HLS, hỗ trợ truy cập trong mạng nội bộ hoặc từ xa qua Internet. Hệ thống đồng thời ghi hình liên tục, tự động phân đoạn video và quản lý dung lượng lưu trữ.

Chức năng điều khiển thiết bị ngoại vi: Khi phát hiện xâm nhập, hệ thống tự động kích hoạt báo động. Ngoài ra, giao diện Web cho phép điều khiển thủ công còi báo động và đèn chiếu sáng, đồng thời đồng bộ trạng thái giữa phần cứng và giao diện quản lý.

Chức năng bảo mật và quản lý dữ liệu: Hệ thống yêu cầu đăng nhập tài khoản quản trị để truy cập. Các hình ảnh và video sự kiện được lưu trữ, cho phép xem lại, quản lý và xóa khi cần thiết. Đồng thời, hệ thống cung cấp các thông tin thống kê cơ bản như tổng số sự kiện xâm nhập và thời điểm phát hiện gần nhất.

Chức năng ứng dụng Web: Website được xây dựng theo mô hình PWA, cho phép cài đặt trực tiếp lên điện thoại hoặc máy tính như một ứng dụng độc lập, giúp truy cập nhanh và nâng cao trải nghiệm sử dụng.

3.4.8.2. Thiết kế ứng dụng người dùng

Để hướng tới sự tối giản và tối ưu hóa trải nghiệm quản trị, nhóm thiết kế giao diện hệ thống tập trung vào tính trực quan, giúp người dùng dễ dàng nắm bắt toàn bộ trạng thái an ninh và điều khiển phần cứng chỉ trên một màn hình làm việc. Hệ thống giao diện được phân chia thành ba trang chức năng chính như sau.

Giao diện đăng nhập: Thực hiện xác thực người dùng trước khi truy cập hệ thống, bao gồm các trường nhập tài khoản, mật khẩu và khu vực hiển thị thông báo lỗi.

Giao diện giám sát trung tâm: Hiển thị luồng video trực tiếp từ camera, trạng thái an ninh của hệ thống, các nút điều khiển thiết bị ngoại vi và thông tin thống kê nhanh về các sự kiện xâm nhập.

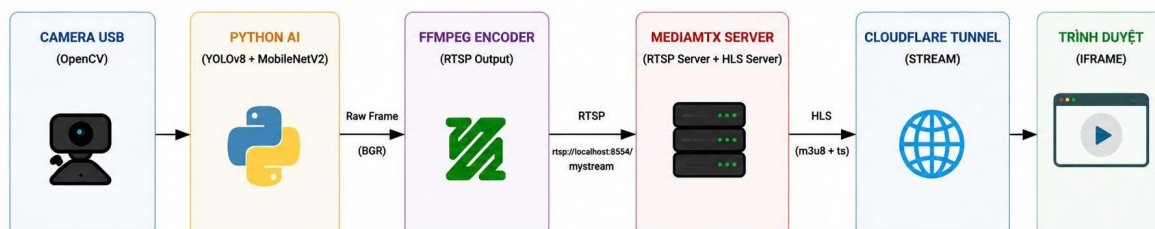
Giao diện thư viện dữ liệu: Cho phép xem lại hình ảnh và video đã được lưu trữ trong quá trình giám sát. Dữ liệu được sắp xếp theo thời gian, hỗ trợ tra cứu và quản lý thuận tiện.

3.4.9. Thiết kế hệ thống truyền phát video và truy cập từ xa

Để đáp ứng yêu cầu giám sát và điều khiển từ xa qua Internet, hệ thống phần mềm được thiết kế theo hai luồng xử lý độc lập nhưng hoạt động đồng thời trên Raspberry Pi 4. Mỗi luồng đảm nhận một chức năng riêng, bao gồm luồng xử lý và truyền video.

3.4.9.1. Luồng phát video

Luồng truyền phát video có nhiệm vụ tiếp nhận hình ảnh đã được xử lý bởi hệ thống AI, thực hiện mã hóa và truyền phát theo giao thức phù hợp để người dùng có thể theo dõi trực tiếp trên trình duyệt thông qua mạng Internet. Quá trình truyền phát sử dụng sự kết hợp giữa OpenCV, FFmpeg, MediaMTX và Cloudflare Tunnel, cho phép người dùng truy cập luồng video trực tiếp từ trình duyệt web.



Hình 3.24. Sơ đồ khối luồng truyền phát video của hệ thống

Trong hệ thống, camera USB liên tục thu nhận hình ảnh thực tế và cung cấp các khung hình cho chương trình Python. Bên trong chương trình, các luồng xử lý AI hoạt động song song để thực hiện phát hiện đối tượng

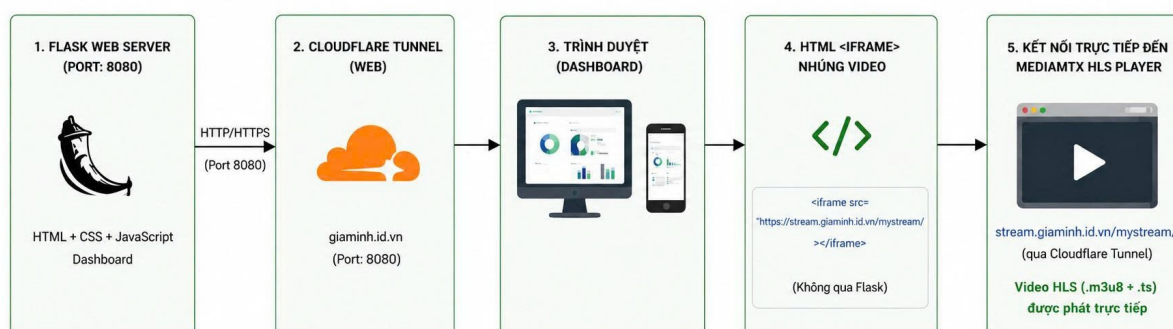
bằng mô hình YOLOv8 và phân loại trạng thái bằng MobileNetV2. Kết quả nhận dạng được cập nhật lên khung hình mới nhất dưới dạng các thông tin trực quan như trạng thái Normal hoặc Intrusion, khung bao đối tượng và các thông tin giám sát khác.

Khung hình sau khi được cập nhật kết quả AI sẽ được chuyển đến FFmpeg Encoder để mã hóa theo chuẩn H.264 và xuất thành luồng RTSP. Luồng RTSP này được gửi đến MediaMTX Server, nơi thực hiện chuyển đổi sang giao thức HLS (HTTP Live Streaming) bằng cách tạo các tệp .m3u8 và .ts phục vụ việc phát video trên trình duyệt.

Để hỗ trợ truy cập từ xa qua Internet mà không cần cấu hình mở cổng trên bộ định tuyến, hệ thống sử dụng Cloudflare Tunnel để công bố dịch vụ HLS ra mạng Internet thông qua tên miền công khai. Người dùng chỉ cần truy cập địa chỉ tương ứng để xem trực tiếp hình ảnh giám sát trên trình duyệt web. Toàn bộ quá trình thu nhận hình ảnh, suy luận AI và truyền phát video được thực hiện đồng thời bằng kiến trúc đa luồng (Multi-threading), nhằm đảm bảo khả năng xử lý gần thời gian thực và duy trì tốc độ truyền phát ổn định.

3.4.9.2. Thiết kế luồng giao diện Web Dashboard

Luồng giao diện Web Dashboard được xây dựng nhằm cung cấp cho người dùng một giao diện trực quan để giám sát và quản lý hệ thống từ xa thông qua trình duyệt web. Luồng này hoạt động độc lập với luồng truyền phát video, trong đó máy chủ Flask chỉ đảm nhiệm việc cung cấp giao diện Dashboard và không trực tiếp truyền tải dữ liệu video.



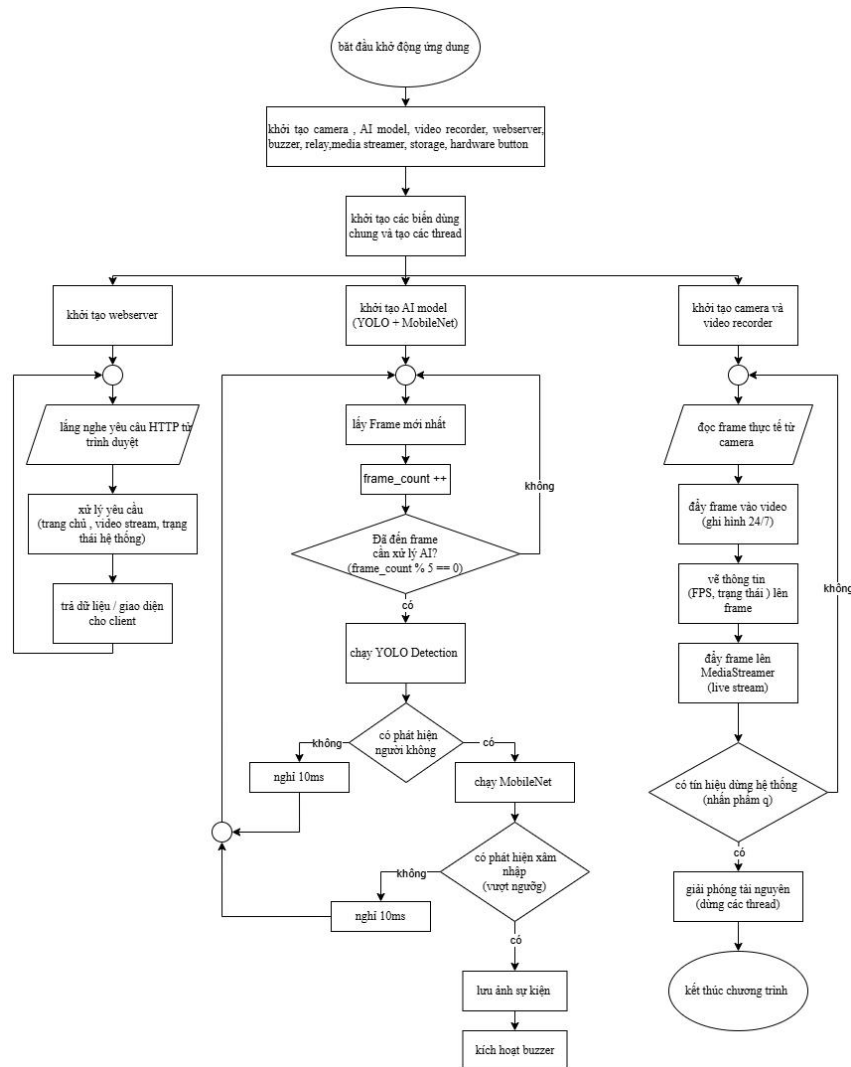
Hình 3.25. Sơ đồ khối luồng giao diện Web Dashboard của hệ thống

Khi người dùng truy cập tên miền của hệ thống thông qua Internet, yêu cầu sẽ được chuyển đến Flask Web Server đang chạy trên Raspberry Pi 4 thông qua Cloudflare Tunnel. Máy chủ Flask có nhiệm vụ cung cấp các thành phần giao diện như HTML, CSS và JavaScript để hiển thị Dashboard, đồng thời cung cấp các thông tin trạng thái và chức năng quản lý của hệ thống.

Sau khi giao diện Dashboard được tải hoàn tất, trang web sẽ sử dụng thẻ <iframe> để nhúng trình phát video HLS. Địa chỉ của <iframe> được cấu hình trỏ trực tiếp đến dịch vụ HLS do MediaMTX cung cấp thông qua Cloudflare Tunnel, do đó luồng video được phát trực tiếp đến trình duyệt mà không đi qua Flask, giúp giảm tải cho máy chủ và đảm bảo hiệu năng truyền phát ổn định ngay cả khi nhiều người dùng truy cập đồng thời.

3.5. THIẾT KẾ THUẬT TOÁN HOẠT ĐỘNG CỦA HỆ THỐNG

3.5.1. Lưu đồ giải thuật chương trình chính của hệ thống



Hình 3.26. Lưu đồ giải thuật chương trình chính của hệ thống

Nguyên lý hoạt động của hệ thống

Dựa trên lưu đồ giải thuật ở Hình 3.26, quá trình hoạt động của hệ thống được thực hiện theo các bước như sau :

Đầu tiên, khi chương trình được khởi động, hệ thống tiến hành khởi tạo toàn bộ các thành phần cần thiết bao gồm Camera, mô hình trí tuệ nhân tạo YOLO và MobileNet, bộ ghi hình Video Recorder, Web Server, Buzzer, Relay, Media Streamer, bộ nhớ lưu trữ và các nút nhấn phần cứng phục vụ điều khiển hệ thống.

Sau khi hoàn thành quá trình khởi tạo, chương trình tiếp tục tạo các biến dùng chung và khởi động các luồng xử lý độc lập để thực hiện nhiều tác vụ song song. Kiến trúc đa luồng này bao gồm ba luồng chính: luồng Web Dashboard, luồng AI Detection và luồng Main Camera & Recorder.

Luồng Web Dashboard : chịu trách nhiệm khởi tạo Web Server, lắng nghe các yêu cầu HTTP từ trình duyệt, xử lý các yêu cầu truy cập như hiển thị giao diện, phát trực tiếp hình ảnh từ camera và cập nhật trạng thái hệ thống. Sau khi xử lý, hệ thống trả dữ liệu hoặc giao diện tương ứng về phía người dùng.

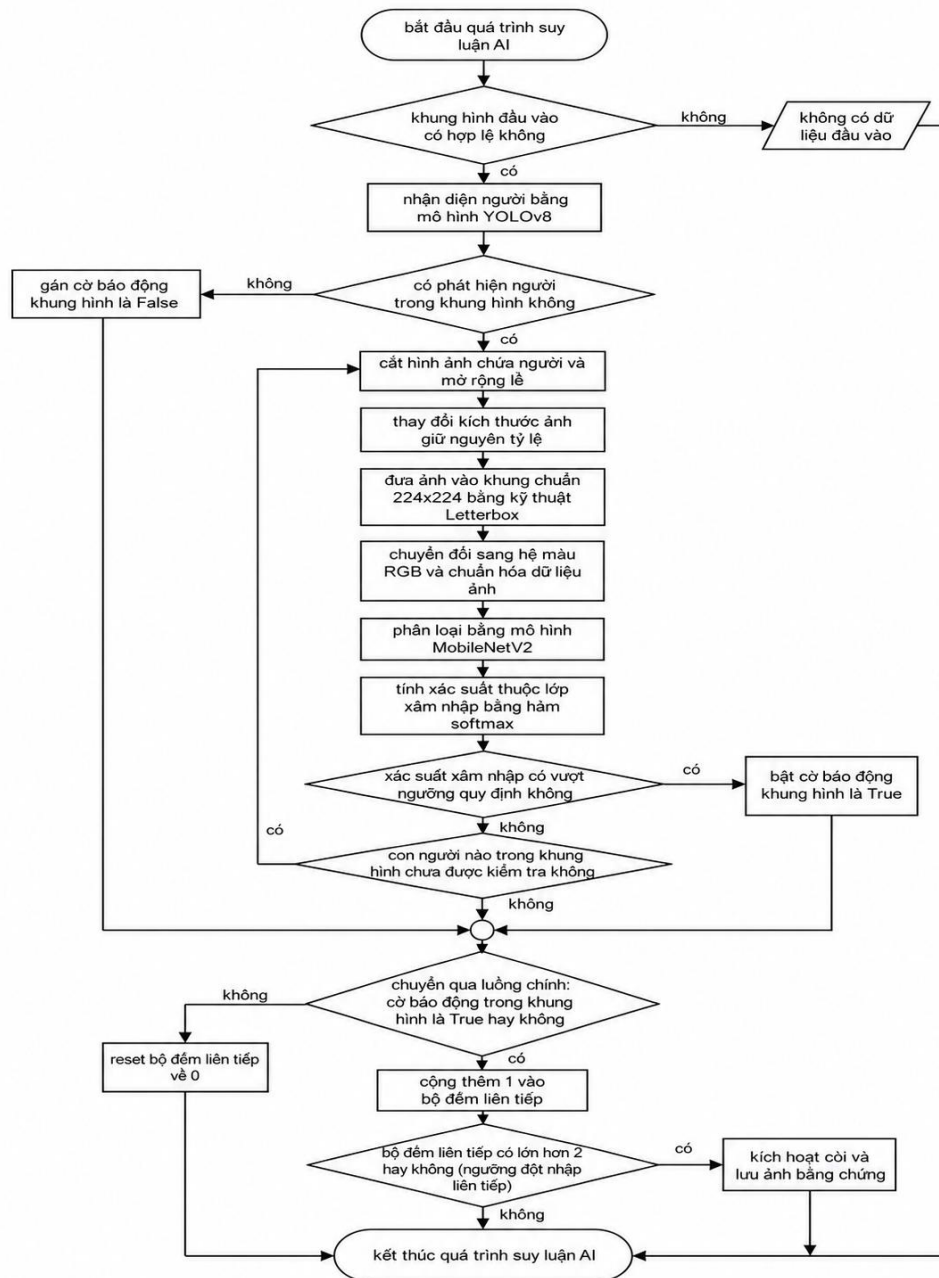
Luồng AI Detection: chịu trách nhiệm thực hiện nhận diện xâm nhập bằng trí tuệ nhân tạo. Luồng này

liên tục lấy khung hình mới nhất từ bộ đệm, tăng bộ đếm frame_count và chỉ thực hiện suy luận AI sau mỗi 5 khung hình nhằm giảm tải cho bộ xử lý. Khi đến thời điểm xử lý, mô hình YOLO được dùng để phát hiện người; nếu phát hiện, mô hình MobileNet sẽ phân loại để xác định đây là người bình thường hay xâm nhập. Trong trường hợp xác định có xâm nhập vượt ngưỡng quy định, hệ thống sẽ lưu ảnh sự kiện và kích hoạt còi cảnh báo Buzzer.

Luồng Main Camera & Recorder : đảm nhiệm việc đọc liên tục các khung hình từ Camera, ghi hình 24/7 vào Video Recorder, vẽ các thông tin trạng thái như tốc độ khung hình FPS lên ảnh và truyền dữ liệu đến Media Streamer để phục vụ chức năng Live Stream trên giao diện Web Dashboard. Luồng này hoạt động liên tục cho đến khi nhận được tín hiệu dừng hệ thống từ người dùng, lúc đó toàn bộ tài nguyên sẽ được giải phóng và các luồng dừng lại trước khi chương trình kết thúc hoàn toàn.

3.5.2. Lưu đồ giải thuật xử lý AI phát hiện xâm nhập

Hệ thống sử dụng pipeline suy luận AI hai giai đoạn gồm YOLOv8 để phát hiện vị trí đối tượng người trong khung hình và MobileNetV2 để phân loại hành vi xâm nhập trên từng vùng ảnh đó. Lưu đồ giải thuật được trình bày trong **Hình 3.27**.



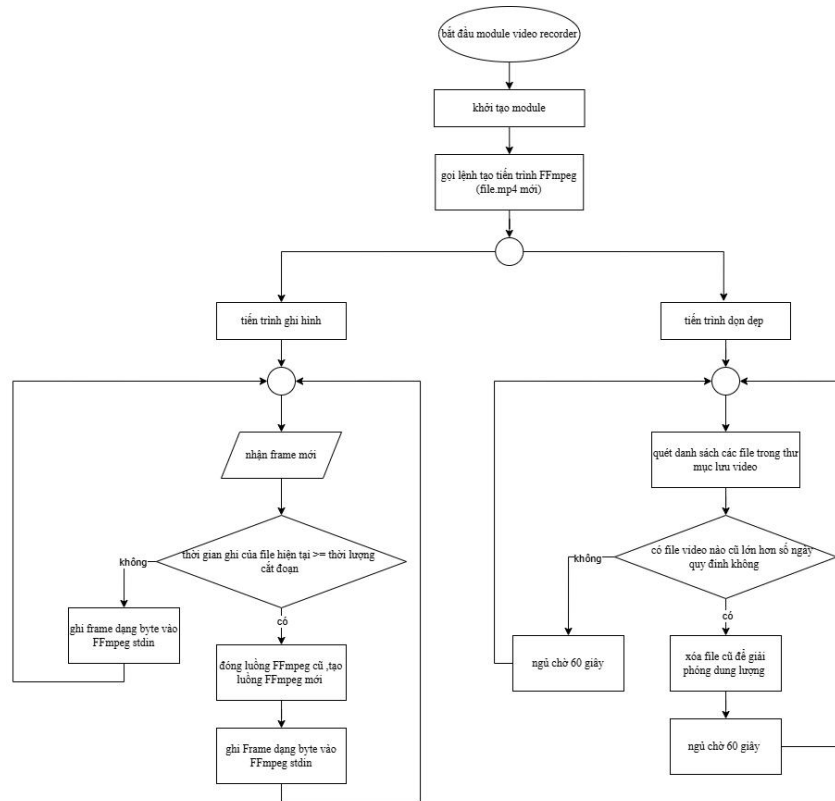
Hình 3.27. Lưu đồ giải thuật xử lý AI phát hiện xâm nhập của hệ thống

Khi nhận một khung hình đầu vào hợp lệ, hệ thống sử dụng YOLOv8 để phát hiện tất cả đối tượng người. Với mỗi đối tượng được phát hiện, vùng ảnh tương ứng được cắt riêng, mở rộng lề, thay đổi kích thước về chuẩn 224×224 bằng kỹ thuật Letterbox, sau đó chuyển sang không gian màu RGB và chuẩn hóa trước khi đưa vào MobileNetV2 phân loại. Xác suất đầu ra được tính qua hàm Softmax; nếu xác suất thuộc lớp xâm nhập vượt ngưỡng quy định, cờ báo động của khung hình được bật. Quá trình này lặp lại cho đến khi tất cả đối tượng trong khung hình được xử lý.

Sau khi hoàn tất toàn bộ đối tượng, hệ thống đánh giá cờ báo động của khung hình. Nếu có xâm nhập, bộ đếm liên tiếp tăng thêm một; ngược lại, bộ đếm được đặt lại về 0. Khi bộ đếm đạt ngưỡng số lần phát hiện liên tiếp quy định, hệ thống kích hoạt còi báo động và lưu ảnh bằng chứng.

3.5.3. Lưu đồ giải thuật module ghi hình 24/7 và tự động dọn dẹp bộ nhớ

Để đảm bảo hệ thống camera giám sát có khả năng ghi hình liên tục và duy trì dung lượng lưu trữ ổn định trong thời gian dài, hệ thống được thiết kế một mô-đun quản lý video chuyên biệt. Lưu đồ giải thuật của mô-đun được trình bày trong **Hình 3.28**.



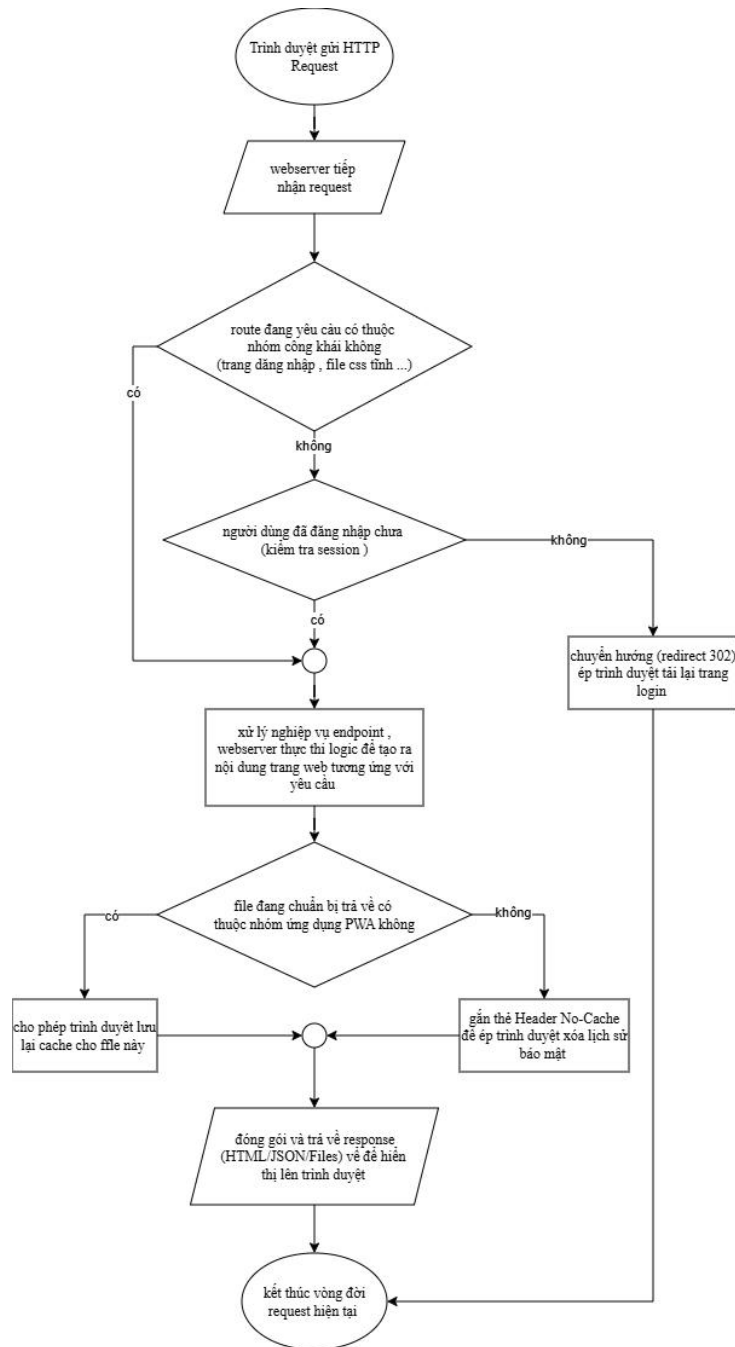
Hình 3.28. Lưu đồ giải thuật mô-đun ghi hình 24/7 và tự động dọn dẹp bộ nhớ

Sau khi khởi tạo, mô-đun tách thành hai tiến trình chạy song song. Tiến trình ghi hình liên tục nhận khung hình từ camera và mã hóa thành tệp video thông qua FFmpeg. Khi thời lượng ghi đạt ngưỡng cắt đoạn đã cấu hình, hệ thống tự động đóng tiến trình FFmpeg hiện tại và khởi tạo một tiến trình mới, chia dữ liệu thành các đoạn video có kích thước hợp lý, thuận tiện cho việc quản lý và trích xuất về sau. Song song đó, tiến trình dọn dẹp định kỳ quét thư mục lưu trữ và xóa các tệp video vượt quá số ngày lưu trữ quy định, sau mỗi chu kỳ sẽ tạm dừng 60 giây trước khi quét tiếp.

Thiết kế hai tiến trình độc lập này cho phép hệ thống vừa đảm bảo ghi hình liên tục 24/7, vừa tự động kiểm soát dung lượng lưu trữ mà không cần sự can thiệp của người dùng, hạn chế nguy cơ mất dữ liệu do đầy bộ nhớ.

3.5.4. Lưu đồ giải thuật vòng đời xử lý HTTP Request của Web Server

Web Dashboard được xây dựng trên nền tảng Python Flask theo mô hình xử lý HTTP Request – HTTP Response. Lưu đồ giải thuật vòng đời xử lý của Web Server được trình bày trong **Hình 3.29**.



Hình 3.29. Lưu đồ giải thuật vòng đời xử lý HTTP Request của Web Server

Khi nhận một HTTP Request, Web Server trước tiên kiểm tra Route yêu cầu có thuộc nhóm tài nguyên công khai hay không. Nếu có, yêu cầu được chuyển thẳng đến giai đoạn xử lý nghiệp vụ mà không cần xác thực. Ngược lại, hệ thống kiểm tra phiên đăng nhập (Session); nếu người dùng chưa xác thực, Web Server trả về Redirect HTTP 302 để chuyển hướng về trang đăng nhập và kết thúc vòng đời Request. Nếu đã đăng nhập, yêu cầu được chuyển tiếp đến khối xử lý nghiệp vụ tương ứng với Endpoint được gọi.

Trước khi đóng gói và trả về Response, hệ thống áp dụng chính sách bộ nhớ đệm: các tài nguyên thuộc nhóm PWA được phép lưu Cache để tăng tốc độ truy cập, trong khi các dữ liệu thông thường được gắn Header No-Cache nhằm đảm bảo bảo mật.

Cơ chế Middleware kết hợp Routing của Flask cho phép hệ thống xử lý thống nhất các yêu cầu xác thực, bảo vệ tài nguyên và quản lý bộ nhớ đệm, góp phần nâng cao tính bảo mật và khả năng mở rộng của Web Dashboard.

CHƯƠNG 4. XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

4.1. MÔI TRƯỜNG XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

Hệ thống được xây dựng trên hai môi trường chính: môi trường huấn luyện có tài nguyên GPU và môi trường triển khai tại biên trên Raspberry Pi 4. Việc tách hai môi trường giúp quá trình huấn luyện sử dụng được năng lực tính toán mạnh, trong khi thiết bị thực tế chỉ giữ các thành phần cần thiết cho suy luận, điều khiển phần cứng, ghi hình và phục vụ Web.

4.1.1. Môi trường phát triển phần mềm

Chương trình huấn luyện được xây dựng bằng Python 3.10+, PyTorch và Torchvision. OpenCV đảm nhận đọc, xử lý và ghi video; Ultralytics YOLO cung cấp mô hình YOLOv8; Scikit-learn tính các chỉ số đánh giá; Matplotlib và Seaborn hỗ trợ trực quan hóa kết quả. ONNX và ONNX Runtime được sử dụng trong giai đoạn chuyển đổi, tối ưu và triển khai mô hình. NumPy hỗ trợ xử lý mảng và chuẩn bị tensor đầu vào.

Quá trình huấn luyện có thể thực hiện trên máy tính cá nhân, Kaggle hoặc Google Colab. Trong nội dung triển khai của đề tài, Kaggle với GPU NVIDIA T4 được sử dụng để huấn luyện 20 epoch với batch size 24. Dataset, checkpoint, file ONNX và metadata được lưu lại trước khi chuyển sang Raspberry Pi.

Bảng 4.1. Môi trường phần mềm và phần cứng của hệ thống

Thành phần	Phiên bản / Cấu hình	Giai đoạn sử dụng
Python	3.12.12	Train + Test
PyTorch	2.10.0	Train + Test
Torchvision	0.25.0 tương ứng PyTorch	Train + Test
OpenCV	4.13.0	Triển khai thực tế
Ultralytics YOLO	8.4.75 (yolov8n.onnx)	Triển khai thực tế
Scikit-learn	1.6.1	Train (metrics)
Matplotlib / Seaborn	3.10.0	Train (biểu đồ)
ONNX Runtime	1.27.0	Export + RPi
Kaggle / Google Colab	GPU T4X2	Train
Raspberry Pi 4	RAM 4GB, OS 64-bit	Triển khai thực tế
Webcam	USB 1080p	Triển khai thực tế
GPIO library	gpiozero hoặc thư viện tương đương	Điều khiển buzzer, relay, reset
Flask / Web Backend	Flask, session, API trạng thái	Giám sát và điều khiển Web
FFmpeg / HLS	Ghi hình, chia đoạn và truyền video	Lưu trữ + Web
Buzzer	5V, khoảng 22mA	Cảnh báo tại chỗ
Relay và đèn	Relay 5V, đèn LED 30W/220VAC	Điều khiển chiếu sáng

4.1.2. Môi trường triển khai trên thiết bị nhúng

Thiết bị triển khai là Raspberry Pi 4 Model B RAM 4 GB, sử dụng Raspberry Pi OS 64-bit trên thẻ microSD 64 GB. Python runtime điều phối chương trình; OpenCV đọc camera; YOLOv8 phát hiện người; ONNX Runtime

chạy MobileNetV2 FLOAT32; thư viện GPIO điều khiển buzzer, relay và nhận tín hiệu reset. Flask, FFmpeg và HLS lần lượt phục vụ Web Dashboard, ghi hình và truyền video.

Các dịch vụ camera, AI, GPIO, ghi hình và Web được tổ chức thành các tiến trình hoặc luồng độc lập nhưng dùng chung trạng thái có kiểm soát. Cách tổ chức này giúp yêu cầu Web và thao tác ghi file không chặn vòng lặp suy luận chính.

4.1.3. Thiết bị phần cứng sử dụng

Bảng 4.2. Danh sách thiết bị và linh kiện chính được sử dụng

Thiết bị/linh kiện	Vai trò trong hệ thống
Raspberry Pi 4 Model B	Chạy AI, Web, ghi hình và điều khiển GPIO.
Webcam USB Full HD	Thu nhận hình ảnh khu vực hàng rào.
Buzzer 1209, 5 V	Phát cảnh báo âm thanh tại chỗ.
Relay 5 V và đèn LED 30 W	Đóng/ngắt khối chiếu sáng hỗ trợ camera.
Nút nhấn reset	Gửi tín hiệu phục hồi hệ thống qua GPIO27.
Board nguồn và ngoại vi	Chuyển đổi, lọc, phân phối nguồn và tích hợp tầng chấp hành.
Linh kiện phụ trợ	Điện trở, transistor NPN, optocoupler EL357N, MOSFET, diode đập xung, LED và terminal.

Nhìn chung, các thiết bị và linh kiện được lựa chọn đáp ứng đầy đủ các chức năng chính của hệ thống, bao gồm thu nhận hình ảnh, xử lý AI, hiển thị Web, ghi hình và phát cảnh báo. Trong đó, Raspberry Pi 4 đóng vai trò trung tâm, liên kết giữa khối camera, khối xử lý phần mềm và các ngoại vi cảnh báo thông qua GPIO.

4.2. XÂY DỰNG CHƯƠNG TRÌNH HUẤN LUYỆN MÔ HÌNH MOBILENETV2

Chương trình huấn luyện được xây dựng bằng PyTorch, thực hiện quy trình từ chuẩn bị dataset, tăng cường dữ liệu, tạo MobileNetV2, fine-tuning, đánh giá đến lưu checkpoint tốt nhất. Ảnh đầu vào của MobileNetV2 là ảnh người đã được crop, không phải ảnh toàn cảnh từ camera.

4.2.1. Chuẩn bị bộ dữ liệu huấn luyện

Dataset được load bằng lớp ImageFolder của torchvision, tự động gán nhãn số nguyên cho từng lớp dựa theo tên thư mục con theo thứ tự bảng chữ cái. Với cấu trúc thư mục đã thiết kế ở **Chương 3**, hai lớp được gán nhãn như sau: intrusion tương ứng với 0 và normal tương ứng với 1.

Sau khi load, chương trình tự động in ra bảng thống kê số lượng ảnh theo từng lớp cho cả tập train và validation, giúp người dùng kiểm tra ngay tính cân bằng của dataset trước khi bắt đầu huấn luyện. DataLoader được cấu hình với shuffle=True cho tập train nhằm đảm bảo thứ tự ảnh ngẫu nhiên mỗi epoch, và shuffle=False cho tập validation nhằm giữ kết quả đánh giá nhất quán giữa các epoch. Tham số pin_memory=True được sử dụng để tăng tốc truyền dữ liệu lên GPU.

```
dataset/
  train/
    intrusion/
    normal/
  val/
    intrusion/
    normal/
```

```
test/  
    intrusion/  
    normal/
```

Tập train được dùng để cập nhật trọng số mô hình, còn tập validation được dùng để đánh giá sau mỗi epoch. Cấu trúc thư mục bảo đảm ImageFolder tự động ánh xạ hai lớp intrusion và normal. Ngoài ra, tập test được sử dụng để đánh giá khả năng phân loại của mô hình trên dữ liệu chưa từng tham gia vào quá trình huấn luyện và validation.

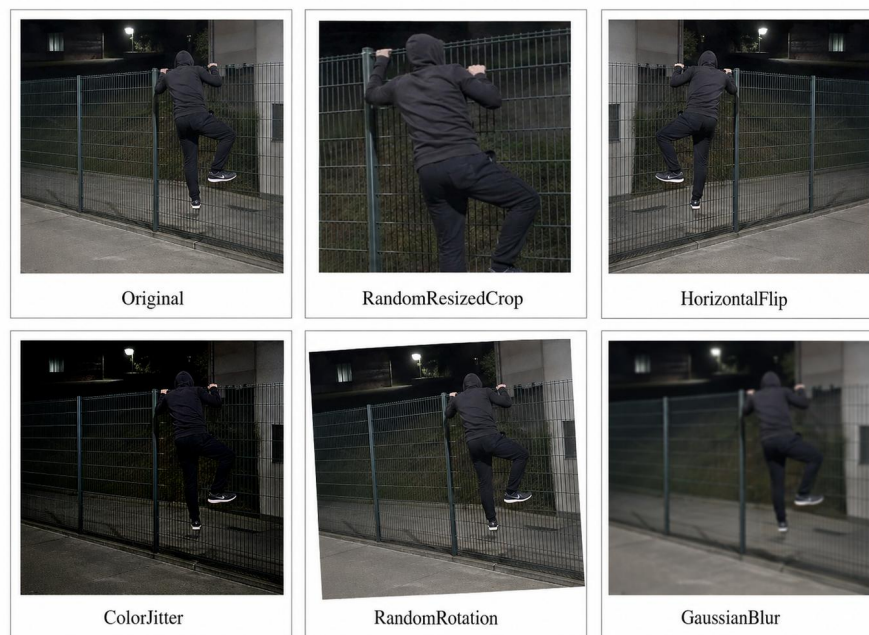
4.2.2. Tiền xử lý và tăng cường dữ liệu

Data augmentation là kỹ thuật tạo ra các biến thể nhân tạo của ảnh huấn luyện bằng cách áp dụng các phép biến đổi ngẫu nhiên, giúp mô hình học được tính bất biến với các thay đổi về góc nhìn, ánh sáng và điều kiện môi trường mà không cần thu thập thêm dữ liệu thực tế. Trong hệ thống này, data augmentation đặc biệt quan trọng vì camera giám sát thực tế có thể đặt ở nhiều góc độ và hoạt động trong nhiều điều kiện ánh sáng khác nhau trong ngày.

Chương trình định nghĩa pipeline augmentation cho tập train trong biến `train_tf` với sáu kỹ thuật được áp dụng tuần tự:

1. `RandomResizedCrop(320, scale=(0.4, 1.0), ratio=(0.65, 1.5))`: cắt ngẫu nhiên một vùng có diện tích từ 40% đến 100% ảnh gốc với tỷ lệ khung hình từ 0.65 đến 1.5, sau đó resize về 320×320 .
2. `RandomHorizontalFlip()`: lật ảnh theo chiều ngang với xác suất 50%. Hành động leo rào từ trái sang phải và từ phải sang trái về bản chất là như nhau, do đó phép biến đổi này hợp lệ.
3. `ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2)`: thay đổi ngẫu nhiên độ sáng, độ tương phản và độ bão hòa màu để mô phỏng sự thay đổi điều kiện ánh sáng.
4. `RandomRotation(10)`: xoay ảnh ngẫu nhiên trong khoảng $\pm 10^\circ$, mô phỏng sự nghiêng nhẹ của camera hoặc của thân người.
5. `RandomPerspective(distortion_scale=0.2, p=0.4)`: áp dụng biến dạng phối cảnh ngẫu nhiên với xác suất 40%, mô phỏng camera đặt ở góc cao hoặc góc chéo.
6. `RandomApply([GaussianBlur(kernel_size=5, sigma=(0.1, 2.0))], p=0.3)`: áp dụng làm mờ Gaussian với xác suất 30%, mô phỏng ảnh bị mờ do lấy nét hoặc chuyển động nhanh.

Sau khi áp dụng các phép biến đổi hình học và màu sắc, ảnh được chuyển sang tensor và chuẩn hóa theo giá trị mean/std của ImageNet. Khác với tập train, tập validation chỉ áp dụng resize và center crop mà không có augmentation ngẫu nhiên, đảm bảo kết quả đánh giá nhất quán và phản ánh đúng hiệu năng thực sự của mô hình.



Hình 4.1. Minh họa các kỹ thuật data augmentation được áp dụng

4.2.3. Xây dựng kiến trúc MobileNetV2

Mô hình MobileNetV2 được xây dựng thông qua hàm `build_model(num_classes)` với ba bước chính: load pretrained weights, cấu hình partial freezing và thay thế classifier head.

Đầu tiên, mô hình được khởi tạo với trọng số pretrained từ ImageNet thông qua tham số `weights=MobileNet_V2_Weights.DEFAULT`. Việc dùng pretrained weights giúp mô hình bắt đầu với bộ đặc trưng tốt thay vì khởi tạo ngẫu nhiên, rút ngắn đáng kể thời gian hội tụ và cải thiện chất lượng đặc trưng học được trên dataset nhỏ.

Tiếp theo, các lớp features được cấu hình đóng băng hoặc huấn luyện theo chỉ số:

Cụ thể, `features[0]` đến `features[7]` được đóng băng hoàn toàn trong hai epoch đầu. Đây là các lớp đầu học đặc trưng cấp thấp như cạnh, góc và kết cấu. Các lớp từ `features[8]` trở về sau được phép cập nhật ngay từ đầu để học đặc trưng ngữ nghĩa cao hơn. Từ `UNFREEZE_EPOCH = 2` trở đi, toàn bộ backbone được mở băng để fine-tune sâu hơn trên dữ liệu leo rào.

Cuối cùng, classifier head mặc định được thay thế bằng cấu trúc gọn hơn, có dạng $1280 \rightarrow 64 \rightarrow num_classes$. Toàn bộ mô hình được đưa lên GPU hoặc CPU thông qua lệnh `model.to(DEVICE)`.

4.2.4. Cấu hình huấn luyện

Các siêu tham số của quá trình huấn luyện được tập trung khai báo trong phần `CONFIG` ở đầu chương trình, giúp dễ dàng điều chỉnh mà không cần tìm kiếm trong code. **Bảng 4.3** liệt kê các tham số chính cùng giá trị và lý do lựa chọn.

Bảng 4.3. Các siêu tham số huấn luyện mô hình MobileNetV2

Tham số	Giá trị	Lý do lựa chọn
Model	MobileNetV2	Nhẹ, phù hợp thiết bị nhúng
Batch size	24	Phù hợp bộ nhớ GPU T4/A100
Epochs	20	Đủ để hội tụ với transfer learning
Learning rate	1e-4	LR thấp, ổn định khi fine-tune
Train image size	320×320	Học đặc trưng chi tiết hơn
Infer image size	224×224	Tăng tốc inference khoảng 2 lần trên RPi
Optimizer	AdamW	Weight decay tách riêng, ổn định hơn
Loss function	CrossEntropy	Phù hợp phân loại đa lớp
UNFREEZE_EPOCH	2	Mở băng backbone từ epoch thứ 2

Đáng chú ý là learning rate được thiết lập khác nhau cho từng nhóm tham số trong mô hình thông qua parameter groups của AdamW:

- features[0:8]: learning rate bằng 1e-6.
- features[8:]: learning rate bằng 1e-5.
- Classifier head: learning rate bằng 1e-4.

Cách thiết lập learning rate theo nhóm như vậy giúp các lớp đầu của backbone, vốn đã học được các đặc trưng cấp thấp tổng quát từ ImageNet, được cập nhật rất chậm hoặc không cập nhật; trong khi đó classifier head được học nhanh hơn để thích nghi với bài toán mới.

Vòng lặp huấn luyện

Thuật toán tối ưu AdamW được cấu hình với ba nhóm tham số có learning rate khác nhau như đã trình bày ở mục 4.2.1, cùng với weight_decay=1e-4 áp dụng đồng đều cho tất cả nhóm. Learning rate scheduler OneCycleLR điều chỉnh learning rate động theo từng batch trong suốt quá trình huấn luyện theo chu kỳ một pha tăng và một pha giảm, giúp mô hình hội tụ nhanh hơn so với learning rate cố định.

Vòng lặp huấn luyện được thực hiện trong hàm run_epoch(loader, is_train) dùng chung cho cả train và validation, với các bước trong mỗi batch:

1. Đưa ảnh và nhãn lên GPU hoặc CPU.
2. Forward pass để tính logits từ mô hình.
3. Tính loss bằng CrossEntropyLoss.
4. Backward pass để lan truyền ngược gradient.
5. Gradient clipping, giới hạn norm gradient tối đa là 1.0.
6. Optimizer step để cập nhật trọng số.
7. Scheduler step để cập nhật learning rate.
8. Tính accuracy trên batch hiện tại.

Gradient clipping là bước quan trọng đảm bảo quá trình huấn luyện ổn định, đặc biệt trong giai đoạn đầu khi backbone vừa được mở băng và gradient có thể tăng đột biến.

4.2.5. Chiến lược fine-tuning và partial freezing

Quá trình fine-tuning được chia theo mức độ cập nhật của backbone. Trong hai epoch đầu, các lớp features[0] đến features[7] được đóng băng để giữ đặc trưng cấp thấp đã học từ ImageNet. Các lớp từ features[8]

trở về sau và classifier head được phép cập nhật. Từ UNFREEZE_EPOCH = 2, toàn bộ backbone được mở băng để thích nghi sâu hơn với dữ liệu leo rào.

AdamW sử dụng learning rate theo nhóm: 1e-6 cho các lớp đầu backbone, 1e-5 cho các lớp sau và 1e-4 cho classifier head. Classifier luôn được huấn luyện, trong khi backbone được cập nhật chậm hơn nhằm hạn chế overfitting và tránh phá vỡ quá nhanh các đặc trưng pretrained.

4.2.6. Đánh giá và lưu mô hình tốt nhất

Sau mỗi epoch, chương trình tự động chạy đánh giá trên tập validation và ghi lại đầy đủ các chỉ số: train loss, validation loss, train accuracy, validation accuracy, Precision, Recall và F1-score tính theo macro average. Toàn bộ lịch sử được lưu trong dictionary history để vẽ biểu đồ sau khi huấn luyện kết thúc. Mỗi epoch in ra một dòng tóm tắt theo định dạng:

```
Epoch [XX/20] | Loss: t_loss/v_loss | Acc: t_acc%/v_acc%  
              | P:x.xxx R:x.xxx F1:x.xxx [saved]
```

Dấu [saved] xuất hiện khi validation accuracy của epoch hiện tại vượt qua giá trị tốt nhất từ trước đến nay, báo hiệu checkpoint đã được lưu lại.

Chiến lược lưu model theo validation accuracy cao nhất được áp dụng thay vì lưu model của epoch cuối cùng. Mỗi khi validation accuracy tăng, chương trình lưu một dictionary checkpoint gồm đầy đủ thông tin cần thiết để tải lại và tái sử dụng mô hình:

```
"model_state_dict": ...,  
"class_names": ["intrusion", "normal"],  
"num_classes": 2,  
"img_size_train": 320,  
"img_size_infer": 224
```

Việc lưu cả class_names và img_size_infer vào checkpoint giúp chương trình test và thực tế có thể tự động tái tạo đúng kiến trúc mô hình và cấu hình tiền xử lý mà không cần phụ thuộc vào bất kỳ file cấu hình riêng nào. File checkpoint được lưu với tên best_mobilenetv2.pth tại thư mục output.

4.3. CHUYỂN ĐỔI VÀ TỐI ƯU MÔ HÌNH PHỤC VỤ TRIỂN KHAI

4.3.1. Xuất mô hình từ PyTorch sang ONNX

Do ONNX Runtime tiêu thụ tài nguyên thấp hơn đáng kể so với PyTorch đầy đủ, định dạng ONNX được chọn để triển khai mô hình trên Raspberry Pi.

Mô hình được load từ best checkpoint, chuyển sang chế độ eval() trên CPU để đảm bảo các lớp Dropout và BatchNorm hoạt động đúng chế độ inference, sau đó export bằng torch.onnx.export(). Dummy input kích thước (1, 3, 224, 224) chỉ phục vụ việc truy vết luồng tính toán, không ảnh hưởng đến trọng số.

Ba tham số kỹ thuật quan trọng được cấu hình:

- **opset_version=18:** Đảm bảo hỗ trợ đầy đủ các toán tử của MobileNetV2.
- **dynamo=False:** Sử dụng TorchScript exporter, ổn định hơn dynamo exporter trong quá trình chuyển đổi.
- **dynamic_axes:** Cho phép mô hình ONNX nhận batch size tùy ý thay vì cố định ở 1.

File đầu ra mobilenetv2_float32.onnx lưu mô hình ở định dạng float32, đóng vai trò là bản gốc và là đầu vào cho bước quantize tiếp theo.

4.3.2. So sánh mô hình ONNX Float32 và INT8

Sau khi huấn luyện mô hình MobileNetV2 trên môi trường máy tính, mô hình được chuyển đổi từ định dạng PyTorch sang định dạng ONNX để phục vụ triển khai trên Raspberry Pi 4. Trong quá trình triển khai, hai phiên bản mô hình được xem xét gồm mobilenetv2_float32.onnx và mobilenetv2_int8.onnx. Phiên bản Float32 giữ nguyên kiểu dữ liệu số thực 32-bit của trọng số, trong khi phiên bản INT8 được tạo ra bằng kỹ thuật lượng tử hóa nhằm giảm kích thước mô hình và tối ưu tài nguyên khi chạy trên thiết bị nhúng.

Phiên bản mobilenetv2_float32.onnx là mô hình ONNX giữ nguyên độ chính xác số học sau khi export từ PyTorch. Do vẫn sử dụng trọng số float32, mô hình này có ưu điểm là độ ổn định cao, hạn chế sai lệch kết quả so với mô hình gốc trong quá trình huấn luyện. Đây là phiên bản phù hợp để kiểm tra độ chính xác ban đầu và được sử dụng làm mốc so sánh khi đánh giá các phiên bản tối ưu hơn.

Bảng 4.4. So sánh mô hình MobileNetV2 ONNX Float32 và INT8

Tiêu chí	ONNX Float32	ONNX INT8
File mô hình	mobilenetv2_float32.onnx	mobilenetv2_int8.onnx
Kiểu dữ liệu trọng số	Float32, mỗi giá trị chiếm 4 byte	Int8, mỗi giá trị chiếm 1 byte
Kích thước file	Khoảng 8,8–9 MB	Khoảng 2–3 MB
Phương pháp tạo mô hình	Export trực tiếp từ PyTorch bằng torch.onnx.export()	Lượng tử hóa từ ONNX Float32 bằng quantize_dynamic()
Độ chính xác	Ổn định, gần tương đương mô hình PyTorch gốc	Có thể giảm nhẹ do lượng tử hóa trọng số
Tốc độ suy luận	Khoảng 55 ms/crop trong thực nghiệm	Cần benchmark riêng trên Raspberry Pi 4
Mức sử dụng bộ nhớ	Cao hơn do dùng trọng số float32	Thấp hơn do trọng số được nén về int8
Ưu điểm	Độ chính xác và độ ổn định cao	Kích thước nhỏ, phù hợp tối ưu thiết bị nhúng
Hạn chế	Dung lượng và tài nguyên tính toán cao hơn	Cần kiểm tra lại độ chính xác và tốc độ thực tế
Vai trò trong hệ thống	Phiên bản triển khai và đánh giá chính	Phiên bản tối ưu tiềm năng cho hướng phát triển

Như vậy, phiên bản Float32 phù hợp trong giai đoạn đánh giá độ chính xác và kiểm chứng tính ổn định của mô hình, còn phiên bản INT8 phù hợp cho hướng tối ưu triển khai trên thiết bị nhúng nhờ kích thước nhỏ và tiềm năng giảm tài nguyên tính toán. Trong phạm vi hệ thống hiện tại, các số liệu tốc độ thực nghiệm được ghi nhận dựa trên phiên bản mobilenetv2_float32.onnx; phiên bản INT8 cần được benchmark riêng để đánh giá chính xác hiệu quả tối ưu.

4.3.3. Chuẩn bị file triển khai

Sau khi export và quantize, chương trình lưu một file metadata model_meta.json chứa tất cả thông tin cần thiết để tải và sử dụng mô hình trên Raspberry Pi mà không cần truy cập vào code huấn luyện gốc. Nội dung file metadata gồm:

```
"class_names": ["intrusion", "normal"],
"num_classes": 2,
"img_size_train": 320,
"img_size_infer": 224,
```

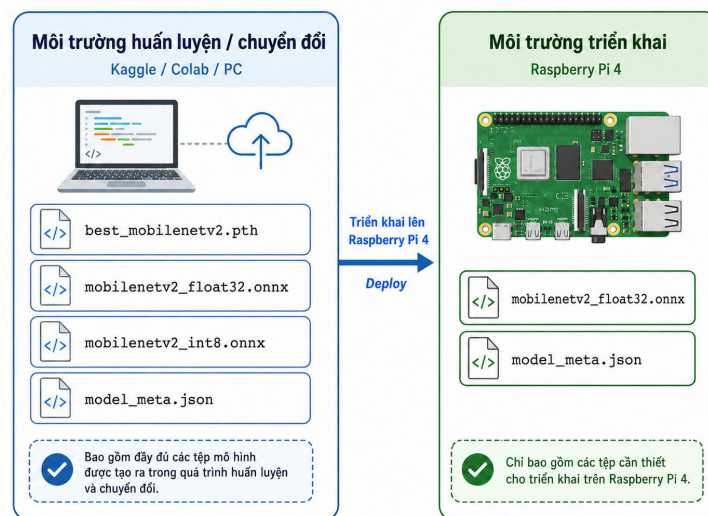
```
"mean": [0.485, 0.456, 0.406],
"std": [0.229, 0.224, 0.225],
"best_val_acc": "<gia tri accuracy tot nhat tren val set>"
```

Thông tin mean và std đặc biệt quan trọng vì chương trình inference trên Raspberry Pi cần chuẩn hóa ảnh đầu vào theo đúng các giá trị này. Nếu dùng sai giá trị chuẩn hóa, mô hình sẽ cho kết quả sai dù trọng số hoàn toàn chính xác. Bằng cách đóng gói thông tin này vào metadata, hệ thống đảm bảo tính nhất quán giữa môi trường huấn luyện và môi trường triển khai.

Hai file cần thiết để triển khai lên Raspberry Pi là mobilenetv2_float.onnx và model_meta.json. Tất cả thông tin cần thiết để chạy inference đã được đóng gói đầy đủ trong hai file này.

```
deployment/
mobilenetv2_float32.onnx
model_meta.json
yolov8n.pt
yolov8.onnx
main_app1.py
```

mobilenetv2_float32.onnx thực hiện phân loại hành động; model_meta.json chứa class, kích thước ảnh, mean và std; yolov8n.onnx phát hiện người. Mã nguồn inference điều phối pipeline AI, mã nguồn Web cung cấp Dashboard và file GPIO khai báo riêng chân buzzer, relay, reset. Việc khai báo GPIO tập trung giúp tránh ghi cứng chân ở nhiều module.



Hình 4.2. Các file cần thiết khi triển khai mô hình lên Raspberry Pi 4

4.4. XÂY DỰNG CHƯƠNG TRÌNH NHẬN DIỆN VÀ CẢNH BÁO

4.4.1. Khởi tạo hệ thống

Khi chương trình bắt đầu, hệ thống đọc file `model_meta.json` để lấy thông tin cấu hình lớp phân loại, kích thước ảnh đầu vào và các tham số liên quan đến mô hình. Sau đó, chương trình khởi tạo phiên suy luận bằng ONNX Runtime cho mô hình MobileNetV2 ở định dạng float32, đồng thời tải mô hình phát hiện người YOLOv8n.onnx. Hai mô hình này tạo thành pipeline xử lý chính, trong đó YOLOv8n đảm nhiệm phát hiện người trong khung hình, còn MobileNetV2 thực hiện phân loại hành động sau khi vùng ảnh chứa người được cắt ra.

Tiếp theo, chương trình mở camera để thu nhận luồng hình ảnh đầu vào, cấu hình các chân GPIO điều khiển buzzer và relay về trạng thái an toàn ban đầu nhằm tránh kích hoạt cảnh báo ngoài ý muốn. Chân GPIO27 được thiết lập làm ngõ vào ở chế độ *pull-up* để phục vụ chức năng reset hoặc phục hồi trạng thái hệ thống khi cần thiết. Đồng thời, chương trình khởi tạo bộ đếm số khung hình liên tiếp phát hiện xâm nhập intrusion frame counter, thư mục lưu sự kiện, đối tượng VideoWriter phục vụ ghi hình và Web Server phục vụ giao diện giám sát.

Bên cạnh các thành phần xử lý cục bộ, trên Raspberry Pi 4, dịch vụ cloudflared được khởi chạy ở chế độ nền để thiết lập Cloudflare Tunnel. Cơ chế này cho phép tạo kết nối bảo mật từ mạng nội bộ ra Internet mà không cần cấu hình *Port Forwarding* trên router hoặc sử dụng địa chỉ *IP Public*. Nhờ đó, giao diện giám sát của hệ thống có thể được truy cập từ xa thông qua tên miền đã cấu hình trên Cloudflare, giúp người dùng theo dõi trạng thái hệ thống linh hoạt hơn.

Nếu một thành phần trong quá trình khởi tạo bị lỗi, chẳng hạn như không mở được camera, không tải được mô hình, lỗi cấu hình GPIO hoặc không khởi động được dịch vụ web, chương trình sẽ ghi log lỗi, giải phóng các tài nguyên đã được mở và giữ buzzer, relay ở trạng thái không kích hoạt. Chỉ khi camera, mô hình AI, GPIO, bộ ghi hình và Web Server đều sẵn sàng, vòng lặp giám sát chính của hệ thống mới được bắt đầu.

4.4.2. Đọc dữ liệu từ camera

Webcam USB được mở bằng cv2.VideoCapture. Chương trình thiết lập độ phân giải và FPS phù hợp với năng lực Raspberry Pi, sau đó kiểm tra giá trị trả về của cap.read() trong từng vòng lặp. Khi mất kết nối, camera được đóng và thử mở lại thay vì tiếp tục xử lý frame rỗng.

Frame gốc có thể được giữ ở độ phân giải cao cho ghi hình và Web; frame đưa vào YOLOv8 được điều chỉnh kích thước để giảm tải tính toán. MobileNetV2 không nhận frame này trực tiếp.

4.4.3. Phát hiện người bằng YOLOv8

YOLOv8 được sử dụng với trọng số yolov8n.onnx để phát hiện đối tượng người trong từng khung hình. Tham số classes = [0] giới hạn kết quả phát hiện ở lớp person, CONF_THRESH = 0.4 được sử dụng để loại bỏ các bounding box có độ tin cậy thấp và IOU_THRESH = 0.45 hỗ trợ loại bỏ các bounding box chồng lặp thông qua thuật toán Non-Maximum Suppression (NMS).

YOLOv8 chỉ thực hiện nhiệm vụ xác định vị trí của người trong khung hình, không phân loại hành vi leo rào hay xâm nhập. Đầu ra của mô hình bao gồm tọa độ bounding box và độ tin cậy (confidence score) của từng đối tượng được phát hiện. Các vùng ảnh tương ứng sau đó được cắt (crop) và đưa vào mô hình MobileNetV2 để thực hiện bước phân loại normal/intrusion.

4.4.4. Crop vùng người và tiền xử lý ảnh

Tọa độ bounding box được giới hạn trong biên frame và mở rộng padding 10% theo chiều rộng, chiều cao. Vùng ảnh sau padding được crop riêng cho từng người, chuyển từ BGR sang RGB, resize lên khoảng 246×246 , center crop về 224×224 và chuẩn hóa theo mean/std ImageNet.

Tensor đầu vào được chuyển về định dạng và thứ tự chiều mà ONNX Runtime yêu cầu. Nếu crop rỗng hoặc không hợp lệ, đối tượng đó được bỏ qua để tránh lỗi suy luận.

4.4.5. Phân loại hành động bằng MobileNetV2

Ảnh crop sau khi tiền xử lý được đưa vào mô hình MobileNetV2 ONNX để thực hiện suy luận. ONNX Runtime trả về vector xác suất thuộc hai lớp Intrusion và Normal thông qua hàm Softmax, trong đó lớp có xác suất cao hơn được chọn làm kết quả phân loại cho đối tượng tương ứng.

Dựa trên kết quả phân loại, hệ thống cập nhật Intrusion Frame Counter — bộ đếm theo dõi số khung hình liên tiếp được phân loại là Intrusion. Nếu một khung hình được phân loại là Normal hoặc không phát hiện người, bộ đếm được reset về 0. Chỉ khi bộ đếm đạt ngưỡng quy định, hệ thống mới xác nhận có hành vi xâm nhập và chuyển sang bước kích hoạt cảnh báo. Cơ chế này giúp lọc bỏ các trường hợp phân loại sai nhất thời do nhiễu hoặc chuyển động đột ngột, tăng độ tin cậy cho quyết định cảnh báo của hệ thống.

4.4.6. Kích hoạt cảnh báo

Khi hệ thống xác nhận trạng thái Intrusion sau khi số lượng khung hình phát hiện xâm nhập đạt ngưỡng thiết lập, chương trình sẽ kích hoạt GPIO_BUZZER, lưu ảnh sự kiện kèm thời gian và cập nhật trạng thái lên Web Dashboard. Người dùng có thể bật hoặc tắt còi báo động thủ công trên giao diện Web trong trường hợp cảnh báo kéo dài hoặc sự kiện đã được kiểm tra, trong khi hệ thống AI vẫn tiếp tục giám sát và sẽ tự động kích hoạt lại cảnh báo khi phát hiện một sự kiện xâm nhập mới thỏa mãn điều kiện xác nhận.

Bảng 4.5. Ánh xạ tín hiệu giữa chương trình và phần cứng

Tín hiệu	Hướng	Chức năng triển khai
GPIO_BUZZER	Output	Điều khiển transistor NPN để bật/tắt buzzer 5V.
USB Camera	Input	Cung cấp frame cho ghi hình, Web và pipeline AI.
Ethernet/Wi-Fi	Hai chiều	Truyền video, trạng thái và lệnh điều khiển Web.

4.4.7. Hiển thị kết quả lên web giám sát

Sau khi hoàn thành quá trình phân loại, chương trình sẽ hiển thị trạng thái của hệ thống trực tiếp trên khung hình giám sát. Khi trạng thái là Normal, khung hình được viền màu xanh lá và hiển thị nhãn "SECURE" ở góc trên bên trái. Ngược lại, khi hệ thống xác nhận trạng thái Intrusion, khung hình sẽ được viền màu đỏ và hiển thị nhãn "INTRUSION" tại cùng vị trí nhằm giúp người dùng dễ dàng nhận biết tình trạng hiện tại của khu vực giám sát.

Khung hình sau khi được cập nhật các thông tin hiển thị sẽ được chuyển đến luồng truyền phát video để mã hóa và phát trực tiếp trên giao diện giám sát, đồng thời được sử dụng để lưu ảnh sự kiện khi hệ thống xác nhận có hành vi xâm nhập.

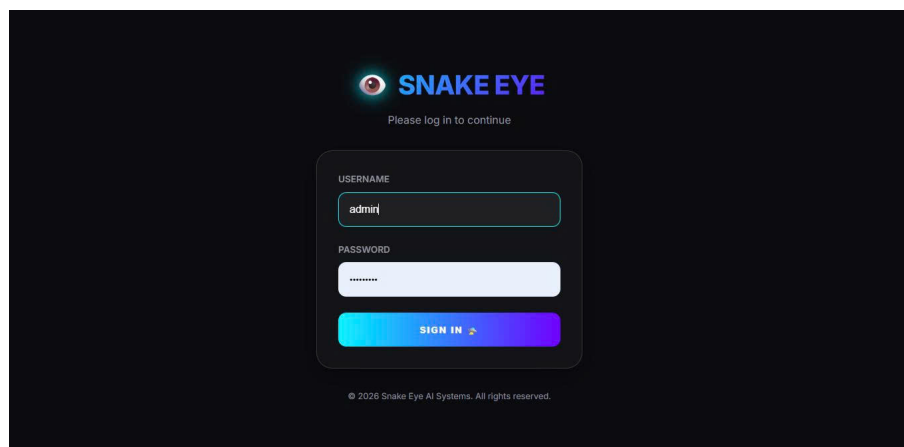
4.5. XÂY DỰNG GIAO DIỆN WEB GIÁM SÁT VÀ ĐIỀU KHIỂN

4.5.1. Trang giao diện đăng nhập

Để đảm bảo an toàn cho hệ thống giám sát, Web Dashboard được xây dựng cơ chế xác thực người dùng trước khi cho phép truy cập các chức năng quản lý và điều khiển. Khi người dùng truy cập địa chỉ của hệ thống, giao diện đăng nhập sẽ được hiển thị để yêu cầu nhập tên đăng nhập (Username) và mật khẩu (Password).

Sau khi người dùng nhấn nút Sign In, thông tin đăng nhập sẽ được gửi đến máy chủ Flask để kiểm tra tính hợp lệ. Nếu thông tin xác thực chính xác, hệ thống sẽ tạo phiên làm việc (Session) và chuyển người dùng đến trang Dashboard chính. Ngược lại, nếu thông tin không hợp lệ, hệ thống sẽ từ chối truy cập và yêu cầu người dùng đăng nhập lại.

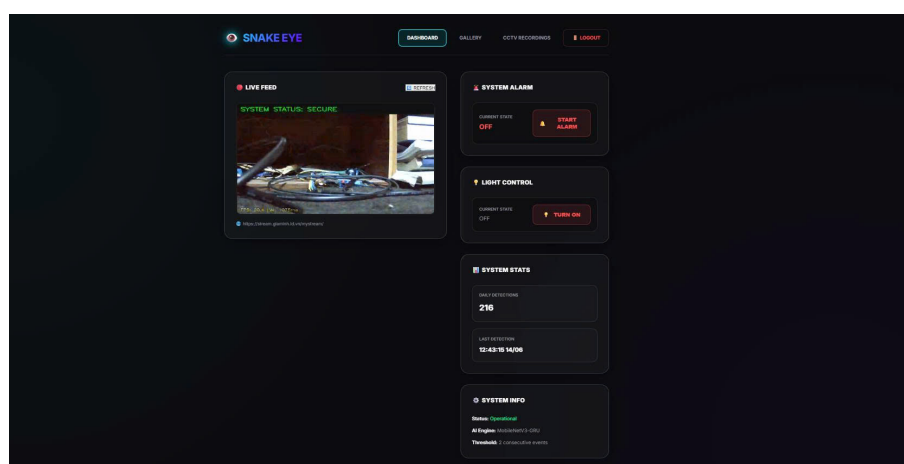
Việc sử dụng cơ chế đăng nhập giúp hạn chế truy cập trái phép, bảo vệ các chức năng quản lý quan trọng của hệ thống. Cơ chế Session của Flask đảm bảo duy trì trạng thái đăng nhập và yêu cầu xác thực lại khi phiên làm việc kết thúc hoặc người dùng đăng xuất.



Hình 4.3. Giao diện trang đăng nhập

4.5.2. Chức năng hiển thị trạng thái hệ thống

Dashboard được thiết kế nhằm cung cấp giao diện giám sát trực quan, cho phép người dùng theo dõi trạng thái hoạt động của toàn bộ hệ thống theo thời gian thực. Các thông tin được hiển thị trên giao diện được cập nhật liên tục từ chương trình backend đang chạy trên Raspberry Pi 4, kết hợp với trạng thái thực tế của các thiết bị ngoại vi thông qua giao tiếp GPIO, bảo đảm dữ liệu hiển thị phản ánh chính xác tình trạng hiện tại của hệ thống.



Hình 4.4. Giao diện Web giám sát trung tâm

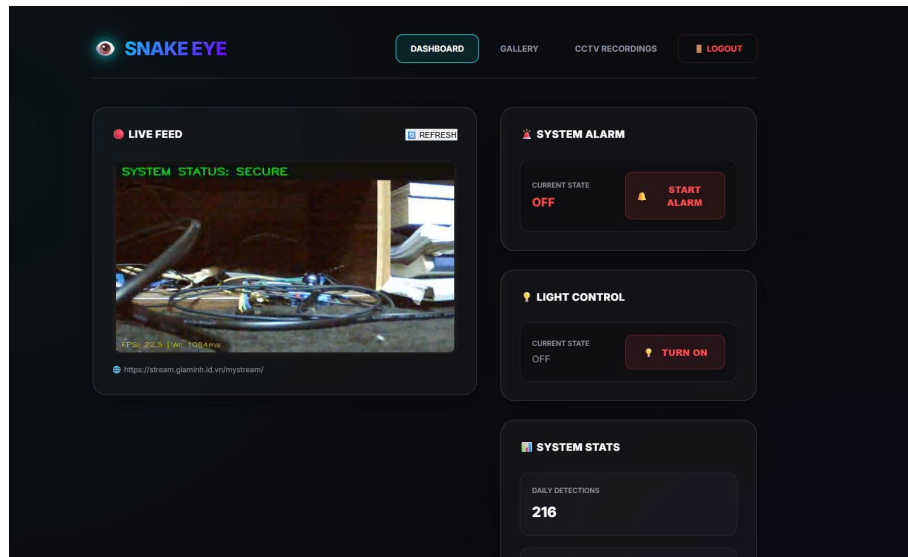
Tại khu vực Live Feed (Hình 4.4), người dùng có thể quan sát trực tiếp luồng video từ camera giám sát. Kết quả phân tích của mô hình AI được hiển thị dưới dạng trạng thái tổng thể của hệ thống thay vì đánh dấu trên từng đối tượng. Khi khu vực giám sát ở trạng thái bình thường, dòng chữ "SYSTEM STATUS: SECURE" được hiển thị bằng màu xanh lá ở góc trên bên trái của khung hình. Ngược lại, khi hệ thống xác nhận có hành vi xâm nhập sau khi vượt ngưỡng của Intrusion Frame Counter, trạng thái sẽ chuyển sang "SYSTEM STATUS: INTRUSION" với màu đỏ và khung hình được viền đỏ để nhấn mạnh cảnh báo, giúp người dùng dễ dàng nhận biết sự kiện bất thường.

Bên cạnh luồng video trực tiếp, Dashboard còn hiển thị các thông số vận hành quan trọng của hệ thống như trạng thái kết nối camera, trạng thái hoạt động của mô hình AI, tốc độ xử lý khung hình (FPS), số lượng sự kiện xâm nhập được ghi nhận trong ngày, thời điểm phát hiện xâm nhập gần nhất, thời gian hoạt động liên tục của hệ thống và các thông báo lỗi nếu có. Các thông tin này được cập nhật định kỳ từ backend và đồng bộ với trạng thái thực tế của hệ thống, hỗ trợ người dùng giám sát hiệu quả quá trình vận hành.

4.5.3. Chức năng điều khiển từ xa

Người quản trị có thể điều khiển từ xa các thiết bị ngoại vi của hệ thống thông qua giao diện Web Dashboard, bao gồm còi báo động và đèn chiếu sáng. Chức năng này cho phép người dùng chủ động xử lý các tình huống thực tế mà không cần thao tác trực tiếp trên Raspberry Pi, góp phần nâng cao tính linh hoạt và khả năng quản lý từ xa của hệ thống.

Mỗi thao tác bật/tắt thiết bị trên giao diện Web sẽ được gửi đến máy chủ Flask qua HTTP. Sau khi xác thực phiên đăng nhập, hệ thống cập nhật trạng thái GPIO tương ứng trên Raspberry Pi và phản hồi lại giao diện để đảm bảo thông tin hiển thị luôn đồng bộ với phần cứng.



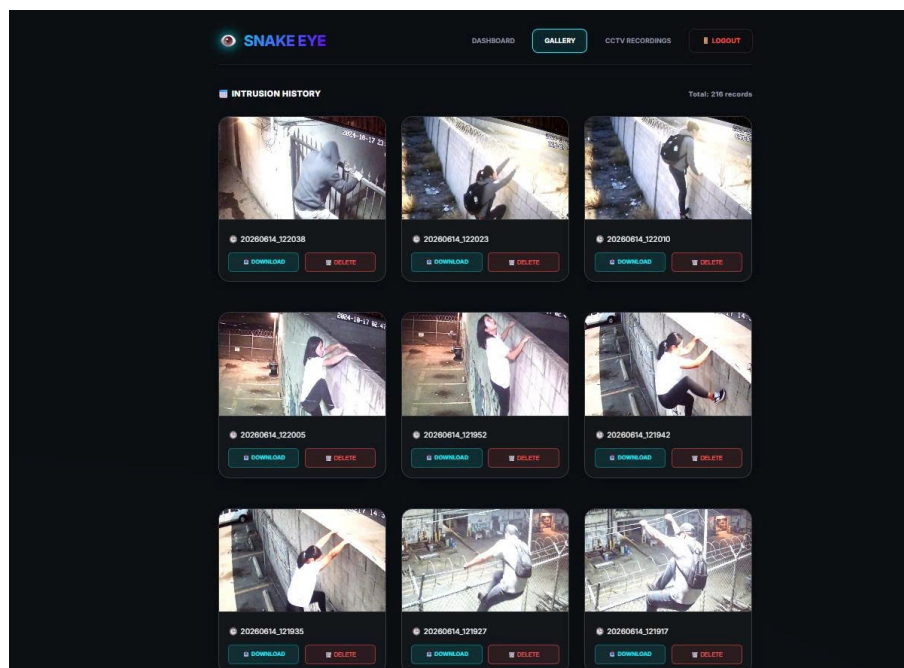
Hình 4.5. Giao diện điều khiển từ xa

Như minh họa trong **Hình 4.5**, giao diện điều khiển được thiết kế đơn giản và trực quan, hiển thị trạng thái còi báo động và đèn chiếu sáng cùng các nút điều khiển tương ứng, cho phép người dùng theo dõi và thao tác chỉ bằng một lần nhấn trên trình duyệt Web. Việc tích hợp điều khiển từ xa với hệ thống giám sát AI giúp người quản trị vừa quan sát tình trạng an ninh vừa chủ động xử lý sự cố ngay cả khi không có mặt tại khu vực giám sát.

4.5.4. Chức năng lưu trữ và xem lại dữ liệu

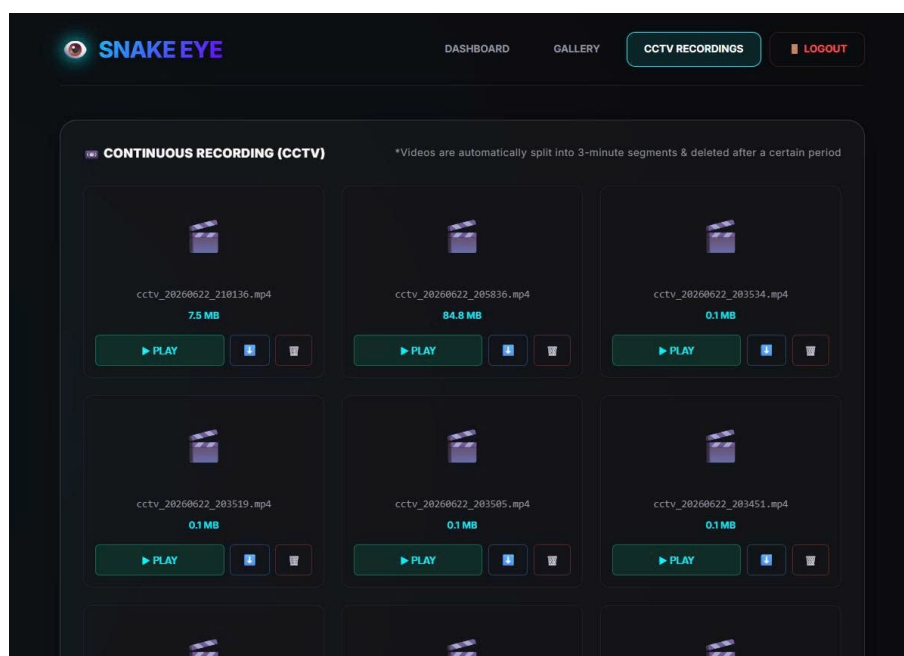
Hệ thống được thiết kế với chức năng lưu trữ dữ liệu nhằm hỗ trợ người dùng xem lại các sự kiện đã xảy ra cũng như phục vụ quá trình tra cứu khi cần thiết. Cơ chế lưu trữ được chia thành hai thành phần độc lập, bao gồm lưu ảnh sự kiện xâm nhập và ghi hình video giám sát liên tục.

Đối với ảnh sự kiện xâm nhập, khi hệ thống phát hiện trạng thái Intrusion và số lượng khung hình liên tiếp vượt ngưỡng xác nhận của Intrusion Frame Counter, chương trình sẽ tự động lưu khung hình hiện tại dưới định dạng JPEG. Tên tệp ảnh được tạo dựa trên thời gian xảy ra sự kiện theo định dạng YYYYMMDD_HHMMSS.jpg, giúp người dùng dễ dàng tra cứu và quản lý dữ liệu theo mốc thời gian. Các ảnh sự kiện được hiển thị trên giao diện Web Dashboard dưới dạng thư viện ảnh, cho phép người dùng xem lại nhanh các lần phát hiện xâm nhập.



Hình 4.6. Giao diện lưu trữ và xem lại ảnh sự kiện xâm nhập

Bên cạnh việc lưu ảnh sự kiện, hệ thống còn thực hiện ghi hình liên tục 24/7 thông qua tiến trình FFmpeg. Luồng video từ camera được mã hóa và lưu thành các tệp video có thời lượng cố định (khoảng 3 phút mỗi tệp) nhằm thuận tiện cho việc quản lý và xem lại dữ liệu giám sát. Cơ chế chia nhỏ video giúp giảm nguy cơ mất dữ liệu khi xảy ra sự cố và hỗ trợ quá trình tìm kiếm theo từng khoảng thời gian cụ thể.



Hình 4.7. Giao diện lưu trữ và xem lại video giám sát

Ngoài ra, (Hình 4.7) hệ thống còn tích hợp tiến trình dọn dẹp dữ liệu chạy nền để định kỳ kiểm tra thư mục lưu trữ và tự động xóa các tệp ảnh hoặc video đã vượt quá thời gian lưu giữ được cấu hình trước. Cơ chế này giúp giải phóng dung lượng lưu trữ trên Raspberry Pi, đảm bảo hệ thống có thể hoạt động liên tục trong thời gian dài mà không bị đầy bộ nhớ.

CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1. MỤC TIÊU VÀ NỘI DUNG THỰC NGHIỆM

Quá trình thực nghiệm được thực hiện nhằm đánh giá hệ thống phát hiện và cảnh báo xâm nhập qua hành động leo rào ở cả cấp độ mô hình AI và cấp độ hệ thống hoàn chỉnh. Việc đánh giá không chỉ dựa trên độ chính xác của MobileNetV2 trên tập validation mà còn xem xét khả năng vận hành của pipeline YOLOv8–MobileNetV2, phản hồi phần cứng, giao diện Web, độ trễ và tốc độ xử lý trên Raspberry Pi 4.

Các nội dung đánh giá chính gồm:

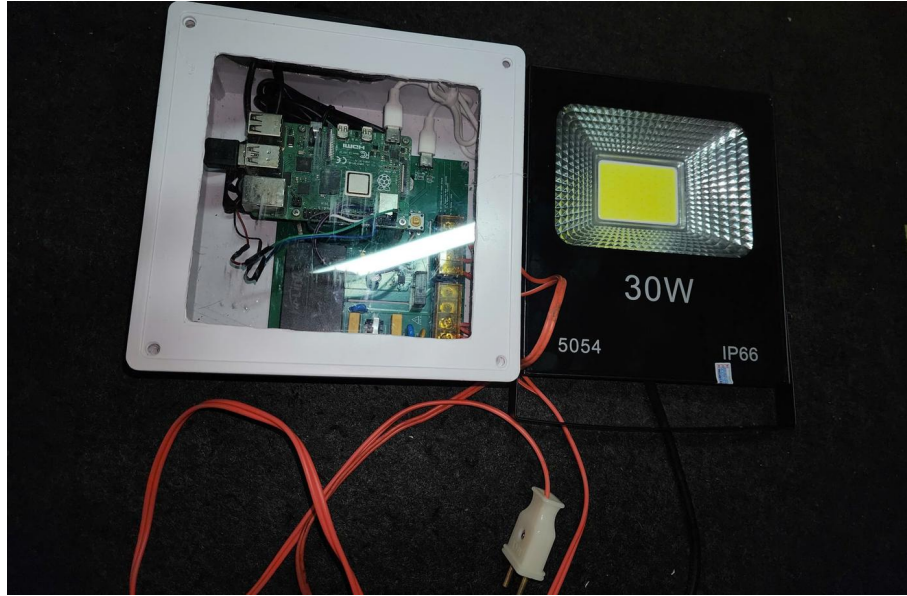
- Đánh giá quá trình hội tụ của MobileNetV2 thông qua train loss, validation loss, train accuracy và validation accuracy theo epoch.
- Đánh giá chất lượng phân loại bằng Accuracy, Precision, Recall, F1-score bằng biểu đồ cột và confusion matrix trên tập validation và tập test.
- Kiểm tra khả năng phát hiện người của YOLOv8 và chất lượng vùng ảnh crop được đưa vào MobileNetV2.
- Đánh giá kết quả phân loại normal/intrusion trên thực tế và hiệu quả ổn định quyết định của mô hình.
- Đánh giá khả năng hiển thị video, trạng thái cảnh báo và lịch sử sự kiện trên Web Dashboard.
- Đánh giá phản hồi của buzzer khi trạng thái intrusion được xác nhận.
- Đo độ trễ end-to-end, tốc độ inference và kích thước mô hình trên Raspberry Pi 4.

5.2. MÔI TRƯỜNG DỮ LIỆU VÀ THỰC NGHIỆM

5.2.1. Cấu hình phần cứng thực nghiệm

Hệ thống được triển khai trên Raspberry Pi 4 RAM 4 GB, CPU ARM Cortex-A72 1,5 GHz. Webcam USB Full HD cung cấp hình ảnh khu vực giám sát với độ phân giải tối đa 1920×1080 . Các khối chấp hành gồm buzzer 5 V, relay, đèn chiếu sáng và nút reset, được kết nối với Raspberry Pi qua board nguồn và ngoại vi.

Raspberry Pi thực hiện đồng thời các chức năng: đọc camera, chạy YOLOv8 để phát hiện người, chạy MobileNetV2 ONNX để phân loại hành động, điều khiển GPIO, ghi hình và cung cấp Web Dashboard. Kết quả AI sau đếm số lần phát hiện liên tiếp được dùng làm điều kiện kích hoạt cảnh báo, không sử dụng trực tiếp dự đoán của một frame đơn lẻ.



Hình 5.1. Hệ thống phần cứng đã lắp đặt thực tế.

5.2.2. Dữ liệu huấn luyện, validation và test

Dataset MobileNetV2 gồm các ảnh người đã được crop. Đây là dữ liệu phù hợp với pipeline thực tế vì MobileNetV2 không nhận trực tiếp ảnh toàn cảnh; ảnh toàn cảnh trước hết được YOLOv8 xử lý để tìm bounding box người, sau đó vùng người mới được crop và đưa vào bộ phân loại.

Bảng 5.1. Phân bố dữ liệu huấn luyện, validation và test.

Lớp	Train	Validation	Test	Tổng
intrusion	9.156	2.049	1.032	12.237
normal	8.208	2.080	1.053	11.341
Tổng	17.364	4.129	2.085	23.578

Tổng số ảnh của dataset sau khi bổ sung tập test là 23.578 ảnh, trong đó lớp *intrusion* có 12.237 ảnh, chiếm khoảng 51,9%, và lớp *normal* có 11.341 ảnh, chiếm khoảng 48,1%. Mức chênh lệch giữa hai lớp không lớn, vì vậy dữ liệu có độ cân bằng tương đối tốt và không cần áp dụng class weighting hoặc oversampling trong quá trình huấn luyện.

5.2.3. Dữ liệu kiểm thử và kịch bản kiểm thử

Kiểm thử được thực hiện trực tiếp trên hệ thống thực tế, sử dụng webcam USB kết nối với Raspberry Pi 4 làm nguồn thu nhận hình ảnh theo thời gian thực. Các tình huống thực nghiệm bao gồm: người đi lại bình thường gần hàng rào, người thực hiện hành vi bình thường nhưng dễ gây nhầm lẫn với hành vi xâm nhập, người thực hiện hành động leo rào tại vị trí lắp đặt camera, cùng các giai đoạn chuyển tiếp giữa đứng, bám và leo. Mỗi tình huống được lặp lại nhiều lần để đánh giá độ ổn định của hệ thống.

Với mỗi khung hình thu được, YOLOv8 chỉ phát hiện lớp person. Bounding box người được mở rộng padding, crop và tiền xử lý trước khi đưa vào MobileNetV2 để phân loại hành động.

Các kịch bản kiểm thử toàn hệ thống gồm:

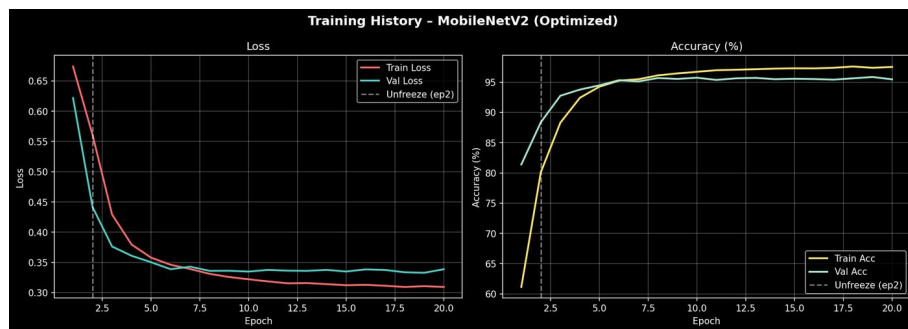
1. Không có người trong vùng giám sát.
2. Có người đi bộ hoặc đứng bình thường gần hàng rào.

3. Có người thực hiện hành động leo rào.
4. Người dùng bật/tắt buzzer và đèn từ Web.
5. Hệ thống lưu ảnh sự kiện và hiển thị lại trong thư viện.

5.3. KẾT QUẢ HUẤN LUYỆN MÔ HÌNH MOBILENETV2

MobileNetV2 được huấn luyện trong 20 epoch với batch size 24. Tập train gồm 17.364 ảnh và tập validation gồm 4.129 ảnh. Backbone được đóng băng một phần ở giai đoạn đầu và được mở băng từ epoch 2 để fine-tuning sâu hơn.

5.3.1. Kết quả loss và accuracy

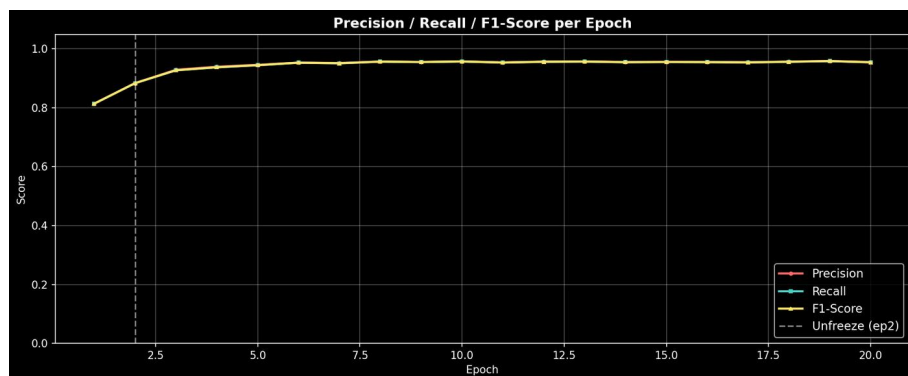


Hình 5.2. Train loss, validation loss, train accuracy và validation accuracy theo epoch.

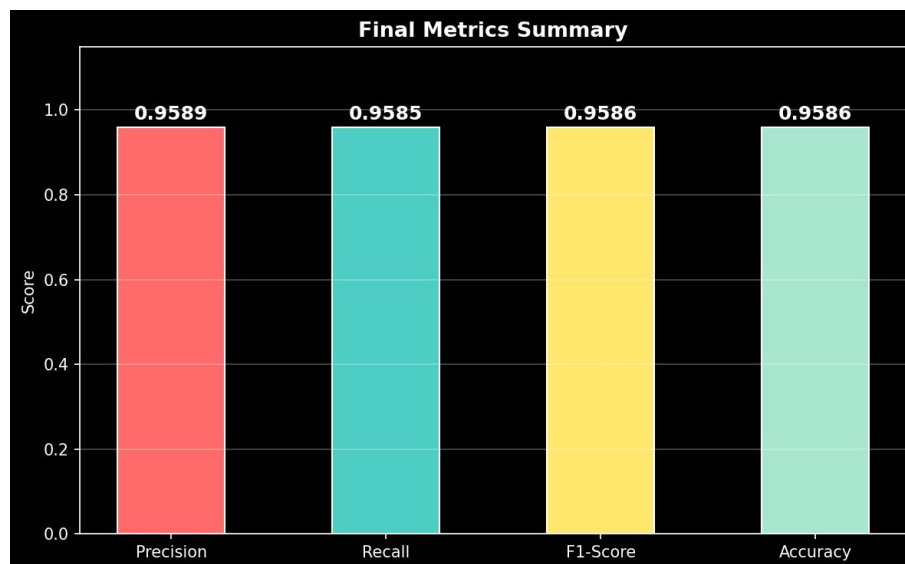
Trong hai epoch đầu, train loss và validation loss lần lượt ở mức khoảng 0,67 và 0,62. Sau khi mở băng backbone ở epoch 2, cả hai giá trị giảm nhanh rồi ổn định dần từ epoch 7, hội tụ về khoảng 0,31 (train) và 0,34 (validation) đến cuối quá trình (Hình 5.2). Khoảng cách giữa hai đường loss duy trì ở mức nhỏ và không phân kỳ ở giai đoạn cuối, cho thấy mô hình chưa có dấu hiệu overfitting rõ rệt.

Tương ứng, accuracy tăng nhanh sau khi mở băng backbone và đạt khoảng 95% từ epoch 10, duy trì ổn định đến cuối quá trình huấn luyện. Train accuracy cuối cùng đạt xấp xỉ 97%, validation accuracy đạt 95,86%. Khoảng cách nhỏ giữa hai giá trị cho thấy mô hình có khả năng tổng quát hóa tốt

5.3.2. Precision, Recall và F1-score



Hình 5.3. Precision, Recall và F1-score trung bình theo từng epoch.

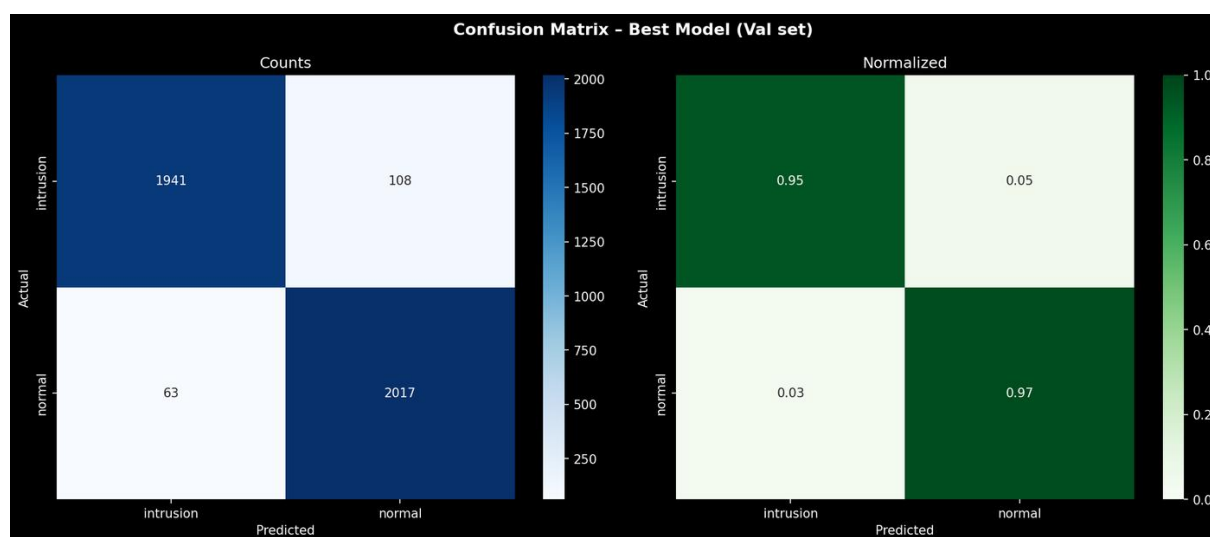


Hình 5.4. Tổng hợp các chỉ số đánh giá của best model.

Hình 5.3 Ba chỉ số Precision, Recall và F1-score tăng từ khoảng 0,81 ở epoch đầu lên trên 0,93 sau giai đoạn mở băng, sau đó ổn định quanh mức 0,95–0,96. Các đường có giá trị gần nhau, phản ánh mô hình đạt sự cân bằng tương đối giữa khả năng hạn chế dự đoán sai và khả năng phát hiện đúng các mẫu của hai lớp

Hình 5.4 Kết quả trên tập validation cho thấy Precision macro đạt 0,9589, Recall macro đạt 0,9585, F1-score macro đạt 0,9586 và Accuracy đạt 0,9586. Các giá trị này được tính theo trung bình macro của hai lớp, phản ánh độ chính xác, khả năng thu hồi và sự cân bằng giữa hai yếu tố này đều ở mức cao và đồng đều. Accuracy 95,86% tương ứng với 3.958/4.129 ảnh validation được phân loại đúng (171 ảnh sai), đáp ứng yêu cầu sử dụng MobileNetV2 làm bộ phân loại hành động sau bước phát hiện người.

5.3.3. Phân tích confusion matrix



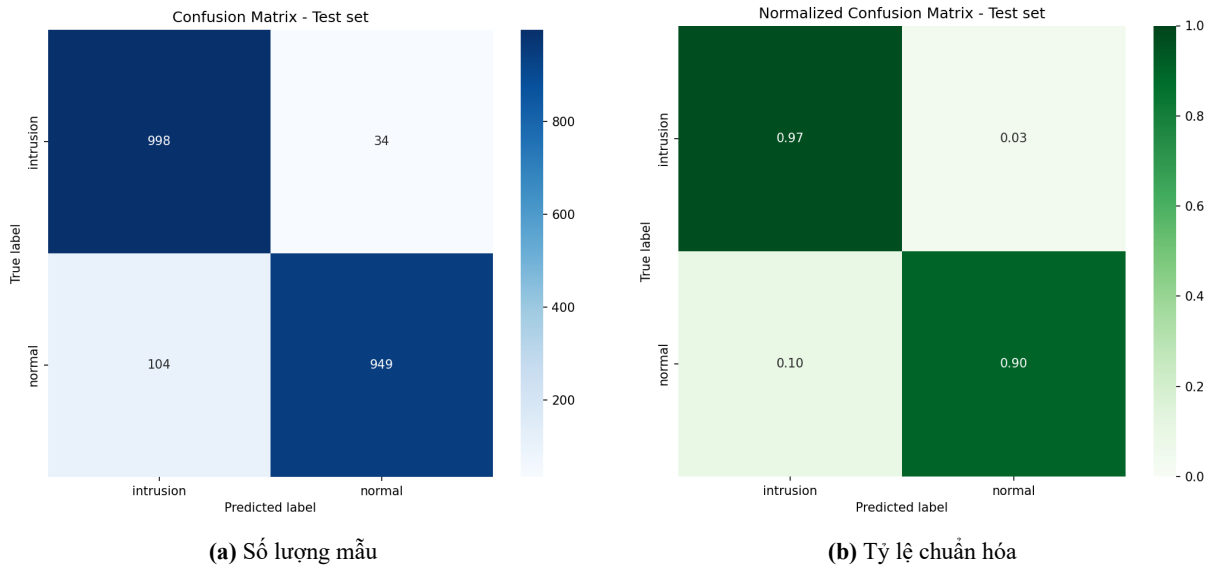
Hình 5.5. Confusion matrix của best model trên tập validation.

Trong 2.049 ảnh intrusion thực tế, mô hình phát hiện đúng 1.941 ảnh, đạt recall khoảng 94,73%, và bỏ sót 108 ảnh (False Negative). Trong 2.080 ảnh normal, có 63 ảnh bị nhầm thành intrusion (False Positive), tương ứng tỷ lệ cảnh báo nhầm trên ảnh normal khoảng 3,03%, trong khi precision của lớp intrusion đạt khoảng 96,86%.

False Negative là loại lỗi cần quan tâm nhất vì có thể làm hệ thống không phát cảnh báo ở một frame xâm nhập (**Hình 5.5**). Kết quả cho thấy mô hình nhận diện lớp top hơn lớp dưới một mức nhỏ, số False Negative lớn hơn False Positive, vì vậy hệ thống ưu tiên giảm bỏ sót xâm nhập trong các phiên bản tiếp theo. Tuy nhiên, confusion matrix được tính trên từng ảnh crop độc lập, trong khi sự kiện leo rào trên video kéo dài qua nhiều frame. Cơ chế đếm số khung hình xâm nhập liên tiếp giúp giảm ảnh hưởng của các khung hình bị phân loại sai đơn lẻ, chỉ xác nhận xâm nhập khi số khung hình liên tiếp đạt ngưỡng đã thiết lập.

5.4. KẾT QUẢ TEST MÔ HÌNH

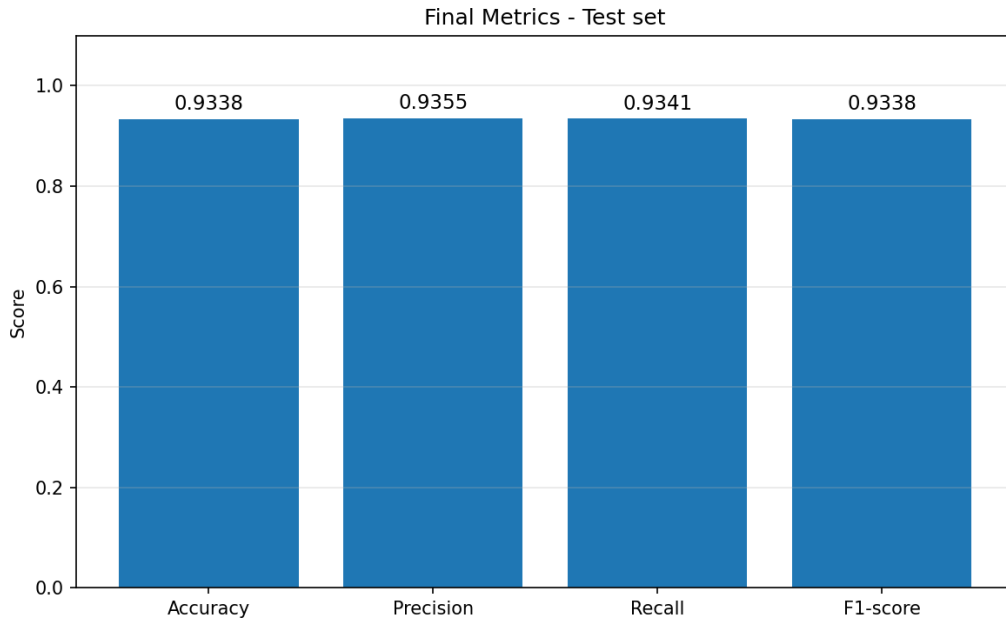
Hình 5.6 trình bày kết quả đánh giá mô hình trên tập test nhằm kiểm tra khả năng phân loại của hệ thống trên dữ liệu chưa được sử dụng trong quá trình huấn luyện và validation. Kết quả đánh giá trên tập test giúp phản ánh khách quan hơn khả năng tổng quát hóa của mô hình khi gặp dữ liệu mới.



Hình 5.6. Ma trận nhầm lẫn của mô hình trên tập test: (a) số lượng mẫu, (b) tỷ lệ chuẩn hóa

Tập test gồm tổng cộng 2.085 mẫu, trong đó có 1.032 mẫu thuộc lớp intrusion và 1.053 mẫu thuộc lớp normal. Mô hình dự đoán đúng 998 mẫu intrusion và 949 mẫu normal, tương ứng tổng cộng 1.947 mẫu được phân loại chính xác; 138 mẫu còn lại bị phân loại sai, gồm 34 mẫu intrusion bị nhận nhầm thành normal và 104 mẫu normal bị nhận nhầm thành intrusion.

Xét theo tỷ lệ chuẩn hóa, lớp intrusion đạt tỷ lệ nhận diện đúng khoảng 96,71% (998/1.032), số trường hợp xâm nhập bị bỏ sót chỉ chiếm khoảng 3,29%, đây là kết quả tích cực đối với bài toán cảnh báo xâm nhập, vì giảm số mẫu intrusion bị nhận nhầm thành normal giúp nâng cao độ tin cậy của hệ thống cảnh báo. Đối với lớp normal, mô hình nhận diện đúng 949/1.053 mẫu, đạt tỷ lệ khoảng 90,12%, còn 104 mẫu (khoảng 9,88%) bị nhận nhầm thành intrusion. Mặc dù nhóm lỗi này có thể làm tăng số cảnh báo sai trong quá trình vận hành thực tế, tỷ lệ nhận diện đúng của cả hai lớp vẫn ở mức cao, cho thấy mô hình đã học được đặc trưng phân biệt tương đối rõ giữa hành vi bình thường và hành vi leo rào.



Hình 5.7. Tổng hợp các chỉ số đánh giá mô hình trên tập test

Hình 5.7 trình bày các chỉ số tổng hợp của mô hình trên tập test. Độ chính xác tổng thể (Accuracy) đạt 0,9338, tương đương 93,38%. Chỉ số Precision đạt 0,9355, Recall đạt 0,9341 và F1-score đạt 0,9338. Các chỉ số này có giá trị khá gần nhau, cho thấy mô hình duy trì được sự cân bằng giữa khả năng phát hiện đúng mẫu xâm nhập và khả năng hạn chế nhầm lẫn đối với mẫu bình thường.

Từ kết quả đánh giá trên tập test, có thể thấy full pipeline của hệ thống đạt hiệu quả phân loại khá tốt với độ chính xác khoảng 93,38%. Các chỉ số Precision, Recall và F1-score tương đối cân bằng, cho thấy mô hình hoạt động ổn định trên cả hai lớp dữ liệu. Tuy nhiên, hệ thống vẫn còn xảy ra một số sai sót là bỏ sót trường hợp xâm nhập và cảnh báo nhầm lẫn đối với mẫu bình thường.

Nguyên nhân chủ yếu là do hiệu quả của full pipeline phụ thuộc vào toàn bộ quá trình từ phát hiện người, cắt vùng ảnh đến phân loại, nên các sai sót ở giai đoạn trước có thể ảnh hưởng đến kết quả cuối cùng. Trong thời gian tới, hệ thống có thể được cải thiện bằng cách bổ sung thêm các mẫu dữ liệu khó, tối ưu bước tiền xử lý và điều chỉnh ngưỡng quyết định để giảm tỷ lệ bỏ sót xâm nhập và hạn chế cảnh báo giả.

Nhìn chung, kết quả thực nghiệm cho thấy hệ thống đã có khả năng phân biệt tương đối tốt giữa hai trạng thái intrusion và normal, đáp ứng yêu cầu cơ bản của bài toán giám sát xâm nhập và có tiềm năng tiếp tục nâng cao hiệu năng khi được tối ưu thêm.

5.5. SO SÁNH VÀ ĐÁNH GIÁ VỚI CÁC NGHIÊN CỨU LIÊN QUAN

Để xác định vị trí của kết quả thực nghiệm trong bối cảnh nghiên cứu hiện nay, đề tài đối chiếu với ba công trình liên quan đến nhận dạng hành động bất thường trên video và phát hiện xâm nhập trên thiết bị nhúng. Khác với phần đánh giá trước đó, mục này sử dụng kết quả trên tập test để so sánh khách quan hơn với các nghiên cứu tham khảo.

Bảng 5.2. So sánh kết quả test của đề tài với các nghiên cứu liên quan.

Nghiên cứu	Bài toán và phương pháp	Kết quả công bố
Đề tài này	Phân loại hai lớp <i>normal/intrusion</i> trên ảnh người do YOLOv8 crop; MobileNetV2 và Intrusion Frame Counter; triển khai trên Raspberry Pi 4.	Test set: Accuracy 93,38%; Precision macro 93,55%; Recall macro 93,41%; F1-score macro 93,38%; Recall intrusion 96,71%; pipeline 4–5 FPS.
Vrskova và cộng sự (2022) [11]	ConvLSTM nhận dạng 11 nhóm hành vi bất thường từ chuỗi video trên bộ dữ liệu do nhóm tác giả xây dựng.	Test/evaluation set: Accuracy 96,19%; Precision 96,50%.
Ullah và Munir (2023) [12]	Dual-Attention CNN kết hợp Bi-GRU để học đặc trưng không gian–thời gian của hành động trong video.	Test set: Accuracy 98,5% trên UCF50; kích thước khoảng 5,4 MB; tốc độ công bố đến 250 FPS trên CPU thử nghiệm.
Wang và Yu (2021) [13]	SSD với MobileNet cải tiến để phát hiện người hoặc vật thể xâm nhập đường sắt; triển khai TensorRT trên Jetson TX2.	Test/evaluation set: mAP 89,60%; 35,63 ms/frame ở FP16, tương đương khoảng 28 FPS.

5.5.1. So sánh độ chính xác nhận dạng

So với ConvLSTM của Vrskova và cộng sự, Accuracy và Precision của đề tài thấp hơn một mức nhất định. Tuy nhiên, hai bài toán không hoàn toàn giống nhau: nghiên cứu của Vrskova tập trung nhận dạng nhiều nhóm hành vi bất thường từ chuỗi video, trong khi đề tài chỉ phân loại hai lớp *normal* và *intrusion* trên ảnh crop, đồng thời ưu tiên khả năng triển khai thực tế trên Raspberry Pi 4 nên lựa chọn MobileNetV2 nhằm cân bằng độ chính xác và tốc độ xử lý.

So với DA-CNN+Bi-GRU của Ullah và Munir, Accuracy của đề tài cũng thấp hơn. Nguyên nhân chính là mô hình này học trực tiếp đặc trưng không gian–thời gian từ chuỗi frame video bằng CNN attention và Bi-GRU, trong khi MobileNetV2 chỉ phân loại ảnh crop độc lập; vai trò chuyển động liên tục được đảm nhiệm bởi Intrusion Frame Counter thay vì học trực tiếp. Cách tiếp cận này đơn giản hơn nhưng đánh đổi một phần độ chính xác để giảm độ phức tạp và tài nguyên khi triển khai trên thiết bị nhúng.

Nhìn chung, kết quả Accuracy, Precision, Recall và F1-score của đề tài đều ở mức 93–94%, phù hợp với mục tiêu ứng dụng thực tế. Đáng chú ý, Recall intrusion đạt 96,71%, phản ánh khả năng phát hiện tốt các trường hợp xâm nhập là tiêu chí quan trọng nhất của một hệ thống cảnh báo an ninh.

5.5.2. Đánh giá khả năng triển khai Edge AI

Nghiên cứu của Wang và Yu cung cấp tham chiếu về triển khai Edge AI cho bài toán phát hiện xâm nhập, đạt mAP 89,60% với thời gian xử lý 35,63 ms/frame trên Jetson TX2. Tuy nhiên, chỉ số mAP của object detection không tương đương với Accuracy/F1-score của bài toán phân loại hành động, nên kết quả này chủ yếu mang tính tham khảo về khả năng triển khai mô hình thị giác máy tính trên thiết bị biên, không so sánh trực tiếp độ chính xác nhận dạng.

So với Jetson TX2 (có GPU CUDA và TensorRT), Raspberry Pi 4 hạn chế hơn đáng kể về năng lực tính toán. Dù vậy, đề tài vẫn triển khai được đầy đủ pipeline YOLOv8n–MobileNetV2–Intrusion Frame Counter kết

hợp cập nhật Web Dashboard và điều khiển GPIO, đạt tốc độ 4–5 FPS với độ trễ 0,9–1,7 giây là mức được xem là phù hợp với bài toán phát hiện leo rào, vốn diễn ra liên tục qua nhiều frame thay vì xuất hiện tức thời.

Điểm mạnh của đề tài nằm ở khả năng tích hợp hoàn chỉnh trên thiết bị chi phí thấp, từ xử lý hình ảnh, ra quyết định, kích hoạt cảnh báo phần cứng đến hiển thị trạng thái trên Web trong cùng một pipeline. Hướng cải tiến tiếp theo có thể tập trung giảm False Positive của lớp normal, tối ưu ngưỡng Intrusion Frame Counter, và benchmark phiên bản INT8 hoặc giảm kích thước đầu vào YOLOv8n để nâng cao tốc độ xử lý.

5.5.3. Nhận xét chung

Kết quả trên tập test cho thấy mô hình đạt hiệu năng ổn định đối với bài toán phát hiện hành vi leo rào, với Accuracy 93,38%, F1-score macro 93,38% và Recall intrusion 96,71%. Đặc biệt, Recall của lớp intrusion ở mức cao cho thấy hệ thống có khả năng phát hiện phần lớn các trường hợp xâm nhập, phù hợp với mục tiêu ưu tiên của bài toán an ninh là hạn chế bỏ sót cảnh báo nguy hiểm.

So với các nghiên cứu liên quan, kết quả của đề tài chưa nhằm khẳng định mô hình vượt trội tuyệt đối, vì mỗi công trình sử dụng bộ dữ liệu, số lớp, phần cứng triển khai và chỉ số đánh giá khác nhau. Tuy nhiên, kết quả test đạt được cho thấy hướng tiếp cận YOLOv8n–MobileNetV2–Intrusion Frame Counter có tính khả thi trong điều kiện triển khai trên Raspberry Pi 4. Hệ thống đạt sự cân bằng tương đối tốt giữa độ chính xác nhận dạng, tốc độ xử lý, chi phí phần cứng và khả năng ứng dụng thực tế trong giám sát an ninh gần thời gian thực.

Trong các nghiên cứu và phiên bản tiếp theo, có thể mở rộng đánh giá theo cấp độ sự kiện video, chẳng hạn tỷ lệ phát hiện đúng từng sự kiện xâm nhập, số cảnh báo giả mỗi giờ và thời gian phát hiện trung bình. Những chỉ số này sẽ giúp phản ánh sát hơn hiệu quả vận hành thực tế của hệ thống so với việc chỉ đánh giá trên từng ảnh crop riêng lẻ.

5.5.4. Đánh giá vai trò của Intrusion Frame Counter

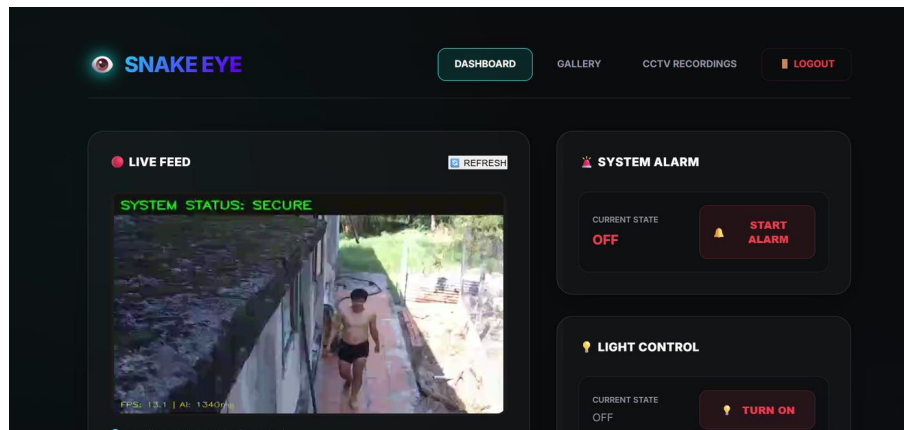
Trong quá trình xử lý video thực tế, kết quả phân loại của MobileNetV2 có thể dao động do sự thay đổi về tư thế người, điều kiện ánh sáng, hiện tượng che khuất hoặc chất lượng vùng ảnh crop. Nếu hệ thống đưa ra quyết định dựa trên từng khung hình độc lập, các dự đoán sai đơn lẻ có thể dẫn đến cảnh báo giả hoặc làm trạng thái cảnh báo thay đổi liên tục.

Để khắc phục hiện tượng này, hệ thống sử dụng Intrusion Frame Counter để đếm số lần phát hiện intrusion liên tiếp. Cảnh báo chỉ được kích hoạt khi kết quả phân loại intrusion xuất hiện liên tiếp 2 lần, qua đó loại bỏ ảnh hưởng của các dự đoán sai xảy ra ở một khung hình đơn lẻ.

Qua quá trình thực nghiệm, cơ chế này cho thấy khả năng giảm đáng kể số lượng cảnh báo giả trong các tình huống bình thường, đồng thời vẫn đảm bảo phát hiện kịp thời khi xuất hiện hành vi leo rào thực sự. Mặc dù việc yêu cầu hai lần phát hiện liên tiếp có thể làm tăng độ trễ cảnh báo một khoảng rất nhỏ, nhưng đổi lại hệ thống đạt được tính ổn định và độ tin cậy cao hơn trong quá trình vận hành thực tế.

5.6. KẾT QUẢ TRÊN GIAO DIỆN WEB GIÁM SÁT

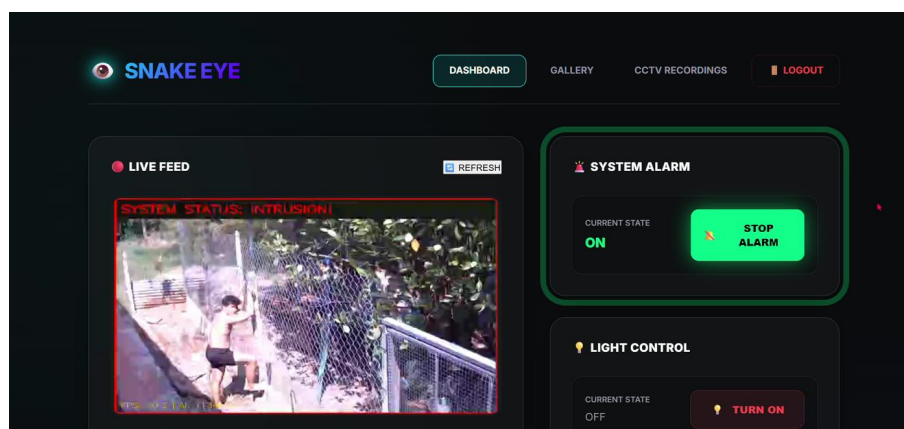
5.6.1. Dashboard ở trạng thái bình thường



Hình 5.8. Web Dashboard ở trạng thái giám sát bình thường.

Dashboard hiển thị video trực tiếp (Hình 5.8), trạng thái hệ thống, trạng thái buzzer, điều khiển đèn và các thông tin thống kê. Khi không có xâm nhập, video hiển thị SYSTEM STATUS: SECURE, cảnh báo ở trạng thái OFF và người dùng vẫn có thể điều khiển thiết bị bằng các nút trên giao diện.

5.6.2. Dashboard khi phát hiện intrusion

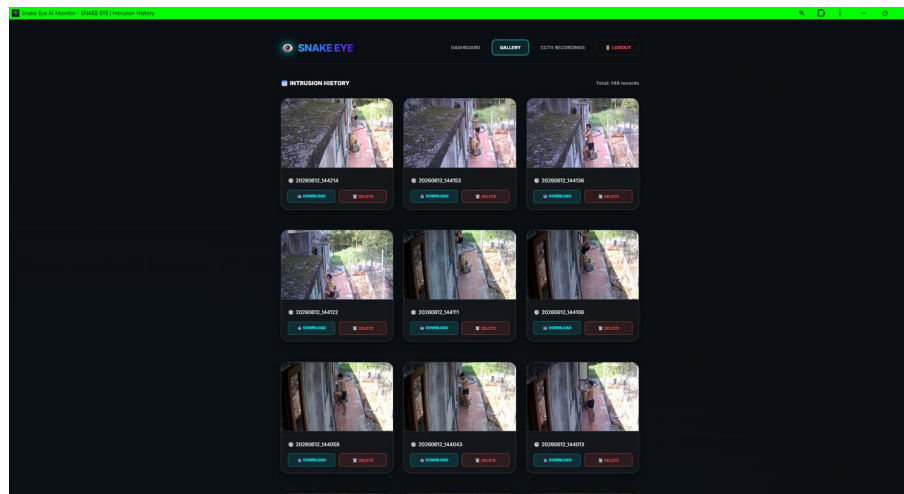


Hình 5.9. Web Dashboard khi hệ thống xác nhận trạng thái intrusion.

Khi hệ thống xác nhận có hành vi xâm nhập (Hình 5.9), còi báo động sẽ được kích hoạt ngay lập tức và trạng thái System Alarm trên Web Dashboard được chuyển sang ON để cảnh báo người quản trị. Đồng thời, backend cũng cập nhật kết quả phân loại lên luồng video giám sát bằng cách hiển thị dòng chữ "SYSTEM STATUS: INTRUSION" và viền khung hình màu đỏ

Do luồng video được truyền phát bằng giao thức HLS, cơ chế phát này sử dụng bộ đệm (buffer) để đảm bảo video phát ổn định nên thường tồn tại độ trễ khoảng vài giây so với thời gian xử lý thực tế. Vì vậy, âm thanh cảnh báo và trạng thái System Alarm trên Dashboard sẽ xuất hiện trước, trong khi thông báo "SYSTEM STATUS: INTRUSION" trên video có thể hiển thị muộn hơn do ảnh hưởng của quá trình đệm HLS. Mặc dù có sự chênh lệch nhỏ về thời gian hiển thị, cả hai cơ chế đều phản ánh cùng một sự kiện xâm nhập được hệ thống phát hiện và xử lý.

5.6.3. Lịch sử sự kiện



Hình 5.10. Giao diện lịch sử ảnh sự kiện intrusion.

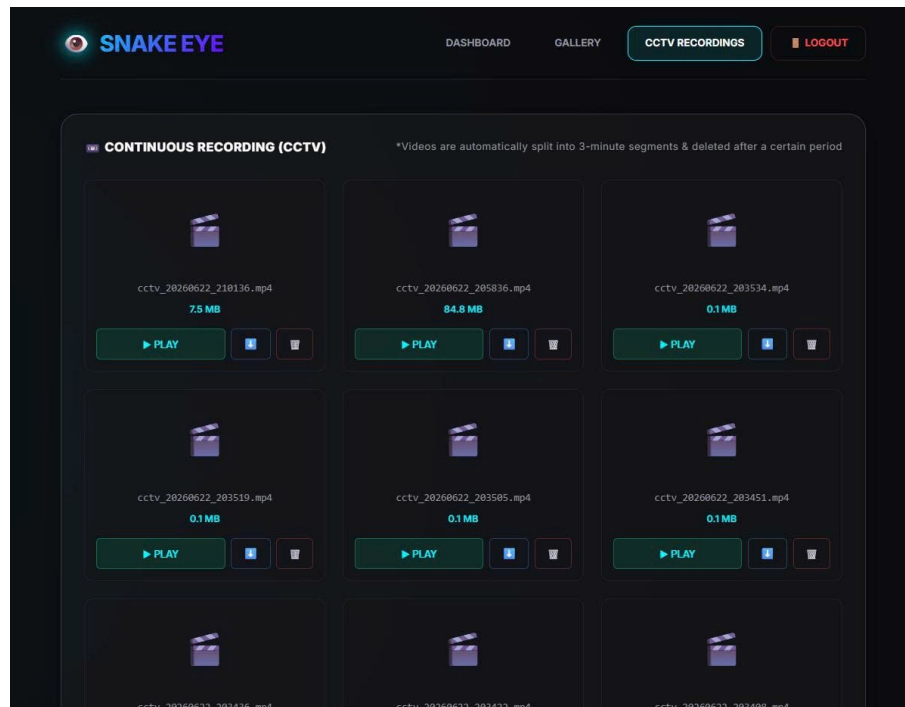
Hình 5.10 trình bày giao diện lịch sử sự kiện của hệ thống sau khi phát hiện trạng thái *intrusion*. Khi mô hình nhận diện có hành động xâm nhập vượt ngưỡng quy định đã thiết lập, hệ thống sẽ tự động lưu lại ảnh sự kiện kèm theo các thông tin liên quan như thời gian xảy ra, nhãn dự đoán và trạng thái nhận diện. Các sự kiện này được hiển thị dưới dạng danh sách ảnh trên giao diện web, giúp người dùng dễ dàng theo dõi và kiểm tra lại những lần hệ thống phát hiện xâm nhập.

Giao diện lịch sử sự kiện cho phép người dùng quan sát nhanh các ảnh đã được lưu khi hệ thống phát hiện có hành vi đột nhập. Mỗi bản ghi sự kiện thể hiện một tình huống được hệ thống phát hiện, giúp người vận hành không cần theo dõi video liên tục nhưng vẫn có thể nắm bắt các trường hợp bất thường đã xảy ra.

Ngoài chức năng xem lại, giao diện còn hỗ trợ thao tác tải xuống hoặc xóa từng bản ghi khi cần thiết. Chức năng tải xuống giúp lưu trữ minh chứng phục vụ cho việc kiểm tra, báo cáo hoặc phân tích sau này. Trong khi đó, chức năng xóa giúp loại bỏ những bản ghi không cần thiết, góp phần quản lý dữ liệu lịch sử gọn gàng hơn.

Kết quả thực nghiệm cho thấy chức năng lưu lịch sử sự kiện hoạt động ổn định và phù hợp với mục tiêu của hệ thống giám sát xâm nhập. Việc lưu lại ảnh sự kiện giúp tăng tính tiện dụng của hệ thống, hỗ trợ truy vết các tình huống đã xảy ra và giảm phụ thuộc vào việc người dùng phải quan sát camera theo thời gian thực.

5.6.4. Lịch sử sự kiện



Hình 5.11. Giao diện lưu trữ và xem lại video sự kiện

Kết quả thực nghiệm cho thấy chức năng lưu trữ video sự kiện hoạt động ổn định (Hình 5.11). Khi hệ thống phát hiện hành vi xâm nhập, video được tự động ghi lại và lưu trữ thành công trên máy chủ. Người dùng có thể truy cập giao diện Web Dashboard để xem lại các video đã lưu cùng với thông tin thời gian xảy ra sự kiện, giúp thuận tiện cho việc theo dõi và kiểm tra sau này.

5.7. KẾT QUẢ PHẢN HỒI PHẦN CỨNG VÀ ĐỘ TRỄ

5.7.1. Phản hồi ở trạng thái normal và intrusion

Ở trạng thái Normal, loa cảnh báo không hoạt động và hệ thống tiếp tục duy trì chế độ giám sát. Khi phát hiện trạng thái Intrusion vượt ngưỡng đã thiết lập, Raspberry Pi xuất tín hiệu điều khiển qua chân GPIO_BUZZER để kích hoạt loa cảnh báo. Việc kích hoạt phần cứng được thực hiện trực tiếp dựa trên kết quả nhận diện xâm nhập từ mô hình AI, giúp hệ thống phát ra cảnh báo kịp thời khi có hành vi leo rào.

Khi hệ thống phát hiện xâm nhập, trạng thái cảnh báo sẽ được duy trì trong khoảng thời gian đã cấu hình trước. Sau khi hết thời gian này hoặc khi người dùng gửi lệnh tắt cảnh báo hợp lệ từ giao diện Web Dashboard, tín hiệu điều khiển tại chân GPIO_BUZZER sẽ được đưa về mức an toàn, loa cảnh báo ngừng hoạt động và hệ thống quay trở lại trạng thái giám sát bình thường.

Kết quả kiểm tra cho thấy phần mềm AI, Web Dashboard và phần cứng chấp hành có thể hoạt động liên kết theo chuỗi:

Camera → Raspberry Pi → YOLOv8 → crop người → MobileNetV2 → Intrusion Frame Counter → GPIO/Web → cảnh báo.

5.7.2. Độ trễ end-to-end

Bảng 5.3. Độ trễ end-to-end đo được của hệ thống.

Giai đoạn	Thời gian
Camera ghi nhận → AI phân loại intrusion	Khoảng 0,2–0,4 giây
AI phân loại → Web cập nhật trạng thái	Khoảng 0,7–1,3 giây
AI phân loại → phần cứng kích hoạt cảnh báo	Khoảng 0,01–0,05 giây
Tổng thời gian end-to-end	Khoảng 0,9–1,7 giây

Từ kết quả đo thực nghiệm có thể thấy tổng độ trễ từ khi camera ghi nhận hình ảnh đến khi hệ thống AI đưa ra quyết định và kích hoạt cảnh báo dao động trong khoảng 0,9–1,7 giây, đáp ứng yêu cầu giám sát gần thời gian thực. Thành phần chiếm nhiều thời gian xử lý nhất là pipeline AI, bao gồm giai đoạn phát hiện người bằng YOLOv8 và phân loại trạng thái bằng MobileNetV2.

Sau khi AI hoàn thành suy luận, trạng thái cảnh báo được cập nhật lên Web Dashboard và tín hiệu điều khiển GPIO được kích hoạt gần như tức thời, với độ trễ rất nhỏ so với thời gian suy luận của mô hình. Riêng luồng video hiển thị trên Dashboard sử dụng giao thức HLS, do cơ chế bộ đệm để đảm bảo phát ổn định nên có thể xuất hiện trễ hơn vài giây so với thời điểm hệ thống kích hoạt cảnh báo.

5.8. ĐÁNH GIÁ TỐC ĐỘ XỬ LÝ VÀ TÀI NGUYÊN

5.8.1. Tốc độ inference trên Raspberry Pi 4

Bảng 5.4. Tốc độ xử lý của các mô hình trên Raspberry Pi 4

Tiêu chí	YOLOv8n ONNX	MobileNetV2 ONNX float32
Chức năng	Phát hiện người	Phân loại hành động
Tốc độ riêng	Khoảng 5–6 FPS	Khoảng 15–20 crop/giây
Thời gian inference	Khoảng 180 ms/frame	Khoảng 55 ms/crop
Kích thước file	12,3 MB	8,8 MB

YOLOv8n là thành phần giới hạn tốc độ chính vì phải xử lý toàn bộ frame. MobileNetV2 chỉ xử lý vùng crop người nên có thời gian inference ngắn hơn. Pipeline hoàn chỉnh đạt khoảng 4–5 FPS khi sử dụng YOLOv8n ONNX và cấu hình SKIP_FRAMES = 5, tức xử lý AI trên một phần các frame trong khi luồng camera và ghi hình vẫn tiếp tục.

Tổng kích thước hai mô hình được đo trong cấu hình này khoảng 21,1 MB, phù hợp với bộ nhớ và lưu trữ của Raspberry Pi 4. Kết quả cho thấy hệ thống đạt được sự cân bằng giữa độ chính xác và tốc độ cho bài toán giám sát gần thời gian thực.

5.8.2. Đánh giá phiên bản Float32

Trong quá trình thực nghiệm, mô hình MobileNetV2 được triển khai và đánh giá chủ yếu ở định dạng ONNX float32. Đây là phiên bản giữ nguyên độ chính xác số học của mô hình sau khi chuyển đổi từ PyTorch sang ONNX, do đó phù hợp để kiểm tra khả năng suy luận ban đầu trước khi áp dụng các kỹ thuật tối ưu hóa như lượng tử hóa INT8.

Kết quả đo trong **Bảng 5.4** cho thấy MobileNetV2 ONNX float32 có kích thước khoảng 8,8 MB, thời gian suy luận trung bình khoảng 55 ms/crop, tương ứng với tốc độ xử lý khoảng 15–20 crop/giây. Với vai trò là mô hình phân loại hành động sau khi YOLOv8 đã phát hiện và crop vùng ảnh người, tốc độ này đáp ứng được yêu cầu xử

lý trên Raspberry Pi 4 trong điều kiện số lượng người xuất hiện trong frame không quá lớn.

Ưu điểm của phiên bản float32 là đảm bảo độ ổn định và độ chính xác của kết quả phân loại, hạn chế sai lệch do làm tròn trọng số hoặc thay đổi kiểu dữ liệu. Vì vậy, phiên bản này được sử dụng làm cơ sở để đánh giá hiệu năng ban đầu của hệ thống và làm mốc so sánh cho các phiên bản tối ưu hơn trong tương lai.

Tuy nhiên, do vẫn sử dụng kiểu dữ liệu float32, mô hình có kích thước và mức sử dụng bộ nhớ lớn hơn so với phiên bản INT8. Điều này có thể ảnh hưởng đến hiệu năng khi triển khai trên thiết bị nhúng có tài nguyên hạn chế. Vì vậy, trong các hướng phát triển tiếp theo, cần benchmark thêm phiên bản INT8 trên cùng Raspberry Pi 4, cùng tập dữ liệu và cùng cấu hình ONNX Runtime để so sánh cụ thể về tốc độ, dung lượng mô hình và độ chính xác phân loại.

5.9. ĐÁNH GIÁ CHUNG

Khi giám sát qua internet, video stream bị trễ khoảng 3 giây so với thực tế do hệ thống sử dụng HLS với buffer phát lại nhằm tránh hiện tượng giật, lag khi browser đọc segment nhanh hơn tốc độ segment mới được tạo ra. Phần overlay trạng thái lên khung hình cũng phụ thuộc vào tốc độ xử lý AI, nên có độ trễ tương ứng với thời gian xử lý mỗi frame. Tuy nhiên, khi phát hiện xâm nhập, còi báo động sử dụng giao thức HTTP nên có thể kích hoạt gần như tức thời, trong khi trạng thái đột nhập trên luồng video hiển thị cho người dùng vẫn trễ hơn một chút do phải qua buffer của HLS.

5.9.1. Ưu điểm

Hệ thống đã được xây dựng và tích hợp hoàn chỉnh theo mô hình end-to-end, gồm mô hình AI, Raspberry Pi 4, camera giám sát, phần cứng cảnh báo, chức năng ghi hình và Web Dashboard phục vụ giám sát từ xa. Mô hình MobileNetV2 đạt Accuracy 93,38% và F1-score macro 0,9338 trên tập test, cho thấy khả năng phân loại hành vi xâm nhập đạt hiệu quả tốt trong điều kiện kiểm thử độc lập.

Dữ liệu huấn luyện được xây dựng từ các vùng ảnh người đã được crop bởi YOLOv8, phù hợp với dữ liệu đầu vào của bộ phân loại trong thực tế, góp phần nâng cao khả năng tổng quát hóa của mô hình. Hệ thống cũng áp dụng cơ chế Intrusion Frame Counter, chỉ kích hoạt cảnh báo khi hành vi Intrusion được phát hiện liên tiếp hai lần, giúp giảm đáng kể cảnh báo giả do sai lệch ở một số khung hình riêng lẻ.

Thời gian phản hồi của tín hiệu GPIO nhỏ hơn 100ms, đáp ứng tốt yêu cầu kích hoạt cảnh báo tại chỗ. Web Dashboard hỗ trợ hiển thị video trực tiếp, theo dõi trạng thái hệ thống, điều khiển thiết bị và lưu trữ lịch sử sự kiện. Toàn bộ pipeline đạt tốc độ xử lý khoảng 4–5 FPS với tổng độ trễ từ 0,9 đến 1,7 giây, đáp ứng yêu cầu giám sát gần thời gian thực trên Raspberry Pi 4.

5.9.2. Hạn chế

Trên tập test, Recall của lớp intrusion đạt khoảng 96,71%, nhưng vẫn còn 34 ảnh xâm nhập bị phân loại nhầm thành normal, cho thấy mô hình có thể bỏ sót một số trường hợp xâm nhập trong điều kiện nhất định.

Hiệu năng xử lý của pipeline phụ thuộc chủ yếu vào YOLOv8n, đặc biệt khi xử lý khung hình độ phân giải cao hoặc nhiều đối tượng cùng lúc. Cơ chế frame skipping giúp giảm tải nhưng có thể bỏ qua các hành động diễn ra nhanh. Ngoài ra, chất lượng nhận dạng còn chịu ảnh hưởng bởi điều kiện ánh sáng, góc camera, vật cản và độ chính xác bounding box của YOLOv8.

Cuối cùng, các kết quả tốc độ chỉ được đo với MobileNetV2 ONNX float32, phiên bản lượng tử hóa INT8 chưa được đánh giá riêng biệt. Hệ thống cũng chưa được kiểm chứng dài hạn với số lượng lớn sự kiện thực tế để đánh giá đầy đủ tỷ lệ phát hiện đúng, báo động giả và độ ổn định khi vận hành liên tục.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. KẾT LUẬN

Sau quá trình nghiên cứu, thiết kế, xây dựng và thực nghiệm, đề tài đã hoàn thành mục tiêu xây dựng hệ thống phát hiện và cảnh báo xâm nhập qua hành động leo rào trên nền tảng thiết bị nhúng Raspberry Pi 4. Hệ thống được tổ chức theo pipeline gồm camera thu nhận hình ảnh, YOLOv8 phát hiện người, MobileNetV2 phân loại hành động, cơ chế ổn định kết quả theo thời gian, Web Dashboard giám sát và khối phần cứng cảnh báo, giúp xử lý luồng dữ liệu từ camera đến cảnh báo theo một chuỗi hoàn chỉnh, phù hợp với yêu cầu giám sát gần thời gian thực.

Về mặt lý thuyết, đề tài đã vận dụng các kiến thức về thị giác máy tính, học sâu và transfer learning làm cơ sở để lựa chọn mô hình phù hợp và xây dựng quy trình huấn luyện triển khai trên thiết bị tài nguyên hạn chế.

Về kết quả thực nghiệm, mô hình MobileNetV2 đạt độ chính xác cao trên tập kiểm thử, đặc biệt thể hiện khả năng nhận diện hành vi xâm nhập tốt. Khi triển khai toàn hệ thống, pipeline YOLOv8–MobileNetV2 hoạt động ổn định trên Raspberry Pi 4 với tốc độ và độ trễ phản hồi đáp ứng được yêu cầu giám sát gần thời gian thực trong phạm vi thực nghiệm.

Bên cạnh các kết quả đạt được, hệ thống vẫn tồn tại một số hạn chế. Mô hình còn bỏ sót một số trường hợp xâm nhập trong điều kiện thiếu sáng, góc quay khó hoặc vùng người bị che khuất. Tốc độ xử lý toàn pipeline còn phụ thuộc nhiều vào YOLOv8 khi xử lý toàn bộ khung hình, và luồng video hiển thị trên Web có độ trễ quan sát cao hơn so với thực tế.

Nhìn chung, đề tài đã chứng minh tính khả thi của việc ứng dụng trí tuệ nhân tạo vào bài toán giám sát an ninh tự động. Hệ thống không chỉ có ý nghĩa thực tiễn trong việc hỗ trợ phát hiện hành vi leo rào, mà còn là nền tảng để phát triển các giải pháp giám sát thông minh để triển khai và có khả năng mở rộng trong thực tế.

6.2. HƯỚNG PHÁT TRIỂN

Trong tương lai, hệ thống có thể được tiếp tục phát triển theo nhiều hướng nhằm nâng cao độ chính xác, hiệu năng và khả năng ứng dụng thực tế. Trước hết, tập dữ liệu huấn luyện cần mở rộng với các tình huống phức tạp hơn như thiếu sáng, đối tượng bị che khuất, khoảng cách xa, nhiều góc quan sát và các tư thế leo rào chuyển tiếp, nhằm giảm tỷ lệ bỏ sót False Negative và tăng khả năng tổng quát hóa. Bên cạnh đánh giá trên tập ảnh crop, hệ thống cần được kiểm thử theo sự kiện video thực tế để xác định tỷ lệ phát hiện đúng sự kiện, tỷ lệ cảnh báo giả theo thời gian và thời gian phát hiện trung bình, nhằm phản ánh chính xác hơn hiệu quả khi triển khai thực tế.

Về tối ưu hóa, có thể so sánh hiệu năng giữa MobileNetV2 ONNX float32 và phiên bản lượng tử hóa, đồng thời tối ưu YOLOv8 bằng cách giảm kích thước đầu vào, dùng phiên bản nhẹ hơn hoặc khai thác AI Accelerator tương thích Raspberry Pi để tăng tốc độ xử lý. Hệ thống cũng có thể tích hợp thêm cảm biến chuyển động hoặc cảm biến rung gắn trên hàng rào, kết hợp với kết quả AI để tăng độ tin cậy và giảm cảnh báo giả trong điều kiện môi trường phức tạp. Chức năng cảnh báo có thể mở rộng qua thông báo từ xa đến điện thoại, Telegram, email hoặc các nền tảng nhắn tin khác.

Cuối cùng, để vận hành lâu dài, hệ thống cần bổ sung cơ chế giám sát tiến trình, tự động khởi động lại khi gặp sự cố, lưu nhật ký hoạt động và theo dõi tình trạng thiết bị, đồng thời tăng cường bảo mật Web Dashboard qua quản lý tài khoản, mã hóa kết nối và chính sách lưu trữ dữ liệu an toàn.

TÀI LIỆU THAM KHẢO

- [1] Y. Wang and P. Yu, "A fast intrusion detection method for high-speed railway clearance based on low-cost embedded GPUs," *Sensors*, vol. 21, no. 21, p. 7279, 2021.
- [2] Ultralytics, "Ultralytics YOLOv8," Ultralytics Documentation, 2023, accessed: Jun. 15, 2026. [Online]. Available: <https://docs.ultralytics.com/models/yolov8/>
- [3] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248--255.
- [4] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 8024--8035. [Online]. Available: <https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>
- [5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://openreview.net/forum?id=Bkg6RiCqY7>
- [6] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427--437, 2009.
- [7] Microsoft, "ONNX Runtime: Cross-platform machine learning model accelerator," ONNX Runtime Documentation, accessed: Jun. 15, 2026. [Online]. Available: <https://onnxruntime.ai/docs/>
- [8] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436--444, 2015.
- [9] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510--4520.
- [10] OpenCV Team, "OpenCV documentation," Open Source Computer Vision Library, accessed: Jun. 15, 2026. [Online]. Available: <https://docs.opencv.org/4.x/>
- [11] Raspberry Pi Ltd, *Raspberry Pi 4 Model B Product Brief*, Raspberry Pi Ltd, Apr. 2026, accessed: Jun. 15, 2026. [Online]. Available: <https://pip.raspberrypi.com/documents/RP-008344-DS-raspberry-pi-4-product-brief.pdf>
- [12] R. Vrskova, R. Hudec, P. Kamencay, and P. Sykora, "A new approach for abnormal human activities recognition based on ConvLSTM architecture," *Sensors*, vol. 22, no. 8, p. 2946, 2022.
- [13] H. Ullah and A. Munir, "Human activity recognition using cascaded dual attention CNN and bi-directional GRU framework," *Journal of Imaging*, vol. 9, no. 7, p. 130, 2023.

- [14] G. Bradski and A. Kaehler, *Learning OpenCV 3: Computer Vision in C++ with the OpenCV Library*. O'Reilly Media, 2017.
- [15] H. Schulzrinne, A. Rao, and R. Lanphier, "RFC 2326: Real time streaming protocol (RTSP)," Internet Engineering Task Force, Request for Comments 2326, Apr. 1998. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2326>
- [16] E. Rescorla, "RFC 8446: The transport layer security (TLS) protocol version 1.3," Internet Engineering Task Force, Proposed Standard 8446, Aug. 2018. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8446>
- [17] Apple Inc., "HTTP Live Streaming (HLS) authoring specification," Apple Developer Documentation. [Online]. Available: <https://developer.apple.com/documentation/http-live-streaming/>
- [18] Cloudflare, Inc., "Cloudflare Tunnel documentation," Cloudflare One Documentation. [Online]. Available: <https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/>
- [19] A. M. Turing, "Computing machinery and intelligence," *Mind*, vol. LIX, no. 236, pp. 433--460, 1950.
- [20] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. [Online]. Available: <https://www.deeplearningbook.org/>
- [21] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779--788.
- [22] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345--1359, 2010.
- [23] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *Journal of Big Data*, vol. 6, no. 1, p. 60, 2019.
- [24] L. N. Smith and N. Topin, "Super-convergence: Very fast training of neural networks using large learning rates," arXiv preprint arXiv:1708.07120, 2017. [Online]. Available: <https://arxiv.org/abs/1708.07120>
- [25] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738--1762, 2019.
- [26] R. T. Fielding, M. Nottingham, and J. F. Reschke, "RFC 9110: HTTP semantics," Internet Engineering Task Force, Internet Standard 9110, Jun. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9110>

PHỤ LỤC

PHỤ LỤC A

MÃ NGUỒN VÀ DỮ LIỆU PHỤC VỤ HUẤN LUYỆN, KIỂM THỬ MÔ HÌNH AI

Do mã nguồn chương trình gồm nhiều tệp và có dung lượng tương đối lớn, báo cáo không trình bày toàn bộ mã nguồn trực tiếp trong nội dung chính. Thay vào đó, toàn bộ mã nguồn và các file liên quan được lưu trữ tại đường dẫn Google Drive sau:

https://drive.google.com/file/d/1j5RuByEng35R_pTwNcmVY7ROCcFkcVAO/view?usp=sharing

1. Các lệnh cài đặt thư viện trên Kaggle

Một số thư viện cần thiết được cài đặt trên Kaggle để phục vụ quá trình xử lý ảnh, huấn luyện mô hình, đánh giá kết quả và chuyển đổi mô hình sang định dạng ONNX.

```
!pip install ultralytics  
!pip install onnxscript onnx  
!pip install onnxruntime
```

2. Danh sách mã nguồn chương trình

Mã nguồn dùng YOLOv8 crop người để tạo dataset: Trước khi huấn luyện MobileNetV2, dữ liệu đầu vào cần được chuẩn hóa thành các ảnh crop vùng chứa người. Chương trình sử dụng YOLOv8 để phát hiện người trong ảnh hoặc video, sau đó lấy bounding box người và crop vùng ảnh tương ứng. Các ảnh sau khi crop được lưu vào thư mục dữ liệu theo từng lớp, gồm intrusion và normal.

Mã nguồn đầy đủ của phần này được lưu trong tệp: **yolov8.py**

Mã nguồn huấn luyện MobileNetV2 trên Kaggle: Mã nguồn huấn luyện MobileNetV2 được thực hiện trên Kaggle. Chương trình đọc dữ liệu từ cấu trúc thư mục dataset, thực hiện tiền xử lý ảnh, tăng cường dữ liệu, khởi tạo mô hình MobileNetV2, huấn luyện mô hình và lưu lại mô hình có kết quả tốt nhất trên tập validation.

Mã nguồn đầy đủ của phần huấn luyện được lưu trong tệp: **Mobilenetv2.py**

Mã nguồn kiểm thử mô hình trên tập test : Sau khi huấn luyện, mô hình được kiểm thử trên tập test riêng nhằm đánh giá khả năng phân loại trên dữ liệu chưa sử dụng trong quá trình huấn luyện. Chương trình đọc ảnh từ hai thư mục intrusion và normal, thực hiện dự đoán bằng mô hình đã huấn luyện, sau đó tính các chỉ số đánh giá và vẽ ma trận nhầm lẫn.

Mã nguồn đầy đủ của phần kiểm thử được lưu trong tệp: **Test.py**

Các file đầu ra sau khi huấn luyện mô hình: Sau quá trình huấn luyện và chuyển đổi mô hình, hệ thống tạo ra các file đầu ra phục vụ quá trình đánh giá và triển khai. Các file này được lưu trữ trong thư mục output của quá trình huấn luyện.

File đầy đủ của phần này sẽ được lưu trong thư mục **Output**

PHỤ LỤC B

TRIỂN KHAI HỆ THỐNG

Mã nguồn đầy đủ để triển khai hệ thống trên Raspberry Pi 4 được lưu trữ tại:

https://drive.google.com/file/d/1L7IW6WdhkIyLypvf-nR0i20qIB0gro5V/view?usp=drive_link

Video kết quả triển khai hệ thống trên thực tế được lưu trữ tại:

<https://drive.google.com/drive/folders/135xQfSbvvl9-9qTsFiALfG9qs0ldsRpP?usp=sharing>

1. Kết nối camera

Kết nối camera USB với một cổng USB 3.0 ở đầu phía trên của Raspberry Pi 4. Sau khi kết nối thành công, khởi động Raspberry Pi 4 để chạy chương trình.

2. Khởi động hệ thống Raspberry Pi 4

Để khởi động hệ thống, người dùng cấp nguồn điện 220V AC vào bo mạch nguồn của hệ thống. Sau khi được chuyển đổi và ổn áp, nguồn 5V DC từ bo mạch sẽ được cấp đến cổng USB Type-C của Raspberry Pi 4. Khi Raspberry Pi 4 khởi động hoàn tất, hệ thống sẽ tự động chạy chương trình giám sát AI đã được cấu hình sẵn.

3. Truy cập giao diện giám sát

Sau khi hệ thống khởi động hoàn tất, người dùng có thể truy cập giao diện giám sát bằng một trong hai cách:

- Nhấp vào liên kết tên miền đã được cấu hình cho hệ thống.
- Mở ứng dụng khách nếu đã được cài đặt trên thiết bị sử dụng.

Sau khi truy cập thành công, giao diện Web Dashboard sẽ được hiển thị và cho phép người dùng theo dõi trạng thái hoạt động của hệ thống.

4. Mở cửa sổ Debug để theo dõi kết quả nhận diện theo từng khung hình

Trong trường hợp cần kiểm tra hoặc đánh giá kết quả nhận diện của mô hình AI trên từng khung hình, người dùng có thể kích hoạt chế độ Debug theo các bước sau:

Bước 1: Dừng chương trình đang chạy dưới dạng dịch vụ:

```
sudo systemctl stop mainapp
```

Bước 2: Mở tệp cấu hình:

```
sudo nano config.py
```

Tệp cấu hình nằm tại đường dẫn:

```
COMPLETE_1/OFFICIAL/config.py
```

Sau đó, kéo xuống cuối tệp và thay đổi giá trị:

```
DEBUG_MODE = False
```

thành:

```
DEBUG_MODE = True
```

Bước 3: Di chuyển đến thư mục chương trình:

```
cd COMPLETE_1/OFFICIAL
```

Bước 4: Chạy chương trình thử công:

```
python3 main_app1.py
```

Sau khi chương trình khởi động, cửa sổ Debug sẽ xuất hiện trên màn hình và hiển thị trực tiếp các khung hình cùng kết quả nhận diện của mô hình AI.

Bước 5: Để dừng chương trình Debug, nhấn tổ hợp phím:

Ctrl + C

Bước 6: Khởi động lại dịch vụ để hệ thống hoạt động tự động như ban đầu:

```
sudo systemctl restart mainapp
```

5. Theo dõi kết quả trên giao diện hệ thống

Sau khi đăng nhập thành công, người dùng có thể theo dõi kết quả hoạt động của hệ thống thông qua các chức năng trên giao diện Web Dashboard:

- **Live View:** Hiển thị video trực tiếp từ khu vực giám sát.
- **Gallery:** Lưu trữ và hiển thị các ảnh chụp khi hệ thống phát hiện sự kiện xâm nhập.
- **Recording:** Lưu trữ các video sự kiện có thời lượng khoảng 3 phút nhằm phục vụ việc xem lại và đối chiếu sau này.
- **Logout:** Cho phép người dùng đăng xuất khỏi hệ thống.

6. Dừng chương trình hoặc khởi động lại hệ thống

Trong trường hợp cần bảo trì hoặc sửa lỗi, người dùng có thể dừng chương trình bằng lệnh:

```
sudo systemctl stop mainapp
```

Nếu cần khởi động lại Raspberry Pi 4, người dùng có thể sử dụng nút Reset được bố trí trên bo mạch hệ thống để thực hiện quá trình khởi động lại thiết bị.

7. Tắt hệ thống

Khi không còn nhu cầu sử dụng, người dùng chỉ cần ngắt nguồn điện 220V AC cấp cho hệ thống. Khi nguồn điện bị ngắt, toàn bộ thiết bị bao gồm Raspberry Pi 4, camera và các khối chức năng liên quan sẽ ngừng hoạt động.